Development of a Web-Based Document Tracking and Records Management System for the Bureau of Local Government Finance – Region II

A Thesis Presented to

the Faculty of the Graduate School

Cagayan State University – Carig Campus

Tuguegarao City

In Partial Fulfillment of the

Academic Requirements for the degree

MASTER OF SCIENCE IN INFORMATION TECHNOLOGY

By:

TOM TOM G. CATRAL

2026

APPROVAL SHEET

In partial fulfilment of the requirements for the degree **MASTER OF SCIENCE IN INFORMATION TECHNOLOGY**, this thesis entitled, "Development of a Web-Based Document Tracking and Records Management System for the Bureau of Local Government Finance – Region II" has been prepared and submitted by **TomTom G. Catral** who is hereby recommended for Oral Examination.

EDISON D. BRAVO, DIT
Adviser

Approved by the Panel of Oral Examination with a grade of _____.

ARTHUR G. IBAÑEZ, ASEAN Engr., PhD
Chairman

ENGR. ROGER P. RUMPON, MS
Member

DENNIS M. BACUYAG, DPA
Member

CHARMIE S. CALVO, DIT
Member

GENERINO P. SIDDAYAO, DIT
Member

_____
Reader

Accepted and approved in partial fulfillment of the requirements for the degree Master of Science in Information Technology.

_____
Date

DENNIS M. BACUYAG, DPA
Dean, Graduate School

ACKNOWLEDGEMENT

The researcher expresses sincere gratitude to the Almighty for the strength and wisdom to undertake this study; to the thesis adviser and panel members for their guidance; to the officials and personnel of the Bureau of Local Government Finance – Region II for sharing their workflow requirements and operational insights; and to family, colleagues, and friends whose encouragement made this work possible.

The researcher also recognizes the value of the open-source community whose technologies made the development of the proposed system feasible. Any remaining errors or limitations are solely the responsibility of the researcher.

DEDICATION

This work is dedicated to my family, mentors, colleagues, and the public servants whose commitment to accountable and efficient records management inspired this study.

TABLE OF CONTENTS

APPROVAL SHEET 2
ACKNOWLEDGEMENT 3
DEDICATION 4
TABLE OF CONTENTS 5
LIST OF TABLES 7
LIST OF FIGURES 8
ABSTRACT 11
CHAPTER I 14
THE PROBLEM AND ITS BACKGROUND 14
1.1 Introduction 14
1.2 Conceptual Framework 16
1.3 Research Paradigm 17
1.4 Objectives of the Study 17
1.4.1 General Objectives 17
1.4.2 Specific Objectives 17
1.5 Scope and Delimitation of the Study 18
1.6 Significance of the Study 19
1.7 Definitions of Terms 20
CHAPTER II 22
REVIEW OF RELATED LITERATURE AND STUDIES 22
2.1 Foreign Literature 22
2.2 Local Literature 28
2.3 Synthesis 34
CHAPTER III 35
RESEARCH METHODOLOGY 35

3.1 General Overview 35
3.2 Locale of the Study 35
3.3 Research Design 35
3.3.1 Agile Development Cycle 36
3.4 System Architecture 37
3.5 Use Case Diagram 39
3.6 Software Development Tools 41
3.6.1 Recommended Hardware and Software Requirements 41
3.7 Participants of the Study 42
3.8 Research Instruments 42
3.9 Data Gathering Procedure 43
3.9.1 Data Analysis 43
3.9.2 Ethical Consideration 45
CHAPTER IV 46
RESULTS AND DISCUSSIONS 46
4.1 Problems Encountered in the Existing System 46
4.2 The Developed System 46
4.2.1 User's View 51
4.2.2 Data Model 80
4.3 The Extent of Compliance of the Developed System to ISO 25010 Software Quality Standards 92
CHAPTER V 98
SUMMARY OF FINDINGS AND RECOMMENDATIONS 99
5.1 Summary of Findings 99
5.2 Conclusions 101
5.3 Recommendations 101

BIBLIOGRAPHY 103

APPENDICES 105

Appendix A. Request Letter for Gathering Data 105

Appendix B. Request for Pilot Testing 106

Appendix C. User's Manual 107

C.1 Accessing the System 107

C.2 Registering a Document 109

C.3 Routing and Acting on a Document 111

C.4 Search, Notifications, QR, and Slips 114

C.5 Administrative Functions 118

C.6 Basic Troubleshooting 120

Appendix D.1 Original ISO Questionnaire Provided by the Adviser (Verbatim) 120

Appendix D.2 Digital Questionnaire Actually Administered 124

Appendix E. Source Code 128

Appendix F. Curriculum Vitae 183

LIST OF TABLES

Table 4.1. Divisions Table Design
Table 4.2. Users Table Design
Table 4.3. Documents Table Design
Table 4.4. Document Routes Table Design
Table 4.5. Document Attachments Table Design
Table 4.6. Audit Logs Table Design
Table 4.7. Envelope Logs Table Design
Table 4.8. Employee Profiles Table Design
Table 4.9. Directory Sections Table Design
Table 4.10. System Settings Table Design
Table 4.11. Notifications Application Store Design

Table 4.12. ISO 25010 Compliance Evaluation Framework

LIST OF FIGURES

Figure 1. Conceptual Framework
Figure 2. Research Paradigm
Figure 3.1. System Architecture
Figure 3.2. Use-Case Diagram of the Implemented WDTMP
Figure 3.3. Context-Level Data Flow Diagram
Figure 3.4. Efficiency Formula
Figure 3.5. Likert Scale for Manual Tracking and WDTMP
Figure 3.6. Likert Scale for ISO 25010 Compliance
Figure 4.1. Hierarchical Input-Process-Output Diagram
Figure 4.2. Input-Process-Output Diagram
Figure 4.3. Entity Relationship Diagram
Figure 4.4. Login And Password Recovery
Figure 4.5. Dashboard
Figure 4.6. Global Document Search
Figure 4.7. Dashboard Document Quick View
Figure 4.8. Notification Drawer
Figure 4.9. Notification Drawer
Figure 4.10. Personnel And Session Directory
Figure 4.11. Account Menu
Figure 4.12. Log New Document Form
Figure 4.13. Log New Document Form
Figure 4.14. Incoming Documents
Figure 4.15. Outgoing Documents
Figure 4.16. Incoming Documents
Figure 4.17. Outgoing Documents
Figure 4.18. Document Details And Routing Flow
Figure 4.19. Document Details Conversation And Attachments
Figure 4.20. Document Details And Recipient Flow
Figure 4.21. Route Forward And Reply Form
Figure 4.22. Routing Follow Up
Figure 4.23. Send Reminder Dialog
Figure 4.24. Document Routing Slip
Figure 4.25. Routing Slip Customization
Figure 4.26. Outgoing Envelope
Figure 4.27. Office Directory
Figure 4.28. Office Directory

Figure 4.29. Directory Title Management
Figure 4.30. Add Personnel Form
Figure 4.31. Qr Code Generator
Figure 4.32. Qr Code Generator
Figure 4.33. Incoming Report
Figure 4.34. Outgoing Report
Figure 4.35. Envelope Report
Figure 4.36. Incoming Report
Figure 4.37. Outgoing Report
Figure 4.38. User Accounts
Figure 4.39. Personnel Accounts Table
Figure 4.40. Create User Account Form
Figure 4.41. Role And Account Permissions
Figure 4.42. Division Management
Figure 4.43. Audit Logs
Figure 4.44. Envelope Dispatch Logs
Figure 4.45. Account Settings
Figure 4.46. Account Settings
Figure 4.47. Simplified Relationship View of the WDTMP Data Model
Figure 4.48. Detailed Physical Entity-Relationship Diagram of the Implemented WDTMP
Figure C.1. User Manual - Login And Password Recovery
Figure C.2. User Manual - Account Menu
Figure C.3. User Manual - Account Settings
Figure C.4. User Manual - Log New Document Form
Figure C.5. User Manual - Incoming Documents
Figure C.6. User Manual - Incoming Documents
Figure C.7. User Manual - Outgoing Documents
Figure C.8. User Manual - Outgoing Documents
Figure C.9. User Manual - Document Details And Recipient Flow
Figure C.10. User Manual - Document Details Conversation And Attachments
Figure C.11. User Manual - Route Forward And Reply Form
Figure C.12. User Manual - Document Details And Routing Flow
Figure C.13. User Manual - Routing Follow Up
Figure C.14. User Manual - Send Reminder Dialog
Figure C.15. User Manual - Dashboard
Figure C.16. User Manual - Dashboard
Figure C.17. User Manual - Global Document Search
Figure C.18. User Manual - Notification Drawer
Figure C.19. User Manual - Qr Code Generator

Figure C.20. User Manual - Document Routing Slip
Figure C.21. User Manual - Routing Slip Customization
Figure C.22. User Manual - Outgoing Envelope
Figure C.23. User Manual - Incoming Report
Figure C.24. User Manual - Outgoing Report
Figure C.25. User Manual - Envelope Report
Figure C.26. User Manual - Incoming Report
Figure C.27. User Manual - Outgoing Report
Figure C.28. User Manual - Office Directory
Figure C.29. User Manual - Add Personnel Form
Figure C.30. User Manual - User Accounts
Figure C.31. User Manual - Role And Account Permissions
Figure C.32. User Manual - Division Management
Figure C.33. User Manual - Audit Logs

ABSTRACT

Title : **Development of a Web-Based Document Tracking and Records Management System for the Bureau of Local Government Finance – Region II**

Name : **Tom-Tom G. Catral**

Course : **Master of Science in Information Technology**

Adviser : **Dr. Edison D. Bravo**

This study presents the design and development of a Web-Based Document Tracking and Records Management System for the Bureau of Local Government Finance – Region II (BLGF Region II). The system responds to operational challenges commonly associated with manual or fragmented document handling, including delayed routing, limited visibility of document status, duplicate route numbers, difficulty locating records, inconsistent accountability, and time-consuming report preparation. The developed application centralizes the registration of incoming and outgoing documents, routing and transfer history, attachments, notifications, employee records, audit logs, outgoing-envelope logs, QR code generation and scanning, document slips, storage configuration, and database export.

The project uses a developmental research approach and an iterative software development process. Its implementation employs Angular 19, Ionic 8 components, and TypeScript 5.8 for the client interface, Express and TypeScript for the application programming interface, and MySQL for persistent storage with a JSON fallback mechanism. Role-based access controls regulate the information and functions available to administrators, records personnel, division chiefs, staff,

and other authorized users. The proposed evaluation adopts the ISO/IEC 25010 product-quality model, focusing on functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability (International Organization for Standardization [ISO], 2011).

The ISO/IEC 25010 evaluation involved 23 respondents composed of 13 BLGF employees and 10 IT experts. The WDTMP obtained a grand mean of 4.97, interpreted as compliance to a Very Great Extent. Maintainability ranked highest with a mean of 5.00, while security obtained the lowest category mean of 4.93 but remained within the Very Great Extent range. The findings indicate that the system provides a traceable document lifecycle, strong maintainability, functional suitability, usability, reliability, portability, performance efficiency, compatibility, and role-based security controls for the intended BLGF Region II workflow.

Keywords: *document tracking, records management, workflow automation, role-based access control, QR code, audit trail, BLGF Region II, ISO/IEC 25010*

CHAPTER I

THE PROBLEM AND ITS BACKGROUND

1.1 Introduction

The digital transformation of public sector institutions is a global imperative, driven by the need for greater operational efficiency, enhanced transparency, and improved citizen service delivery. In the Philippines, the foundational policy for this shift was established with the enactment of the E-Commerce Act of 2000 (Republic Act No. 8792), which aimed to facilitate the use of electronic data and documents in official and commercial transactions. However, despite the passage of the aforementioned law twenty (20) years ago, the widespread adoption and integration of information technology (IT) for core government processes did not progress as envisioned leading to a persistent digital divide within the bureaucracy (Placido, 2020; Information and Communications Technology Office, 2011).

This gap between policy intent and on-the-ground implementation is symptomatic of broader challenges in Philippine e-governance, including limited digital infrastructure, resistance to change, and fragmented IT initiatives (Alampay, 2020). As a result, several key government agencies including the Bureau of Local Government Finance Regional Office No. 2 (BLGF RO2) continue to operate with predominantly manual, paper-based systems for critical internal workflows. The BLGF RO2 as a regional office of the Bureau of Local Government Finance under the Department of Finance plays a crucial role in supervising local treasury operations, ensuring sound local government fiscal management and implementing financial policies. The efficiency and security of its internal document workflows covering memoranda, fiscal reports, policy directives and audit documents are therefore paramount to its regulatory and supervisory functions.

Presently, the bureau relies on a manual document routing and tracking system. This process involves physical document tracking slips, handwritten logbooks and the personal delivery of paper files between offices and personnel. This method is inherently inefficient, prone to errors and vulnerable to security risks. Common operational dilemmas include misplaced or lost documents, significant

delays in retrieval and routing, difficulty in determining real-time document status or location, lack of version control leading to discrepancies and challenges in maintaining an organized audit trail for accountability (Reyes & De Leon, 2019). These inefficiencies not only consume valuable staff time and resources but also create bottlenecks that can delay financial oversight, policy implementation, and service to local government units.

The limitations of manual systems are well-documented in both academic literature and practical case studies. Research consistently shows that paper-based workflows hinder transparency, reduce productivity, and increase administrative overhead (United Nations Department of Economic and Social Affairs, 2022). Conversely, the adoption of electronic document management systems (EDMS) has been linked to significant improvements in process speed, record security, regulatory compliance, and overall organizational agility (Alavi & Leidner, 2001; Sedera et al., 2016).

Consequently, there is a critical and timely need for the BLGF RO2 to transition from its archaic manual system to a modern, integrated digital solution. The development and implementation of a Web-Based Document Tracking and Management Platform (WDTMP) for Secure and Efficient Document Workflow Automation is proposed to directly address these challenges. Such a platform would automate the entire document lifecycle from creation, routing and approval to archiving and retrieval within a secure online environment. With the integration of web technology, the platform would enable role-based access control, ensure data integrity, provide real-time tracking and notifications and maintain a complete, tamper-evident audit log. This digital transformation aligns with the national E-Government Masterplan (EGMP) 2022 and the Philippine Development Plan 2023-2028 which prioritize the digitization of government services to foster a resilient, inclusive and digitally empowered society (National Economic and Development Authority, 2022).

Ultimately, this study seeks to explore the design and potential impact of such a platform for BLGF RO2. It aims to bridge the identified gap in the agency's operational capabilities by providing a framework for a system that enhances document security, streamlines workflow automation, and supports the bureau's

mandate of promoting excellence in local government finance through modern, efficient, and transparent administrative practices.

1.2 Conceptual Framework

This section discusses the main concept and research paradigm of the study.

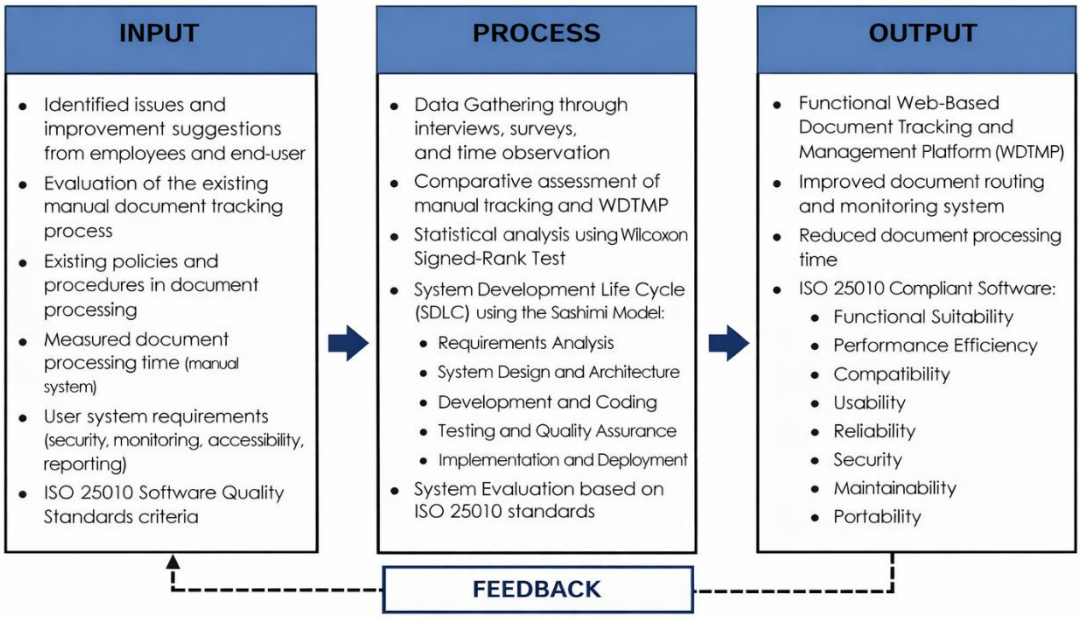


Figure 1. Conceptual Framework

The research paradigm of the study follows the Input–Process–Output (IPO) framework to illustrate how the proposed Web-Based Document Tracking and Management Platform (WDTMP) was conceptualized and developed.

The inputs of the study consist of the identified concerns, suggestions, and observations gathered from participants regarding the existing manual document handling procedures and the initial electronic tracking practices. These include issues related to delays, monitoring difficulties, document misplacement, and reporting inefficiencies. Furthermore, established institutional policies and guidelines on document processing are considered to ensure that the proposed system aligns with standard procedures for proper recording, routing, and monitoring of official documents.

The process involves the adoption of an Agile iterative software development approach. This approach is selected because it allows overlapping phases of the Software Development Life Cycle (SDLC), enabling continuous improvement and early resolution of issues during development. The stages include requirements analysis, system design, development, testing, implementation, and evaluation. Throughout the process, system performance and functionality are assessed to ensure alignment with user needs and organizational standards.

1.3 Research Paradigm

As a result of these inputs and development procedures, a Web-Based Document Tracking and Management Platform (WDTMP) was designed, implemented, and evaluated. The system enhances document monitoring by providing real-time status updates, secure access, and efficient record management. Ultimately, the platform promotes improved operational efficiency, accountability, data security, and user satisfaction.

1.4 Objectives of the Study

1.4.1 General Objectives

This study aims to develop a Web-based Document Tracking and Management Platform for the Regional Bureau of Local Government Finance.

1.4.2 Specific Objectives

Specifically, it answers the following objectives:

1. To identify the issues and problems encountered in the existing document tracking system of the Bureau of Local Government Finance.
2. To design and develop a web-based document tracking and management system tailored for the document workflow requirements of BLGF RO2.
3. To determine the compliance of the developed web-based with ISO 25010 Software Quality Standards along:
   a. Functional Suitability
   b. Performance Efficiency

c. Compatibility

d. Usability

e. Reliability

f. Security

g. Maintainability

h. Portability

1.5 Scope and Delimitation of the Study

The primary focus of this study is the development and implementation of the Web-Based Document Tracking and Management Platform (WDTMP) specifically for the Bureau of Local Government Finance Regional Office No. 2 (BLGF RO2). The scope of the system is centered on automating the internal document lifecycle which includes the digital creation of tracking profiles, real-time routing among various divisions and the monitoring of document status from reception to final archiving. To ensure data integrity and security, the platform incorporates role-based access control and a tamper-evident audit trail. The system allows for the attachment of digital copies in PDF formats with a predefined file size limit to support efficient storage management. Evaluation of the platform is delimited to the ISO 25010 software quality standards focusing on functional suitability, security, and performance efficiency.

Conversely, the study is delimited in several functional areas to maintain project focus. The WDTMP is primarily a tracking and workflow automation tool; therefore, it does not function as a full-scale word processor for document creation nor as a permanent record management system for unalterable historical archives. The system requires an active internet or local network connection for real-time updates and does not support offline data synchronization. Furthermore, while the platform facilitates the routing of documents, it does not replace the physical movement of original hard copies where legal or agency policies mandate wet-ink signatures. Finally, the study focuses solely on the internal administrative workflows of BLGF RO2 and does not extend to the external public-facing portals of local government units.

1.6 Significance of the Study

The development and implementation of a Web-Based Document Tracking and Management Platform (WDTMP) for the Bureau of Local Government Finance Regional Office No. 2 (BLGF RO2) will provide substantial benefits to the following stakeholders:

The Administrators of BLGF RO2. The results of the study will provide managers and supervisors with a clear, real-time overview of document processes within the office. This improved visibility will enable a more improved decision-making process, better resource allocation and the identification of bottlenecks leading to more strategic management of the bureau's document workflow and supporting its mandate of fiscal oversight.

The Employees of BLGF RO2. This study will significantly enhance how personnel track and monitor financial and administrative documents. It will eliminate the difficulties associated with manually locating documents and determining their status thereby streamlining daily workflows, reducing administrative burdens and increasing overall operational productivity.

The Clients (Local Government Units and Stakeholders). The developed web-based platform will facilitate more efficient, transparent and accountable processing of financial documents, reports and communications. This can lead to improved service delivery, faster response time and greater trust in the bureau's regulatory and support functions.

The Researcher. This study enhanced the researcher's ability to design, develop and implement a robust and well-tested software application. It also contributed significantly to the researcher's technical expertise in system analysis, secure web development and workflow automation within the public sector context.

Future Researchers. This study will serve as a reference for future researchers interested in developing electronic document tracking systems or conducting comparative studies on document management and workflow automation, particularly within government financial agencies or similar bureaucratic environments.

1.7 Definitions of Terms

To better understand the context of the study, the following terms are defined operationally.

Audit Trail – refers to the recorded history of all actions performed on a document within the system, including routing, approval, modification, and status updates.

Compatibility – refers to the ability of the WDTMP to operate effectively across different devices, browsers, and network environments used by BLGF RO2.

Document Workflow Automation – refers to the electronic routing, approval, monitoring, and archiving of documents within the WDTMP.

Document Lifecycle – refers to the complete process a document undergoes from creation, routing, approval, storage, retrieval, up to archiving.

Functional Suitability – refers to the degree to which the WDTMP provides appropriate functions that meet the document workflow requirements of BLGF RO2.

Implementation – refers to the deployment and actual use of the WDTMP within BLGF RO2 for document processing and monitoring.

Maintainability – refers to the ease with which the WDTMP can be modified, updated, or enhanced when necessary.

Manual Document Tracking System – refers to the existing paper-based document routing method of BLGF RO2 that uses physical tracking slips, handwritten logbooks, and manual file delivery.

Performance Efficiency – refers to the ability of the WDTMP to process documents within acceptable response time and resource utilization.

Reliability – refers to the capability of the WDTMP to perform its functions consistently without system failure.

Real-Time Tracking – refers to the capability of the system to instantly display the current status and location of a document during processing.

Requirements Analysis – refers to the phase of system development where user needs, workflow procedures, and operational challenges are identified and documented.

Role-Based Access Control (RBAC) – refers to the security feature of the system that restricts access to documents and functions based on assigned user roles and permissions.

Security – refers to the protection mechanisms implemented in the WDTMP to safeguard documents from unauthorized access, alteration, or loss.

System Design – refers to the planning stage of the WDTMP where system architecture, database structure, user interface, and workflow processes are structured.

System Development – refers to the coding and actual creation of the WDTMP based on identified requirements and design specifications.

System Testing – refers to the evaluation process conducted to verify whether the WDTMP functions correctly and meets user requirements before full implementation.

Usability – refers to the ease with which employees of BLGF RO2 can learn, understand, and use the WDTMP.

Web-Based Document Tracking and Management Platform (WDTMP) – refers to the online system developed in this study to automate, monitor, secure, and manage document workflows within BLGF RO2.

CHAPTER II

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter presents a synthesis of relevant literature including foreign and local sources on topics directly related to the development of a web-based document tracking and management platform designed to secure and automate document workflows.

2.1 Foreign Literature

The origins of formalized document management can be traced to the late 19th century. In 1898, Edwin Grenville Seibels patented the vertical filing cabinet, a system designed to organize loose papers in drawers within a freestanding cabinet (Grenville, 1902, as cited in Thornton, 2018). This invention marked the transition from piled desktops and bound ledgers to a searchable, categorized physical system. For decades, this model dominated offices worldwide. However, its limitations became increasingly apparent: substantial spatial demands, labor-intensive retrieval processes, and vulnerability to physical damage, loss, or deterioration (Allen, 2019).

The paradigm began to shift with the dawn of the computing age. The 1980s introduced the first significant wave of digitization. The development of affordable microcomputers and optical character recognition (OCR) scanners enabled the conversion of paper records into digital images. Early systems, often called Document Image Management (DIM), focused primarily on storage and retrieval of scanned documents. However, they were largely standalone, with minimal workflow capabilities and major challenges in version control when documents needed editing (Bannon, 1991). The true precursor to modern Electronic Document Management Systems (EDMS) emerged with the integration of document imaging with database technologies, allowing for metadata tagging and basic search functions.

A critical evolution occurred in the mid-1990s to early 2000s with the rise of the internet and client-server architecture. Systems evolved from simple image

repositories to comprehensive Electronic Document Management Systems (EDMS) and Enterprise Content Management (ECM) platforms. These systems incorporated workflow engines, audit trails, and permission-based access controls. The introduction of standards like the World Wide Web Consortium's (W3C) recommendations for web accessibility and data interchange further propelled development, allowing for more interoperable systems (Berners-Lee, 1999). The term "document" expanded beyond text and images to encompass emails, web pages, and multimedia files.

The 2010s heralded the current era of cloud-based and intelligent document management. The proliferation of Software-as-a-Service (SaaS) models made powerful EDMS accessible without significant on-premise infrastructure. This period saw the convergence of document management with Business Process Management (BPM), giving birth to sophisticated workflow automation platforms. Furthermore, the integration of Artificial Intelligence (AI) and Machine Learning (ML) introduced capabilities like automatic data extraction, smart categorization, and predictive routing, transforming passive repositories into active, intelligent components of organizational infrastructure (Davenport & Ronanki, 2018).

Today, the historical trajectory points toward context-aware, secure, and highly automated platforms. Modern systems are expected to ensure compliance with stringent data protection regulations (e.g., GDPR, Data Privacy Act of 2012), provide seamless mobile and remote access, and offer deep analytics on process performance. The history of document tracking is no longer just about managing files but about orchestrating knowledge, securing information assets, and automating the very pathways of organizational decision-making and service delivery.

The purpose of this study is to design and implement a Web-Based Document Tracking and Management Platform to fundamentally transform the manual and paper-intensive document workflows of the Bureau of Local Government Finance (BLGF) Regional Office No. 2. This system aims to modernize the agency's internal processes by replacing outdated, time-consuming

procedures with a secure, automated, and intelligent digital ecosystem. In addition to this, WDTMP have the following benefits:

1. Data Integrity and Security – Tracking your documents provides security, especially if your office frequently deals with important or confidential social welfare information. You will have complete control over: who accesses a document and can see exactly who has viewed it; who has made changes and when those changes were made; and whether the document was sent to anyone.
2. Workflow Productivity – Tracking your documents allows you to see edits and progress in real-time, which can help you keep your team on track and on the same page. It eliminates the manual effort of following up on document status, allowing staff to focus on frontline service delivery.
3. Automated Audit Trails – Document tracking makes it incredibly easy for your organization to create an audit trail by automatically keeping track of all changes made to each document, complete with timestamps. This ensures that every action is recorded and verifiable, simplifying the work of internal auditors.
4. Universal Accessibility – Many document tracking systems enable access to your documents from anywhere and from any device, allowing you and your employees to make changes or provide approvals anytime. This is particularly vital for regional operations that require coordination between different provincial and municipal offices.
5. Low Environmental Footprint – A document tracking system helps your company go paperless, which benefits both you and the environment by eliminating your mounds of hard copies. This supports the government's push for sustainable, green administration.
6. System Interoperability – A document tracking service enables you to integrate with other productivity-boosting applications. This enables a seamless transfer of information across all facets of your organization, ensuring that financial data, case files, and administrative records are synchronized.

Furthermore, a Web-Based Document Tracking and Management Platform for Secure and Efficient Document Workflow Automation not only helps the user to store and find information quickly, but also categorizes that information to make sure the right people can access specific documents at the right time (Monday, 2021). This prevents unauthorized access while ensuring data is available to authorized decision-makers instantly.

Along with the tendencies of many document workers in losing track of document paths, it is vital that these systems are able to trace the movement of documents from origin to destination/s (UP Information Technology Development Center, 2022). This ensures that critical social service documents are never "lost in the system" and can be located at any stage of the process.

The development of the Web-Based Document Tracking and Management Platform represents a strategic intervention with multidimensional impact, significantly contributing to both organizational efficiency and broader sustainable development objectives. Aligned with the United Nations' 2030 Agenda, this initiative directly supports multiple Sustainable Development Goals (SDGs) by transforming manual workflows into secure, automated processes that enhance institutional performance while promoting sustainable practices (United Nations Department of Economic and Social Affairs, 2023). The platform's impact manifests most prominently in its support of SDG 8: Decent Work and Economic Growth through substantial productivity gains. By automating document routing, approval, and tracking, the system reallocates human resources from administrative tasks to value-added analytical work, fostering a more efficient and intellectually engaging work environment that supports sustainable economic practices (Giang et al., 2022).

Furthermore, the platform's paperless architecture significantly advances SDG 12: Responsible Consumption and Production by eliminating physical document storage and reducing paper consumption, thereby minimizing the environmental footprint of government operations (Hilty et al., 2022). This transition to digital workflows not only reduces resource use but also aligns with circular economy principles by decreasing waste generation across the

document lifecycle. The platform's robust security framework, incorporating encryption and comprehensive audit trails, reinforces SDG 16: Peace, Justice and Strong Institutions by ensuring data integrity and fostering transparency in public financial management (Kshetri, 2022). This enhanced accountability builds public trust and strengthens governance mechanisms essential for sustainable development. Additionally, by enabling efficient information exchange and collaboration with local government units and development partners, the platform supports SDG 17: Partnerships for the Goals, demonstrating how digital tools can strengthen multi-stakeholder cooperation and resource mobilization for development objectives (United Nations Development Programme, 2021). Collectively, these impacts position the platform not merely as a technological upgrade but as a catalyst for institutional transformation that aligns local governance improvements with global sustainability commitments, creating a replicable model for digital transformation across the Philippine public sector.

The development of the Web-Based Document Tracking and Management Platform represents a strategic intervention with multidimensional impact, significantly contributing to both organizational efficiency and broader sustainable development objectives. Aligned with the United Nations' 2030 Agenda, this initiative directly supports multiple Sustainable Development Goals (SDGs) by transforming manual workflows into secure, automated processes that enhance institutional performance while promoting sustainable practices (United Nations Department of Economic and Social Affairs, 2023). The platform's impact manifests most prominently in its support of SDG 8: Decent Work and Economic Growth through substantial productivity gains. By automating document routing, approval, and tracking, the system reallocates human resources from administrative tasks to value-added analytical work, fostering a more efficient and intellectually engaging work environment that supports sustainable economic practices (Giang et al., 2022).

Furthermore, the platform's paperless architecture significantly advances SDG 12: Responsible Consumption and Production by eliminating physical document storage and reducing paper consumption, thereby minimizing the environmental footprint of government operations (Hilty et al., 2022). This

transition to digital workflows not only reduces resource use but also aligns with circular economy principles by decreasing waste generation across the document lifecycle. The platform's robust security framework, incorporating encryption and comprehensive audit trails, reinforces SDG 16: Peace, Justice and Strong Institutions by ensuring data integrity and fostering transparency in public financial management (Kshetri, 2022). This enhanced accountability builds public trust and strengthens governance mechanisms essential for sustainable development. Additionally, by enabling efficient information exchange and collaboration with local government units and development partners, the platform supports SDG 17: Partnerships for the Goals, demonstrating how digital tools can strengthen multi-stakeholder cooperation and resource mobilization for development objectives (United Nations Development Programme, 2021). Collectively, these impacts position the platform not merely as a technological upgrade but as a catalyst for institutional transformation that aligns local governance improvements with global sustainability commitments, creating a replicable model for digital transformation across the Philippine public sector.

Challenges in Implementing a Web-based Document Management System

The implementation of a web-based document management platform faces significant technical, organizational, and human-centric challenges, particularly within government contexts. A primary technical challenge involves overcoming system interoperability and legacy integration issues, as new platforms must often interface with outdated proprietary systems not designed for modern API-based communication, creating data silos and migration complexities (Mergel et al., 2019). Additionally, ensuring robust data security and privacy compliance presents a formidable hurdle, requiring the platform to meet stringent national regulations such as the Data Privacy Act of 2012 while protecting sensitive information from evolving cyber threats (Kshetri, 2022).

Organizational resistance to change constitutes another major barrier, as employees accustomed to manual, paper-based workflows often exhibit technological aversion and digital skill gaps. This resistance can stem from a lack of familiarity, fear of job displacement, or distrust in new systems, which

hinders user adoption and undermines the platform's potential impact (Venkatesh & Bala, 2022). Furthermore, sustained leadership commitment and adequate budgetary allocation are critical yet frequently unstable, as digital transformation projects compete for limited government funding and may lose priority amid shifting political agendas (Cordella & Bonina, 2022).

Infrastructure limitations, particularly in regional offices, pose a foundational challenge. Inconsistent internet connectivity and inadequate hardware can severely restrict the functionality and reliability of a cloud-based platform, exacerbating the digital divide between central and local government units (Misuraca & Viscusi, 2021). Finally, establishing standardized processes and metadata protocols across diverse departments remains a persistent difficulty, as inconsistent filing conventions and approval hierarchies can lead to disorganized digital repositories, undermining the system's searchability and auditability (Janssen et al., 2020). These multifaceted challenges necessitate a holistic implementation strategy that addresses technical, human, and institutional dimensions simultaneously.

2.2 Local Literature

International implementations of Web-Based Document Tracking and Management Platforms demonstrate effective strategies for secure and efficient workflow automation. In 2020, Lee and Park conducted a study of the Singapore Ministry of Finance's Digital Document Hub, which implemented a cloud-based platform using role-based workflow automation and end-to-end encryption. The system reduced document processing time by 68% and achieved 99.5% accuracy in automated routing of financial documents, validated through the Technology Acceptance Model (TAM) across departmental users (Lee & Park, 2020).

In the academic sector, Thompson and Rodriguez (2022) studied the University of Cambridge Document Workflow Platform, which automates research grant documentation and administrative approvals. The system implements automated version control, electronic signature integration, and real-time collaboration features, reducing administrative overhead by 45% while improving compliance with research funding requirements. Their implementation

approach emphasized phased deployment and extensive user training as critical success factors.

These international implementations collectively demonstrate that successful Web-Based Document Tracking and Management Platforms share common characteristics: cloud-based architecture for accessibility, configurable workflow automation for efficiency, multi-layered security protocols for data protection, and user-centered design for adoption. The documented outcomes—including significant reductions in processing time, improved accuracy, and enhanced security—provide a compelling evidence base for developing a similar platform for Philippine government agencies, particularly in adapting these proven approaches to local operational contexts and regulatory requirements.

Several Philippine academic institutions have pioneered the development of electronic document tracking systems, providing valuable insights for the creation of a secure and efficient workflow automation platform. The University of the Philippines Information Technology Development Center (UP ITDC) developed a comprehensive Document Tracking System that monitors the complete document lifecycle within UP offices (UP ITDC, 2022). This system captures metadata including originating and receiving offices, personnel involved, time elapsed between departments, and supports document attachments, revisions, updates, and remarks. The implementation demonstrated how structured workflow documentation enhances institutional accountability and process transparency in academic administration.

In a notable implementation at the Southern Philippines Agri-Business and Marine and Aquatic School of Technology (SPAMAST), Rellon et al. (2020) created a web-based document tracking system utilizing barcode technology with SMS notification capabilities. This system managed diverse document types within the Office of Records through automated status updates, significantly reducing document search time and improving response rates to document requests. Their approach demonstrated how relatively low-cost technologies can be effectively deployed to streamline document management in resource-constrained educational environments.

At Isabela State University, Echague Campus, Constantino et al. (2020) implemented an "Electronic Documents Location Tracking System" that facilitated real-time monitoring of document movement across different administrative offices. System evaluation revealed high effectiveness in streamlining document tracking processes, recording document locations, and facilitating inter-office transfers. The study emphasized user-friendly interface design and intuitive workflow navigation as critical factors for successful adoption, with users reporting significant reductions in time spent locating documents and tracking approval statuses.

These local implementations collectively demonstrate several critical success factors for document tracking systems in Philippine institutional contexts: user-centered design principles, gradual implementation strategies, appropriate technology selection based on institutional resources, and comprehensive user training programs. The documented improvements in operational efficiency, document security, and process transparency provide compelling evidence for the potential benefits of implementing a similar Web-Based Document Tracking and Management Platform within government agencies like the BLGF, while also highlighting the importance of adapting implementation approaches to local organizational cultures and technological infrastructures.

Testing of the Efficiency of the Web-Based Document Tracking and Management Platform

The effectiveness of a Web-Based Document Tracking and Management Platform for Secure and Efficient Document Workflow Automation must be quantitatively and qualitatively assessed to validate its operational improvements. This evaluation employs statistical methods and performance metrics to measure gains in processing efficiency, security robustness, and user productivity compared to legacy manual systems.

Statistical validation of efficiency improvements utilizes the Wilcoxon Signed Rank Test, a non-parametric statistical method appropriate for analyzing paired, non-normally distributed data (McDonald, 2021). This test compares performance metrics—such as document processing time, retrieval speed, and error rates—

between the traditional manual system and the new web-based platform. For instance, a study by Garcia and Lim (2022) applied this methodology in assessing a government document automation system, finding a statistically significant median reduction of 58% in approval cycle times, thereby providing empirical evidence of enhanced workflow efficiency.

A time-motion analysis framework is essential for capturing detailed efficiency metrics. This approach, as implemented in the evaluation of the National Archives of Singapore's digital records system, involves systematically recording the duration of each workflow stage—document creation, routing, review, approval, and archiving—under both manual and automated conditions (Tan & Wong, 2023). This granular analysis identifies specific bottlenecks alleviated by automation and provides verifiable data on time savings, directly demonstrating the platform's contribution to efficient document workflow automation.

Evaluating the platform's security posture is integral to the efficiency assessment, as robust security should not impede performance. A framework proposed by the Information Systems Audit and Control Association (ISACA, 2022) recommends measuring the security-efficiency ratio, which balances the strength of controls (e.g., encryption, access logging) against their processing overhead. For example, an audit of a provincial treasury's document system in Malaysia found that implementing AES-256 encryption added negligible latency (under 0.3 seconds per transaction) while achieving full compliance with data protection standards, proving that security and efficiency are synergistic in a well-designed platform (Abdullah & Chen, 2023).

The web-based architecture's performance under load is another critical test. Stress and scalability testing, following benchmarks set by the International Organization for Standardization's ISO/IEC 25010 standards for software quality, assess system responsiveness, concurrent user handling, and uptime (ISO, 2021). A case study of a regional government platform in Taiwan demonstrated 99.95% availability while processing over 5,000 simultaneous document transactions, confirming that a cloud-native, web-based design is fundamental to reliable and efficient document workflow automation (Lin et al., 2022).

Finally, user-centric efficiency metrics are gathered through surveys based on the Technology Acceptance Model (TAM). Research by Santos and Ocampo (2023) on a Philippine local government unit's document system implementation showed a direct correlation between perceived ease of use, perceived usefulness, and actual reductions in task completion time. This qualitative data complements quantitative metrics, providing a holistic view of the platform's impact on operational efficiency.

In conclusion, a multi-method evaluation strategy—combining rigorous statistical testing, detailed time analysis, security auditing, system performance benchmarking, and user feedback—is necessary to conclusively prove that the Web-Based Document Tracking and Management Platform delivers on its promise of secure and efficient document workflow automation. This evidence-based validation is crucial for stakeholder buy-in and successful adoption within the Bureau of Local Government Finance and similar agencies.

ISO 25010 Software Quality Standards for Testing Compliance of the Web-Based Document Tracking and Management Platform

The development and validation of a Web-Based Document Tracking and Management Platform for Secure and Efficient Document Workflow Automation must adhere to established international standards to ensure its quality, reliability, and fitness for purpose. The ISO/IEC 25010 standard provides a comprehensive quality model that serves as the foundational framework for evaluating the system's characteristics against explicit and implicit stakeholder requirements (International Organization for Standardization [ISO], 2011). This model is pivotal for determining the specific quality attributes that define a superior software product, ensuring the final platform delivers tangible value to the Bureau of Local Government Finance (BLGF) and its clients.

The ISO 25010 model is structured into two primary dimensions: Software Product Quality and Software Quality in Use (QinU). The Product Quality dimension evaluates inherent properties of the software itself, while the Quality in Use dimension assesses the effectiveness, efficiency, and satisfaction experienced by users during real-world interaction with the system in a specific context (Atoum et

al., 2018). This dual perspective is critical for a platform whose success hinges not only on technical robustness but also on seamless adoption within a government workflow environment.

For this specific platform, the Product Quality characteristics take on targeted significance. Functional Suitability is paramount, requiring the system to provide the exact functions necessary for secure document routing, version control, and approval automation. Performance Efficiency must be rigorously tested to ensure the web-based platform handles concurrent users and large document volumes without degradation, directly supporting the goal of efficient document workflow automation. Security, as a standalone characteristic in the ISO 25010 model, is non-negotiable; the platform must demonstrate capabilities in confidentiality, integrity, non-repudiation, accountability, and authenticity to protect sensitive fiscal data (Sánchez-Gordón & Colomo-Palacios, 2020). Furthermore, Usability and Reliability are essential to ensure the system is learnable, operable, and available when needed by government personnel.

Equally important is the Quality in Use dimension. This evaluates outcomes such as Effectiveness (can users accurately and completely achieve their document tracking goals?), Efficiency (can they do so with appropriate expenditure of resources and time?), and Satisfaction (is the user experience free from discomfort and positive in attitude?). Testing against these QinU metrics, as applied in a study of e-government services by Alharbi and de Doncker (2019), ensures the platform transcends technical functionality to become a genuinely useful tool that enhances, rather than hinders, public sector productivity.

Applying this ISO 25010 framework to the evaluation of related international and local systems reveals a common thread: successful implementations are those that score highly across these interconnected quality characteristics. The documented dilemmas in traditional document management often stem from deficits in one or more of these areas—be it poor performance efficiency leading to delays, inadequate security causing data vulnerability, or low usability resulting in low adoption. Therefore, using the ISO 25010 standard as a compliance checklist and testing guide during the platform's development lifecycle is not merely an academic exercise. It is a strategic imperative to ensure the final Web-Based

Document Tracking and Management Platform is engineered to a high standard of quality, capable of delivering the secure and efficient document workflow automation required to modernize government operations effectively and sustainably.

2.3 Synthesis

The reviewed concepts converge on five requirements: reliable document identification, explicit workflow states, controlled access, traceable actions, and usable retrieval. Records-management principles explain what must be preserved; workflow systems explain how tasks move; security models govern who may act; QR technology connects physical and digital records; and ISO/IEC 25010 supplies an evaluation structure.

The proposed system integrates these requirements in a single application tailored to BLGF Region II. Its contribution is not the isolated use of a database or web interface, but the combination of registration, role-based routing, status history, attachments, alerts, storage, employee records, dispatch logs, QR-assisted lookup, and audit functions around the office's operational document lifecycle.

CHAPTER III
RESEARCH METHODOLOGY

This chapter outlines the systematic approach used to develop and evaluate the WDTMP. It details the software development life cycle, the research design for performance testing, and the methods for data collection.

3.1 General Overview

This chapter presents the methodology used to develop and evaluate the Web-Based Document Tracking and Management Platform. It describes the locale, research design, Agile development cycle, system architecture, use-case and data-flow models, development tools and requirements, participants, research instruments, data-gathering procedure, data analysis, and ethical safeguards applied in the study.

3.2 Locale of the Study

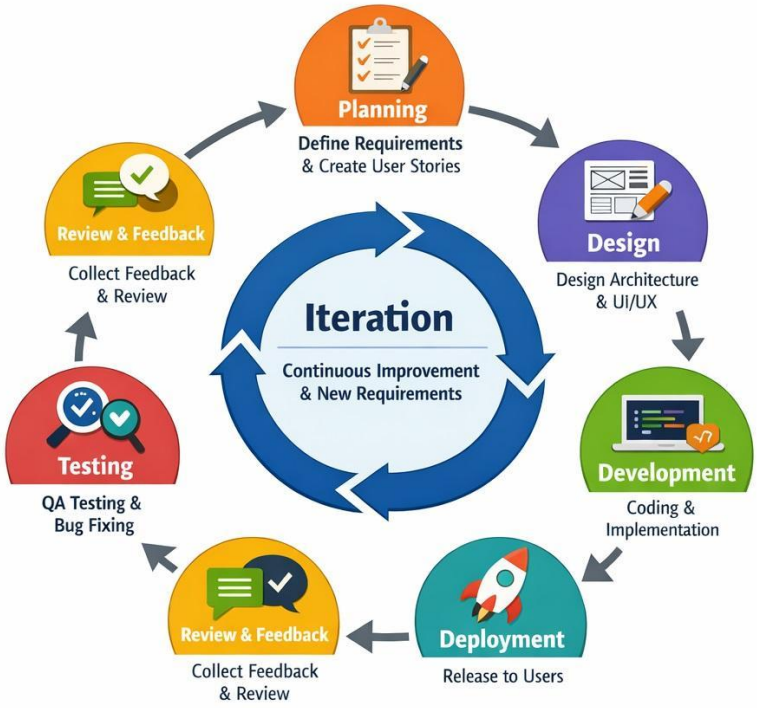
The study will be conducted at the Bureau of Local Government Finance Regional Office II (BLGF RO2). The respondents answered an online questionnaire for both the pre-test and post-test, which also evaluated the WDTMP based on ISO/IEC 25010 Software Quality Standards. The data gathering, system development, testing, and implementation were conducted from January to March 2026.

3.3 Research Design

The researcher utilized a dual-method approach consisting of Descriptive and Developmental Research Designs to systematically address the document management challenges at BLGF RO2. The descriptive component was employed to investigate the current operational landscape focusing on the respondents' perspectives, experiences and the specific problems encountered within the manual document tracking system. According to McCombes (2020), descriptive research is designed to accurately and systematically describe a population,

situation, or phenomenon, which in this study involved using surveys and interviews to identify bottlenecks and security risks in the existing workflow. This baseline data provided the necessary justification for the transition to a digital platform.

For the technical creation of the system, the study applied a Developmental Research Design centered on the Systems Development Life Cycle (SDLC). As noted by Banks (2021), the SDLC provides the essential phases for creating software components that integrate into a functional, holistic system. Specifically, the researcher adopted the Sashimi Model, an overlapping SDLC approach in which requirements analysis, design, development, testing, implementation, and evaluation may proceed with controlled feedback between adjacent phases. This supported early correction of routing, access-control, attachment-visibility, notification, reporting, and audit requirements discovered during iterative validation.



Agile Development Cycle

3.3.1 Agile Development Cycle

The development activities followed an iterative Agile cycle in which requirements analysis, design, implementation, testing, evaluation, and refinement

were completed through controlled increments. Feedback was used to correct workflow, access-control, routing, notification, reporting, and usability concerns before succeeding iterations.

3.4 System Architecture

The Web-Based Document Tracking and Management Platform (WDTMP) utilizes a multi-tier architecture designed to decouple the user interface, application logic, and data storage layers. This structural approach ensures the system's scalability, high performance, and continuous availability, which are critical for the regulatory and supervisory functions of the BLGF RO2. By separating the architecture into distinct tiers—the Presentation Tier (Web Browser), the Logic Tier (Server-side processing), and the Data Tier (Database)—the platform achieves enhanced security because direct access to the database from the client-side is restricted. The Presentation Tier provides the interface for staff to access tracking profiles, while the Logic Tier, supported by DNS resolution and caching, manages the workflow rules and routing. Finally, the Data Tier handles the persistent storage of metadata and audit trails, creating an architecture that facilitates easier maintenance and allows for future upgrades without disrupting the entire document workflow.

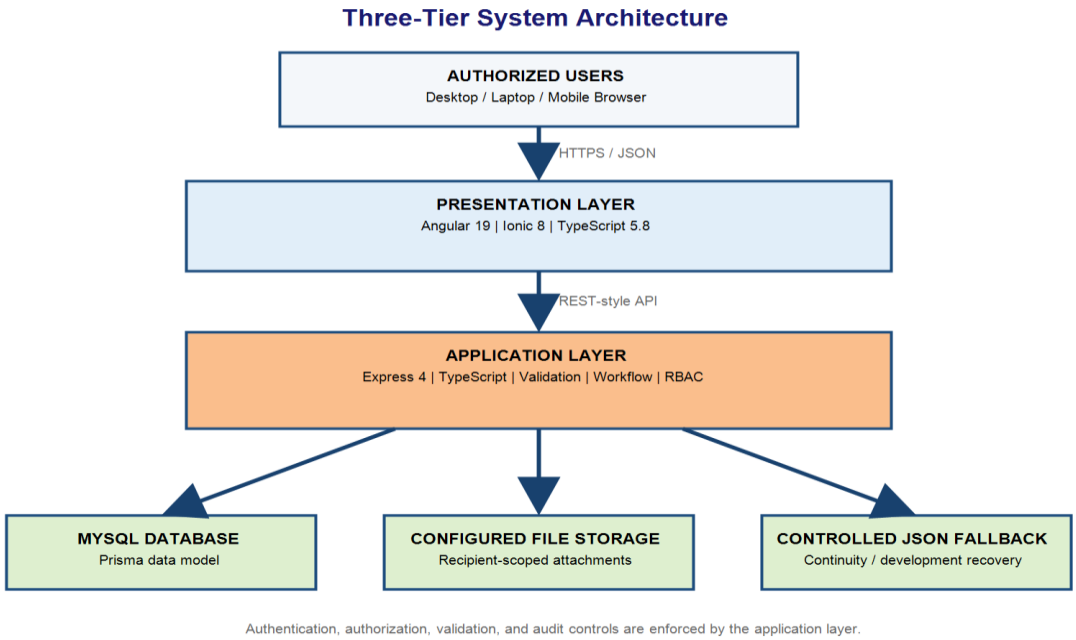


Figure 2. System Architecture

The Web-Based Document Tracking and Management Platform (WDTMP) utilizes a multi-tier architecture designed to decouple the user interface, application logic, and data storage layers. This structural approach ensures the system's scalability, high performance and continuous availability which are critical for the regulatory and supervisory functions of the BLGF RO2. By separating the architecture into distinct tiers, the Presentation Tier (Web Browser), the Logic Tier (Server-side processing) and the Data Tier (Database - the platform achieves enhanced security because direct access to the database from the client-side is restricted. The Presentation Tier provides the interface for staff to access tracking profiles, while the Logic Tier, supported by DNS resolution and caching, manages the workflow rules and routing. Finally, the Data Tier handles the persistent storage of metadata and audit trails, creating an architecture that facilitates easier maintenance and allows for future upgrades without disrupting the entire document workflow.

3.5 Use Case Diagram

The simplified use-case diagram identifies three practical actor groups and separates administrator monitoring/configuration from operational document actions. The System Administrator can monitor all transactions and manage accounts, permissions, divisions, settings, reports, reminders, and logs, but does not perform operational approval, routing, reply, or completion. Records or originating users register documents and begin or continue authorized routes. Recipients and other authorized users receive, decide, reply, attach scoped files, complete assigned branches, and provide final instructions.

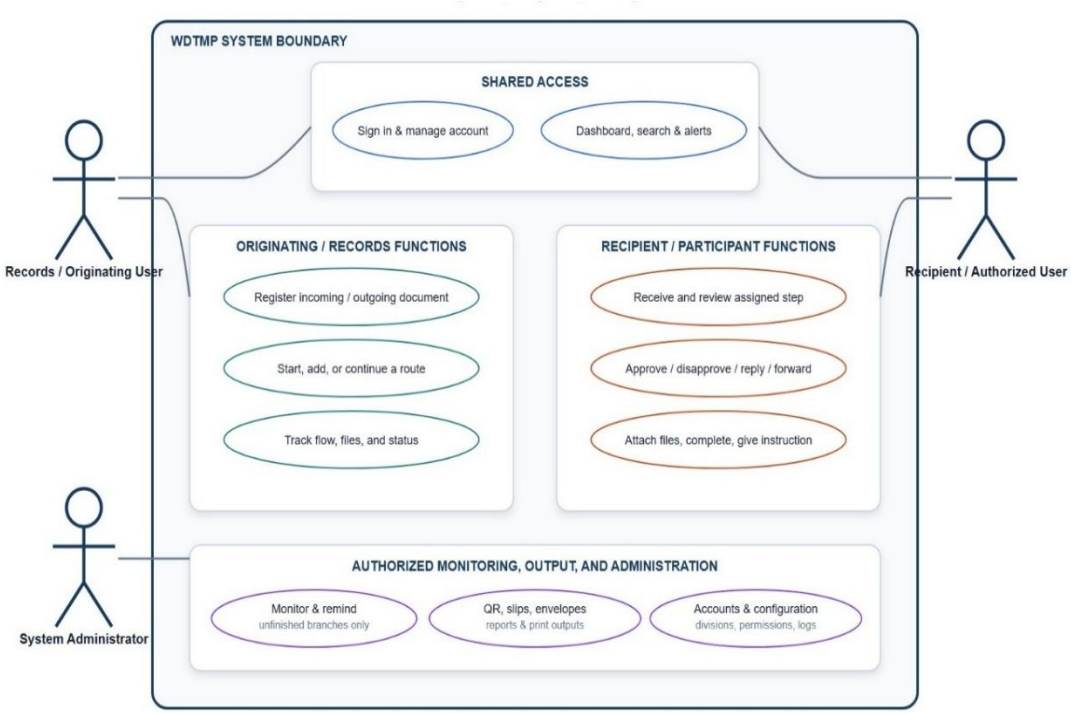


Figure 3. Use-Case Diagram of the Implemented WDTMP

All actors use authentication and account controls according to permission. Document registration distinguishes incoming and outgoing transactions, including BLGF employee and external recipients. The workflow covers tracking, route and reply, recipient-scoped conversation and attachments, completion and final instruction, reminders, continued routing, QR generation, routing slips, envelope printing, and transaction reports.

Context-Level Data Flow Diagram

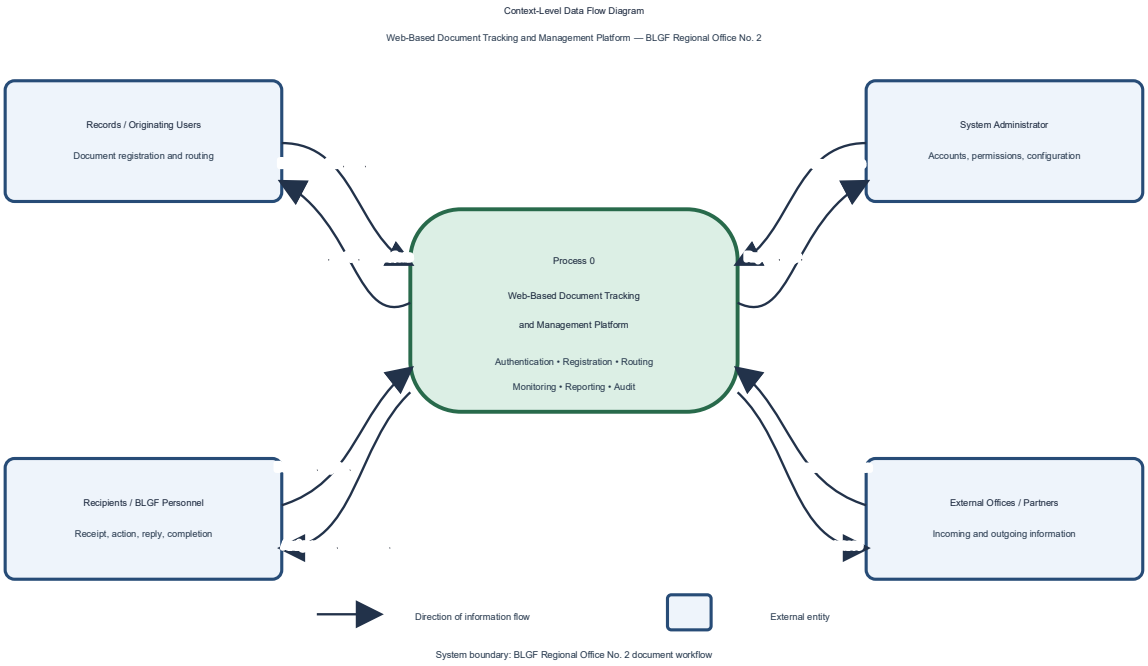


Figure 4. Context-Level Data Flow Diagram

Figure 4 presents the Web-Based Document Tracking and Management Platform as one central process and identifies the external entities that exchange information with it. The diagram defines the boundary of the WDTMP and shows the direction of each major input and output without exposing internal implementation details.

Explanation of Data Flows

1. Records or Originating Users send document metadata, electronic files, intended recipients, and routing instructions to the WDTMP. The platform returns the permanent route number, current document status, notifications, and traceable transaction history.

2. Recipients and authorized BLGF personnel receive assignments, requested actions, and instructions. They return receipt information, decisions, replies, scoped attachments, completion details, and final instructions.

3. The System Administrator provides user accounts, roles, permissions, divisions, and configuration settings. The WDTMP provides organization-wide monitoring information, system reports, alerts, and audit logs while preserving the separation between administrative monitoring and operational document action.

4. External offices and partner agencies provide incoming document and sender information. They receive outgoing dispatch information and document references through the authorized BLGF workflow. Arrowheads indicate the direction in which each information group moves across the system boundary.

3.6 Software Development Tools

Computing Scheme: Intranet-based

Server Requirement Specifications:

- Backend Platform: Node.js with Express and TypeScript
- Frontend Framework: Angular 19 with Ionic 8 and TypeScript 5.8
- Database: MySQL accessed through the Prisma data model; JSON fallback is retained only for controlled continuity and development recovery
- Deployment Scheme: BLGF-authorized intranet or secured web-server environment

Training/Development Environment:

- IDE/Code Editor: Visual Studio Code or equivalent
- Runtime Environment: Node.js (Latest LTS Version)
- Package Manager: npm (Node Package Manager)
- Development Tools: Angular CLI, npm, Prisma CLI, Git, and browser developer tools

3.6.1 Recommended Hardware and Software Requirements

Client Hardware Requirement Specifications:

- **Operating System:**
  - Windows 10 and above
  - macOS (latest versions)
  - Android / iOS (for mobile deployment)
- **Processor:**
  - Dual-core processor (Intel/AMD) or higher
- **Memory (RAM):**

- Minimum: 4 GB
- Recommended: 8 GB or higher

- **Storage:**
  - At least 1 GB free space (for application and dependencies)
- **Internet Connection:**
  - Required for authenticated client-server communication, in-app notifications, file transfer, reporting, and synchronized document status updates
- **Supported Browsers (Latest Versions):**
  - Google Chrome
  - Mozilla Firefox
  - Microsoft Edge

3.7 Participants of the Study

The researcher utilized a total enumeration sampling method, wherein all employees of the Bureau of Local Government Finance Regional Office II (BLGF RO2) who are directly involved in document processing were included as primary respondents. These employees are considered "document processors" due to the nature of their roles. Furthermore, 10 IT experts were engaged to evaluate the system, particularly in assessing its compliance with ISO/IEC 25010 Software Quality Standards.

Positions/Role	Frequency	Percentage
Information Technology Expert	10	43.48%
Administrative Assistant	1	4.35%
Administrative Officer	3	13.04%
Financial Analyst	3	13.04%
Local Treasury Operations Officer	3	13.04%
Local Assessment Operations Officer	3	13.04%
TOTAL	**23**	**100%**

3.8 Research Instruments

The following methods were used to gather relevant data and information for the study:

A. **Survey Form** – The researcher utilized a digital questionnaire distributed through Google Forms. This method allowed the researcher to collect data, observe the current system, and identify common issues, transactions, and user concerns. It also helped in understanding the requirements needed for the improvement and development of the system.

B. **Interview** – The researcher conducted interviews by asking questions relevant to the study. This approach was used to clarify important details and gather additional information necessary for the development and design of the proposed system.

The original questionnaire supplied for the study is reproduced verbatim in Appendix D.1. For digital administration, the indicators were implemented in Google Forms as the questionnaire shown in Appendix D.2. The statistical results reported in Chapter IV were computed from the 23 responses exported from that administered digital form.

3.9 Data Gathering Procedure

At the outset of the data gathering, the researcher sought permission from the Regional Director through a formal letter requesting approval to conduct the study (see Appendix A). Upon approval of the request, the researcher distributed the questionnaires in digital form to the respondents. Furthermore, phone and face-to-face interviews were conducted to elicit the respondents' insights regarding the challenges encountered and their recommendations concerning both the manual tracking system and the WDTMP.

3.9.1 Data Analysis

The data gathered from the previous section were tallied and organized for proper analysis and interpretation using SPSS statistical software version 29.0.1.0 (171). The researcher used the Wilcoxon Signed-Rank Test to determine if there is a significant difference between manual tracking and the developed WDTMP. According to Bobbitt, the Wilcoxon Signed-Rank Test is a non-parametric

alternative to the paired t-test and is used to compare two related samples. Since the data collected are at the ordinal level, this test was used instead of the paired t-test.

Additionally, frequency count and mean will be used to analyze and interpret the challenges and recommendations in the use of both the manual system and the WDTMP. The mean was also used to assess the level of compliance of the WDTMP with ISO/IEC 25010.

In determining the efficiency between manual tracking and the WDTMP, the researcher adapted the formula used by Dizon et al. (2017):

$$\textbf{Efficiency of the WDTMP} = \frac{\textbf{Manual} - \textbf{Proposed}}{\textbf{Proposed}} \times 100\%$$

The researcher also recorded the total turnaround time for document processing in both systems.

A Likert scale was used to measure the responses of the participants in both the pre-test and post-test. The five-point scale had corresponding descriptive equivalents, which were used to evaluate the level of compliance with ISO/IEC 25010.

Figure 3.5. Likert Scale for the assessment of usage between manual tracking and WDTMP.

Scale	Range	Descriptive Interpretation
5	4.21-5.00	Strongly Agree
4	3.41-4.20	Agree
3	2.61 - 3.40	Neither Agree or Disagree
2	1.81-2.60	Disagree
1	1.00-1.80	Strongly Disagree

Figure 3.6. Likert Scale for the extent of compliance of WDTMP to the ISO 25010.

Scale	Range	Descriptive Interpretation
5	4.21-5.00	Very Great Extent

4	3.41-4.20	Great Extent
3	2.61 - 3.40	Moderate Extent
2	1.81-2.60	Little Extent
1	1.00-1.80	No Extent

3.9.2 Ethical Consideration

Permission to conduct the study and pilot testing was sought from the appropriate BLGF Regional Office II authority. Participation is voluntary, respondents are informed of the study purpose, and responses are reported in aggregate. Unnecessary personal identifiers are not collected, test activities use non-sensitive or authorized sample records, and access to research data is limited to authorized persons in accordance with confidentiality, responsible research practice, and the Data Privacy Act of 2012.

CHAPTER IV
RESULTS AND DISCUSSIONS

This chapter presents the identified workflow problems, the developed system, its implemented data structure, and the framework used to evaluate compliance with ISO/IEC 25010.

4.1 Problems Encountered in the Existing System

The requirements addressed by the system indicate recurring risks in a manual or fragmented workflow:

a. difficulty locating a document after endorsement
b. dependence on personal follow-up
c. inconsistent route-number recording
d. duplicate entries
e. limited visibility of pending actions and target dates
f. attachments stored separately from transaction records
g. incomplete chain-of-custody information
h. delayed preparation of logs or reports.

These conditions may result in unnecessary processing time, uncertainty regarding responsibility, weak auditability, and difficulty retrieving historical records. They also make continuity dependent on individual knowledge instead of shared and controlled organizational information.

4.2 The Developed System

The developed WDTMP is an Angular/Ionic and Express application with a MySQL database. It manages authentication, document registration, routing, status monitoring, scoped attachments, notifications, reports, QR lookup, printing, and audit history according to user permissions.

Document Registration and Tracking

Incoming registration records the sender and originating office. Outgoing registration distinguishes a BLGF employee recipient selected from the Office Directory from an outside recipient whose identity, position, office, and address are entered manually. A Register Only transaction appears immediately in Recent Document Activity and the proper Incoming or Outgoing list with Not Yet Routed status. An issued route number is never reused, even when a transaction is later removed.

Routing, Conversation, Files, and Completion

Every recipient is represented by an independent routing branch with aligned status milestones. Dispatch notes are shown only for the intended recipient. Step Conversation and Attachments records sender, recipient, time, instruction or reply, decision, and permitted route-scoped files in chronological order. End - Transaction Completed appears only under the branch actually completed. A completed branch retains its final instruction and history, while an authorized originator may add another route when further endorsement is required.

Monitoring, Notifications, Reports, and Physical Outputs

Dashboard cards, global search, Incoming and Outgoing lists, Recent Document Activity, QR lookup, and Routing Follow-up provide current status visibility. Authorized monitors may send an editable in-app reminder, which does not approve or complete a document. Incoming, Outgoing, Completed-status, and Envelope reports use live records and expandable route history. Routing slips, QR codes, standard envelopes, and FROM/TO cut-outs may be previewed and printed from stored data.

Administration, Directory, Installation, and Accountability

The System Administrator monitors all transactions and manages users, roles, granular permissions, divisions, directory titles, personnel, system settings, audit logs, and envelope logs, while operational routing and decisions remain with authorized document users. The Office Directory supports BLGF and external contacts and controlled employee folders. The account menu provides progressive-web-application installation guidance for supported desktop and mobile browsers. Audit and envelope logs retain actor, role, action, route reference, details, date/time, and network address.

Role Boundary

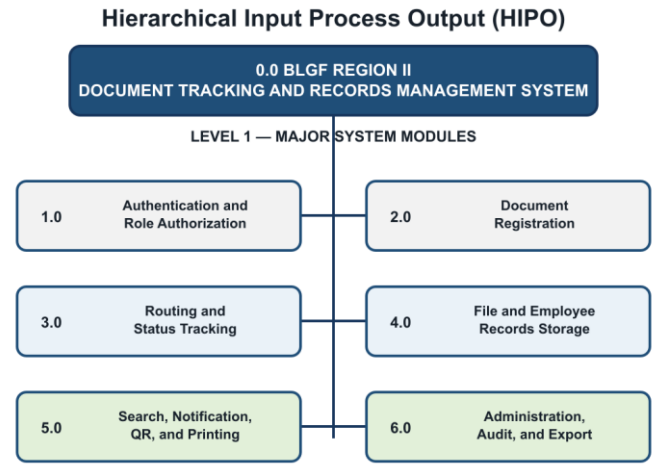
A visible transaction does not automatically grant an action. System Administrator access is organization-wide for monitoring and authorized correction; operational

users act only on branches and commands granted to their account; least-privilege users see only related documents, recipient-specific instructions, scoped attachments, and permitted menus. This separation protects confidential files and prevents monitoring functions from being mistaken for routing authority.

Hierarchical Input Process Output (HIPO)

Figure 6 presents the Hierarchical Input Process Output (HIPO) model of the developed system. At Level 0, the complete BLGF Region II Document Tracking and Records Management System acts as the parent function. Level 1 decomposes the system into six major modules that correspond to the active functions implemented in the frontend and backend: authentication and role authorization; document registration; routing and status tracking; file and employee-record storage; search, notification, QR, and printing; and administration, audit, and export.

The lower portion of the figure provides an IPO summary for every Level 1 module. Each row identifies the data accepted by a module, the principal validation or transformation performed by the application, and the resulting record, view, history entry, notification, file, or administrative output. This structure shows both the functional hierarchy and the flow of information without treating unsupported legacy schema tables as active system modules.



INPUT–PROCESS–OUTPUT SUMMARY BY MODULE

INPUT	PROCESS	OUTPUT
1. Credentials and assigned user role	Validate login; apply role permissions	Authorized session and permitted views
2. Metadata, route number, attachments	Validate uniqueness; create and store record	Registered incoming or outgoing document
3. Document, recipient, action, and remarks	Route or transfer; update status, history, and alert	Updated route history, status, and notification
4. Employee data, folders, and files	Validate file; store data; update file metadata	Organized employee and document file records
5. Query, filter, document selection	Search/filter; generate QR code or routing slip	Search results, QR, dashboard, printed slip
6. Users, settings, logs, and database	Manage configuration; audit and export data	Updated accounts/settings, logs, and export file

Figure 4.1. Hierarchical Input-Process-Output Diagram

Input Process Output (IPO) Diagram

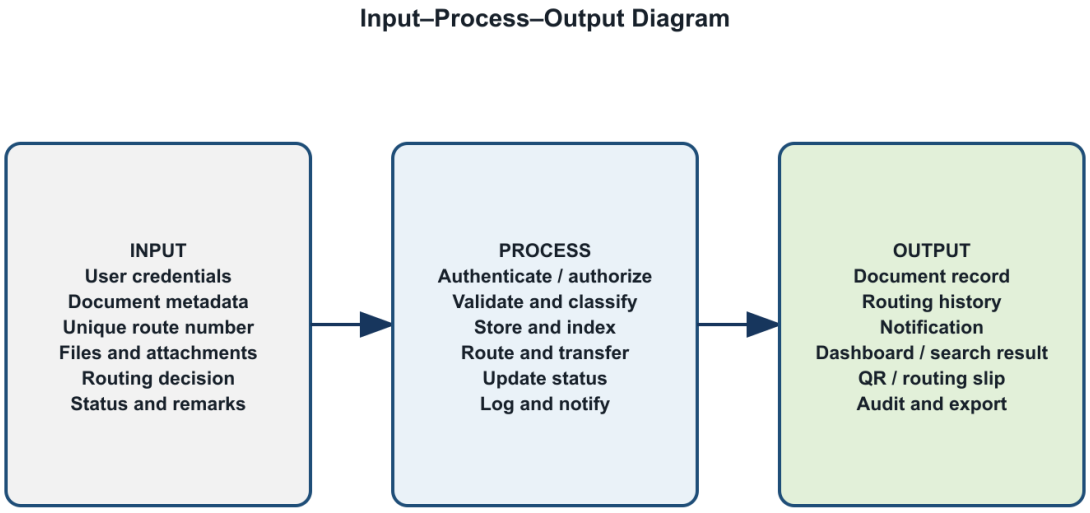


Figure 4.2. Input-Process-Output Diagram

Entity Relationship Diagram (ERD)

Figure 8 presents the active data relationships used by the developed system. The core workflow centers on the documents entity, which stores the unique tracking number, direction, subject and classification data, current status, current division, assignment, and relevant dates. A document may have many routing steps and attachments. Notifications may refer to a document and are assigned to a user. Users are grouped by division and their significant actions are recorded in audit logs.

Employee records form a related but distinct branch: one employee profile may contain many folders, and one folder may contain many files. Solid relationship lines identify foreign keys explicitly enforced in the current MySQL schema. Dashed lines identify application-level logical references such as division_code, user_id, and notification document_id, which are used by the application but are not declared as foreign-key constraints in the schema. This distinction prevents the diagram from claiming database constraints that do not exist.

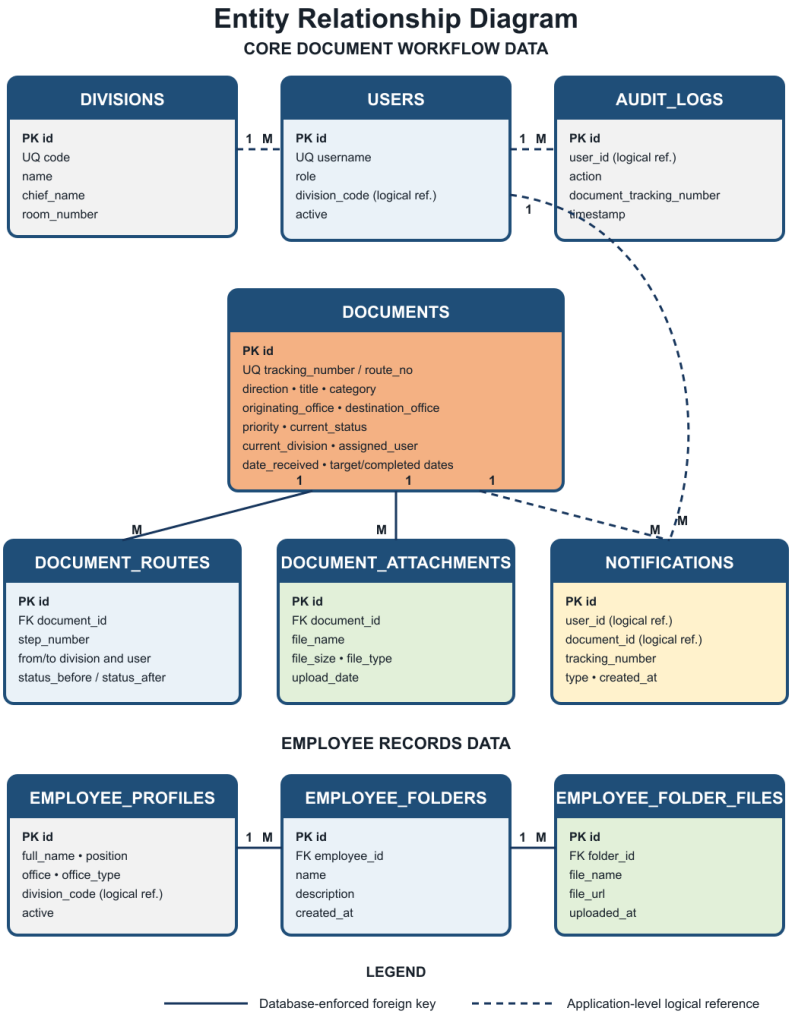


Figure 4.3. Entity Relationship Diagram

4.2.1 User's View

This section documents the actual interface captured from the running Angular/Express system connected to the active MySQL database on October 7, 2026. Following the presentation pattern used by Duque, every module is presented through: (a) Data Environment, (b) Data Entry, and (c) Data Description. Screen availability is controlled by the authenticated account role and granular permissions.

User's View 1: Authentication and Password Recovery Module

a. Data Environment

Primary data: users. The account ID uniquely identifies the user; username is unique; active and permissions determine whether authentication succeeds and which functions are available.

b. Data Entry

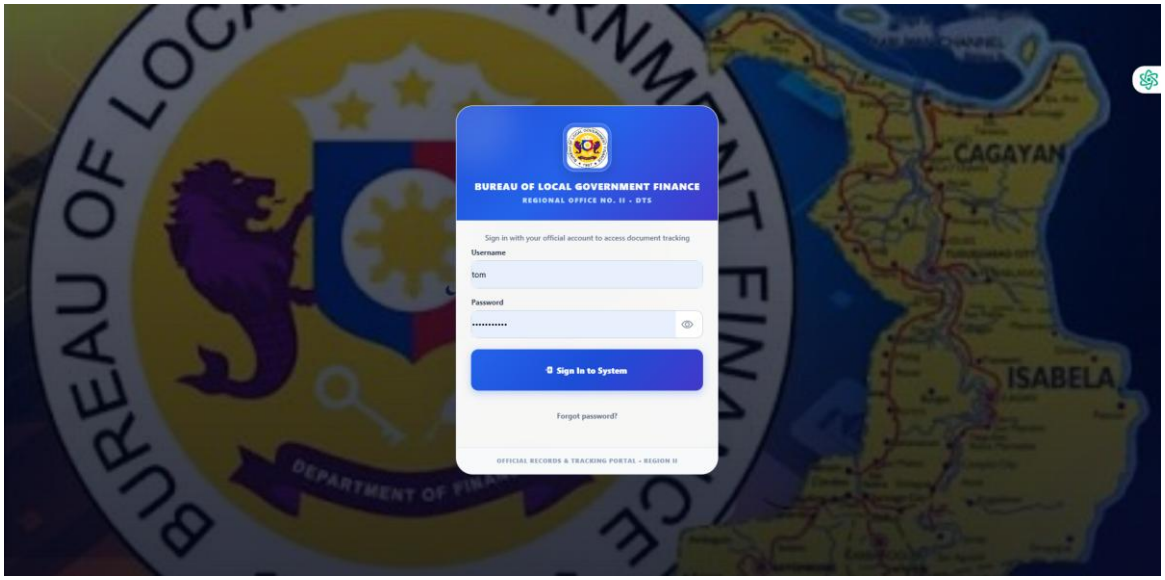


Figure 4.4. Login And Password Recovery

Procedure:

Open the login page; enter the registered username and password; select Log In; for recovery, select Forgot Password, submit the registered account identifier, validate the temporary credential, and create a new password.

c. Data Description

The authentication module validates active user credentials before protected records are loaded. It also supports controlled password recovery and temporary-password replacement. Successful authentication restores only the menus, documents, notifications, and actions allowed by the saved account permissions.

User's View 2: Dashboard and Global Retrieval Module

a. Data Environment

Primary data: users, documents, and document routes. Dashboard counts and search results are computed only from documents related to or visible to the authenticated account.

b. Data Entry

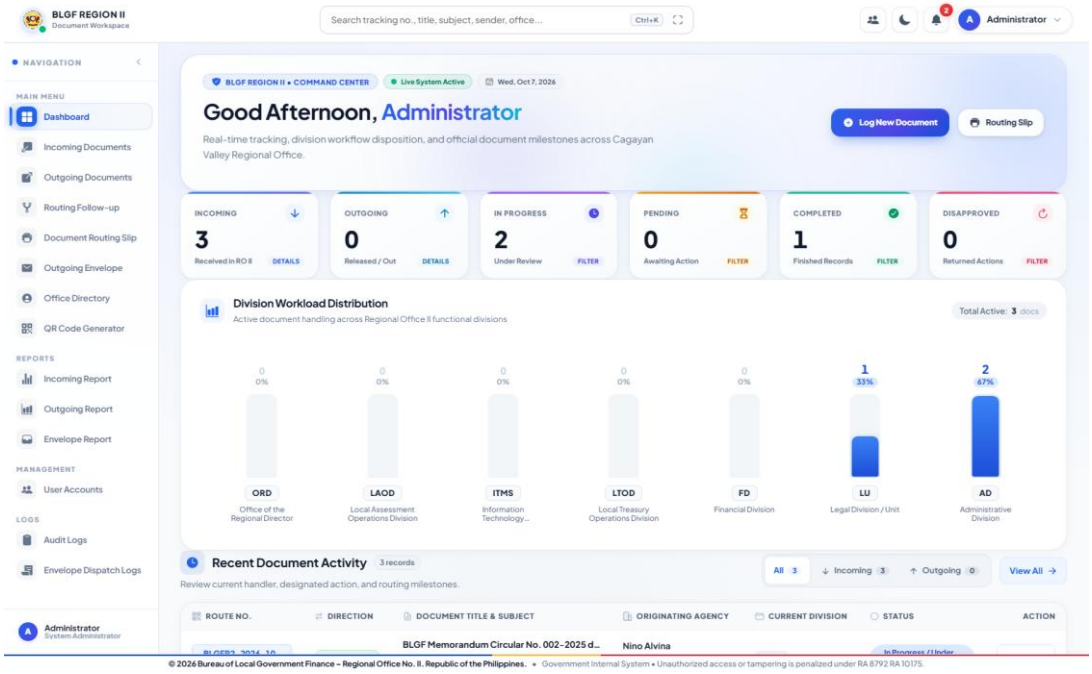


Figure 4.5. Dashboard

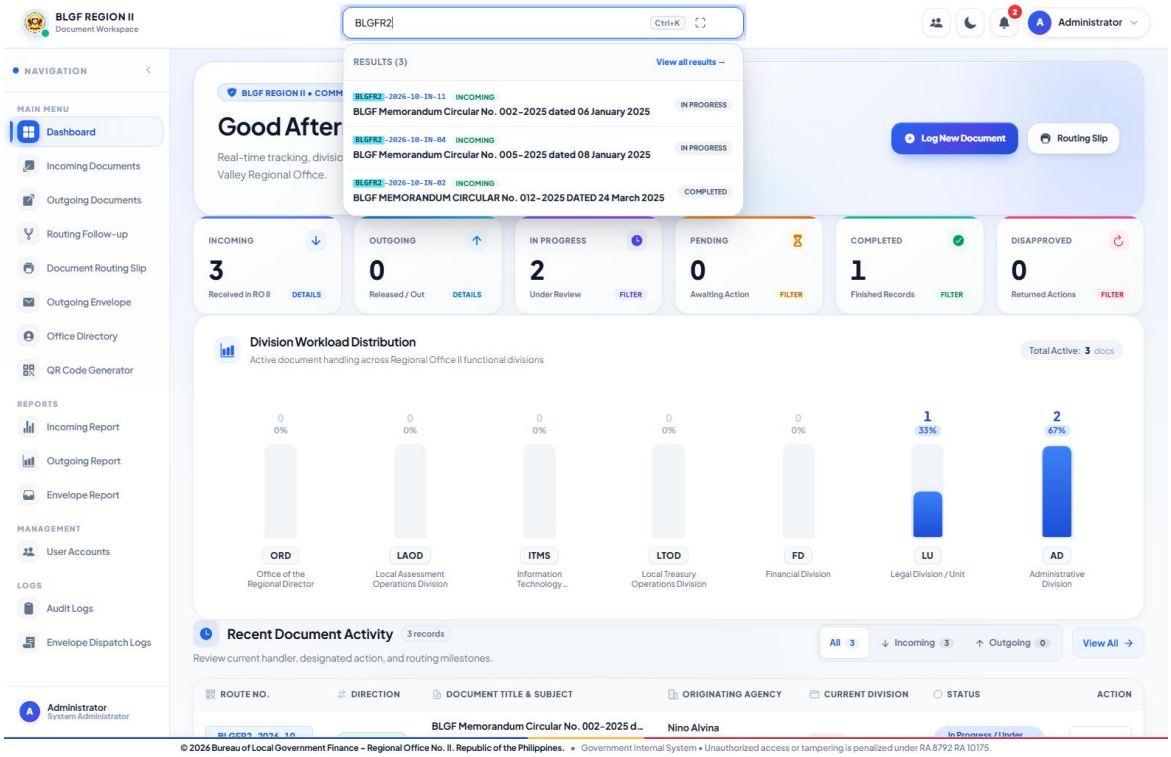


Figure 4.6. Global Document Search

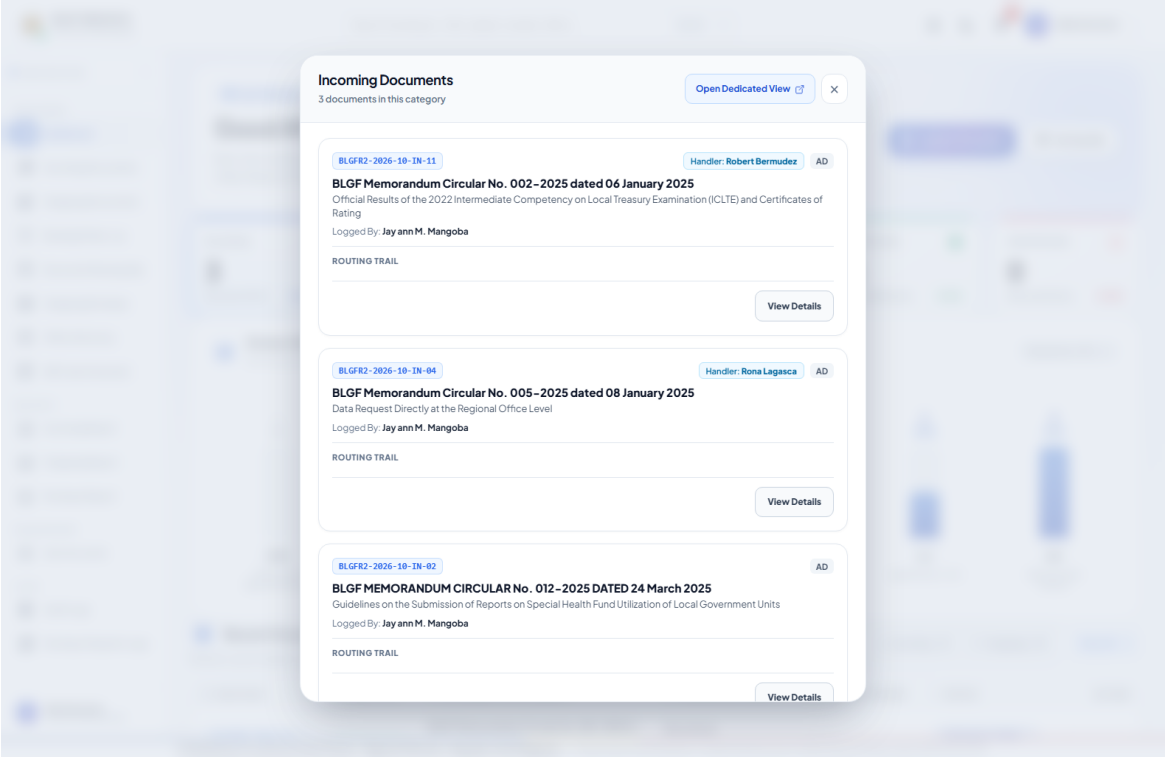


Figure 4.7. Dashboard Document Quick View

Procedure:

Sign in with an authorized account; review the dashboard counters and recent activities; use global search or a quick-view command; select the matching route number to open its transaction record.

c. Data Description

The dashboard displays access-aware document totals, current workload, status summaries, recent activity, quick registration, and document retrieval. Global search returns permitted documents by route number, title, subject, sender, or office. The Administrator receives organization-wide monitoring information, while a least-privilege user receives only related transactions.

User's View 3: Notifications, Personnel Sessions, and Account Menu Module

a. Data Environment

Primary data: users and derived notifications. Notifications are computed from document and route activity for the intended recipient; they are not a separate approval record.

b. Data Entry

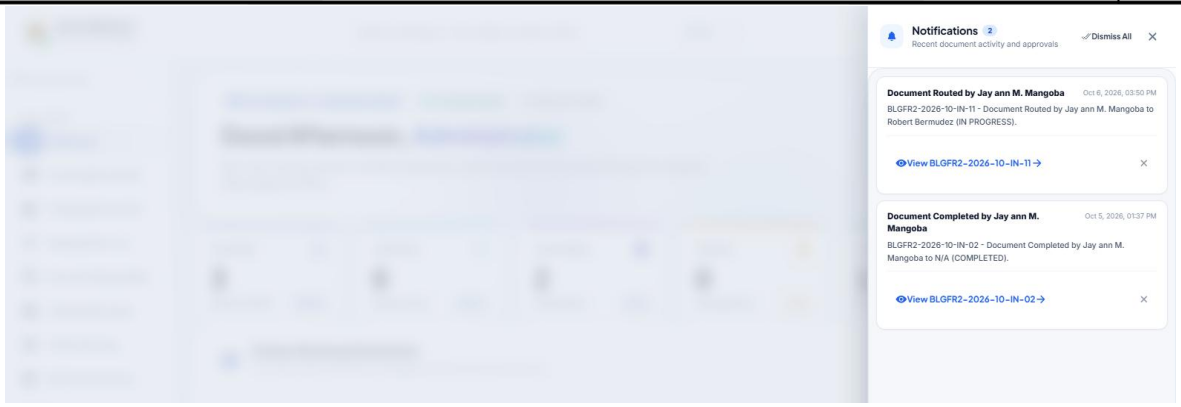


Figure 4.8. Notification Drawer



Figure 4.9. Notification Drawer

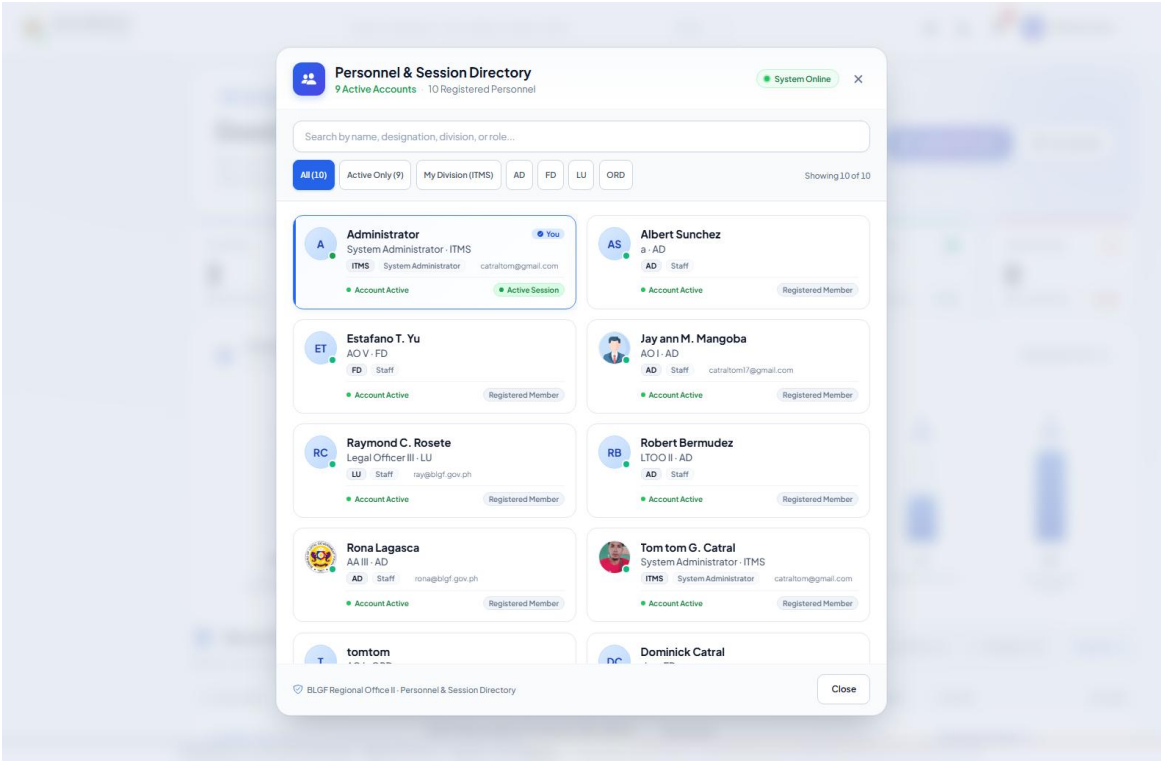


Figure 4.10. Personnel And Session Directory

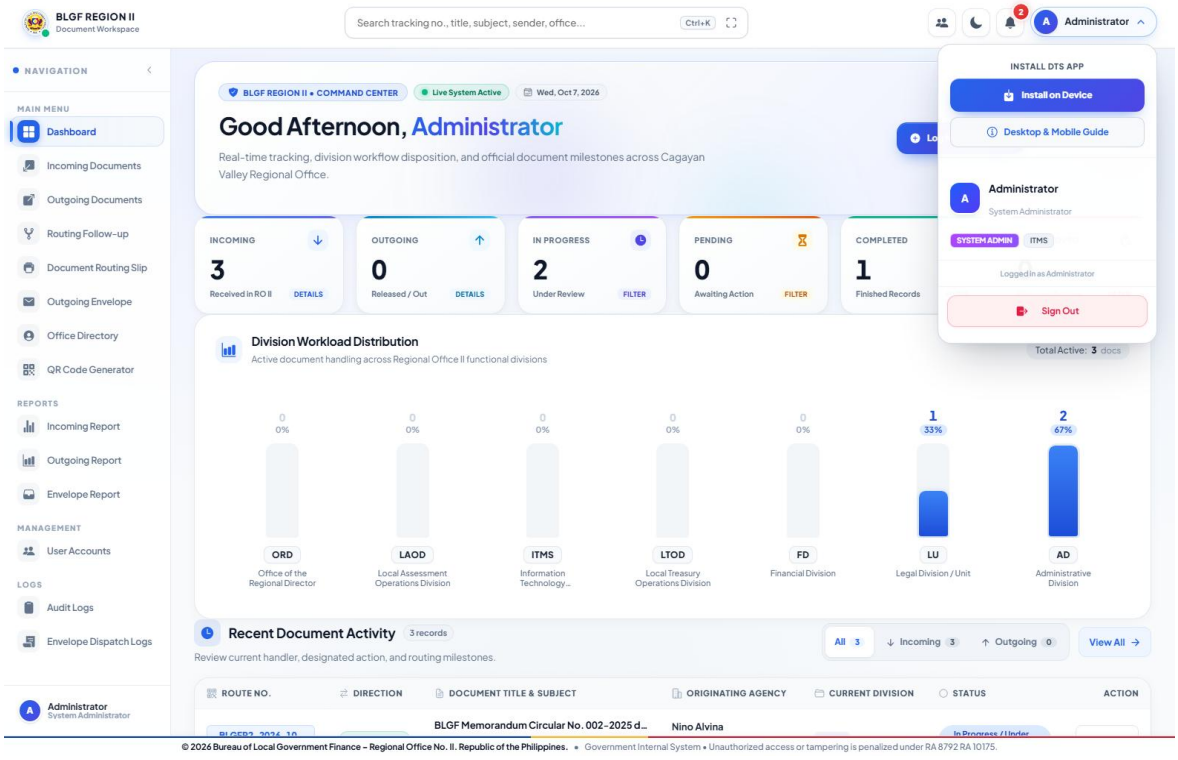


Figure 4.11. Account Menu

Procedure:

Open the notification icon to review routed-document and reminder alerts; select an alert to inspect the permitted record; use the personnel/session directory only for identity and availability reference; use the account menu to sign out safely.

c. Data Description

Notifications identify routed documents and reminder messages for the intended user. The Personnel and Session Directory shows registered personnel, division, role, account state, and the current session without granting document access. The account menu provides sign-out and progressive-web-application installation guidance.

User's View 4: Document Registration Module

a. Data Environment

Primary data: documents and document_attachments. One document receives one permanent tracking number and may own many files. Register Only creates no route; Register and Route creates recipient route records.

b. Data Entry

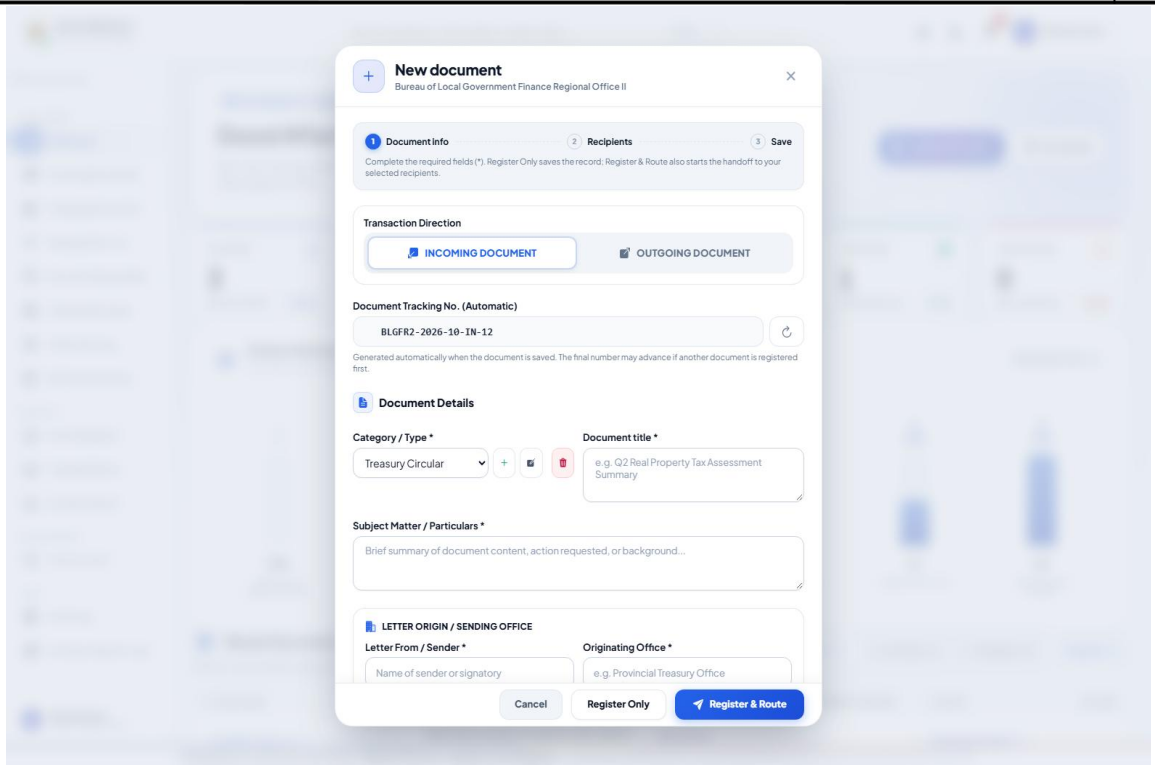


Figure 4.12. Log New Document Form

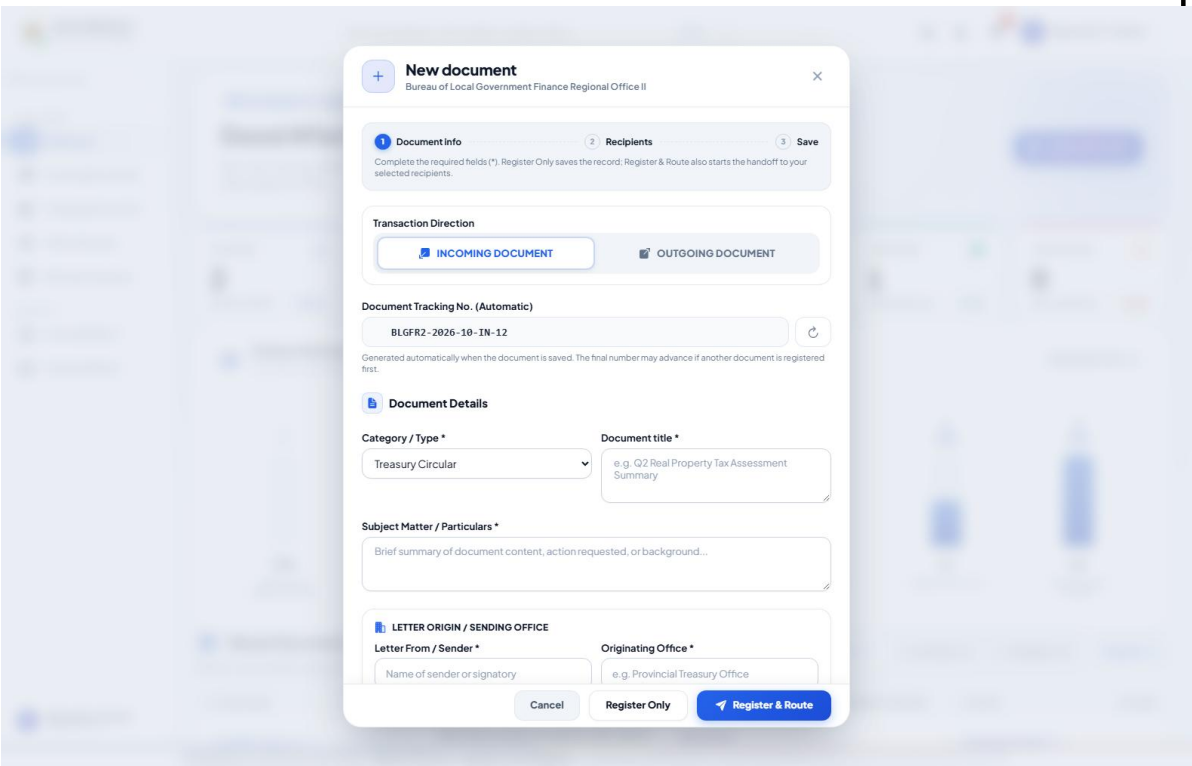


Figure 4.13. Log New Document Form

Procedure:

Select Log New Document; choose Incoming or Outgoing; complete the required metadata; add recipients, target date, action, note, and permitted PDF files; select Register Only or Register and Route; verify the issued route number.

c. Data Description

The registration form records direction, automatically generated route number, category, title, subject, sender, office, address, division, recipients, requested action, priority, target date, files, and initial instruction. Register Only creates a visible Not Yet Routed record. Register and Route creates the record and its first recipient branches. Issued route-number sequences are not reused.

User's View 5: Incoming and Outgoing Monitoring Module

a. Data Environment

Primary data: documents, users, and document_routes. Direction separates Incoming and Outgoing lists; current assignment and route participation control ordinary-user visibility.

b. Data Entry

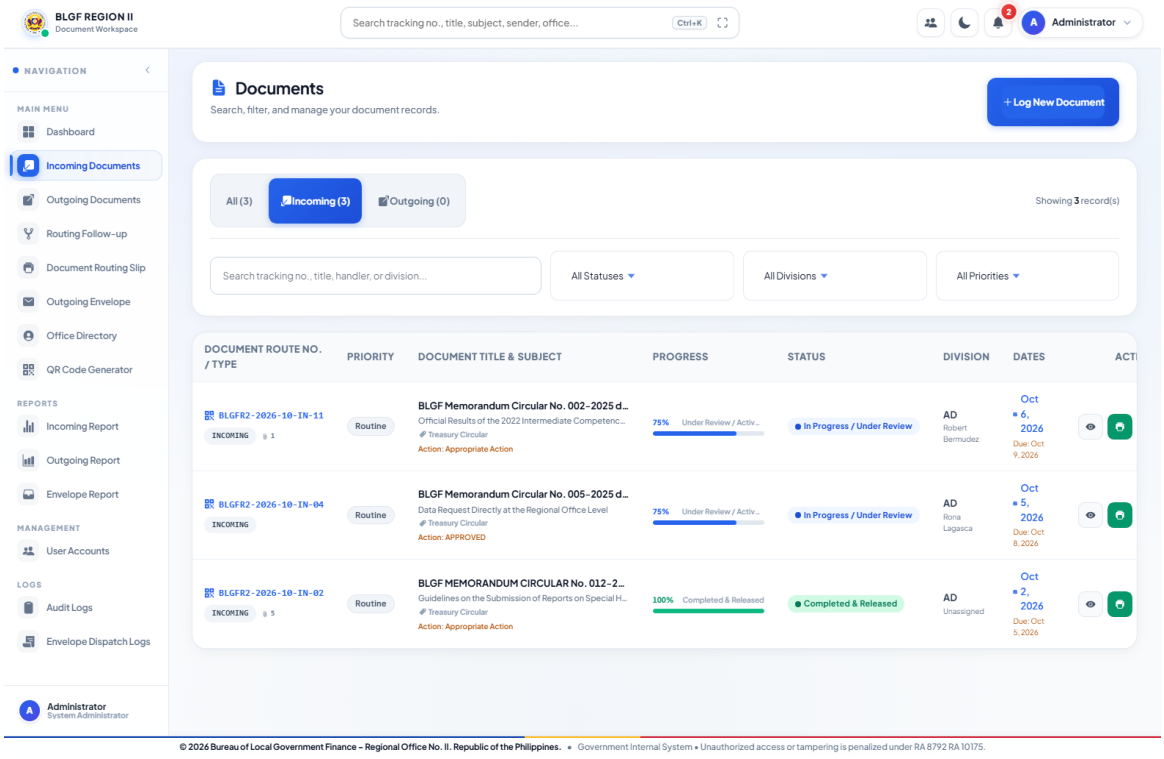


Figure 4.14. Incoming Documents

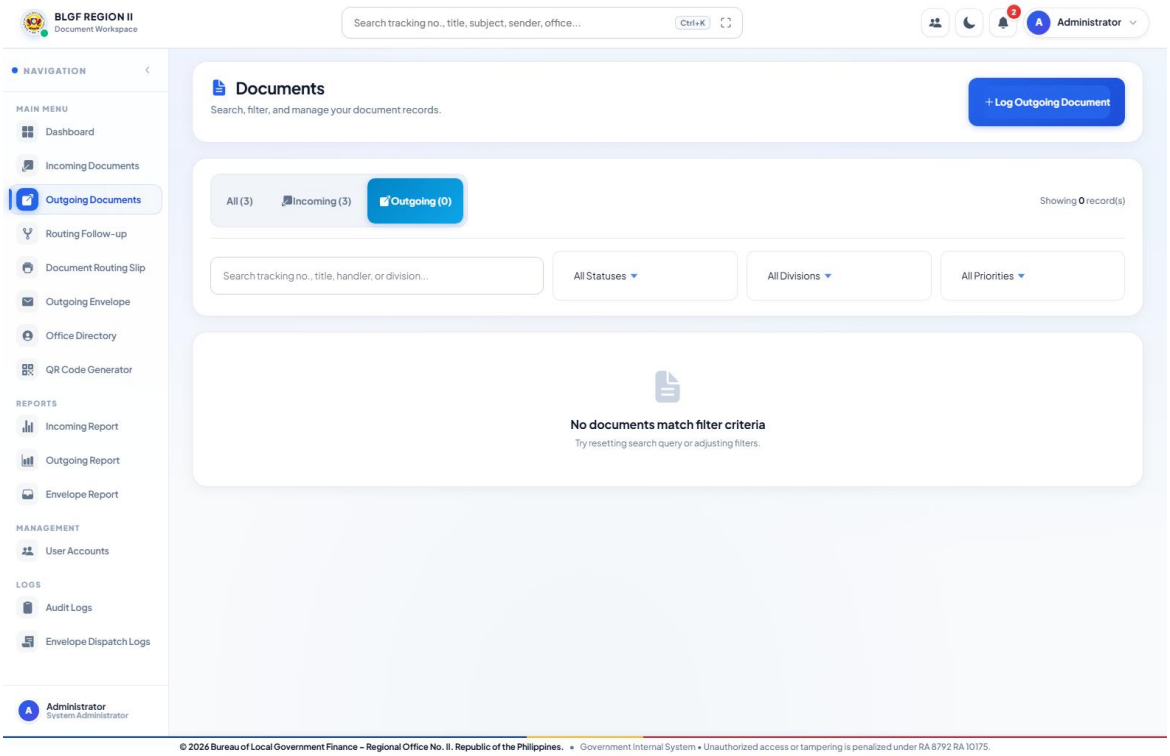


Figure 4.15. Outgoing Documents

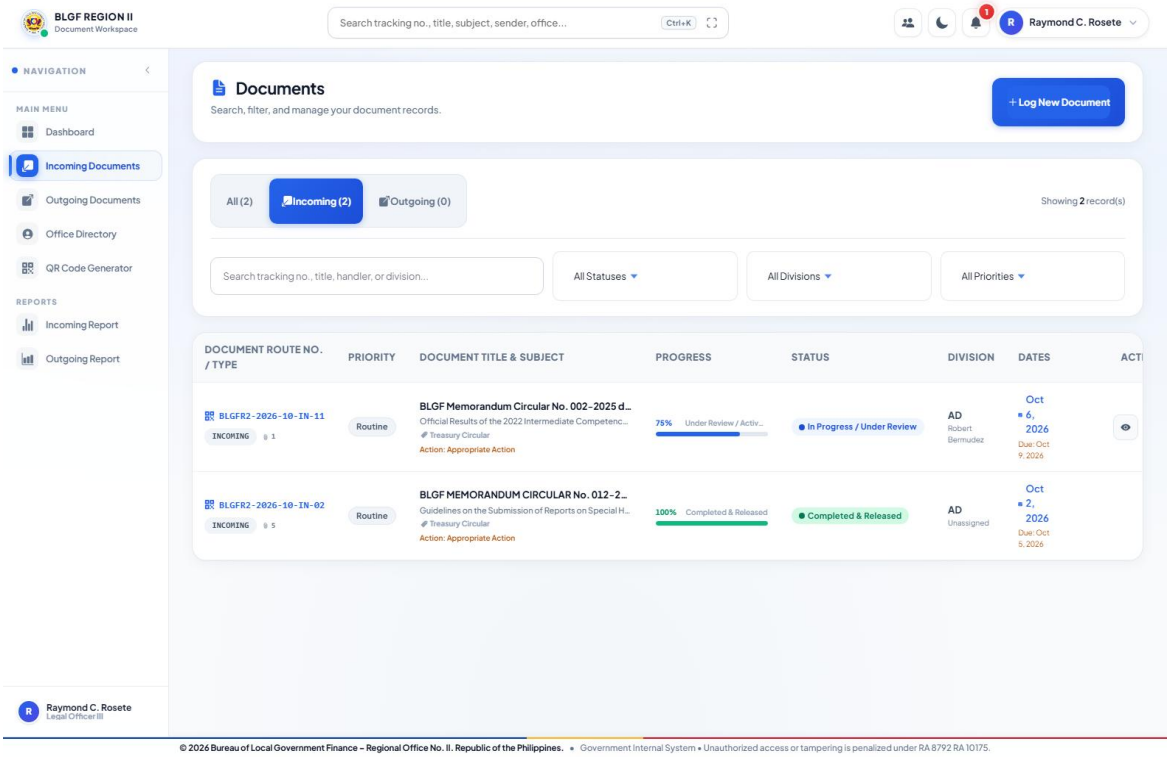


Figure 4.16. Incoming Documents

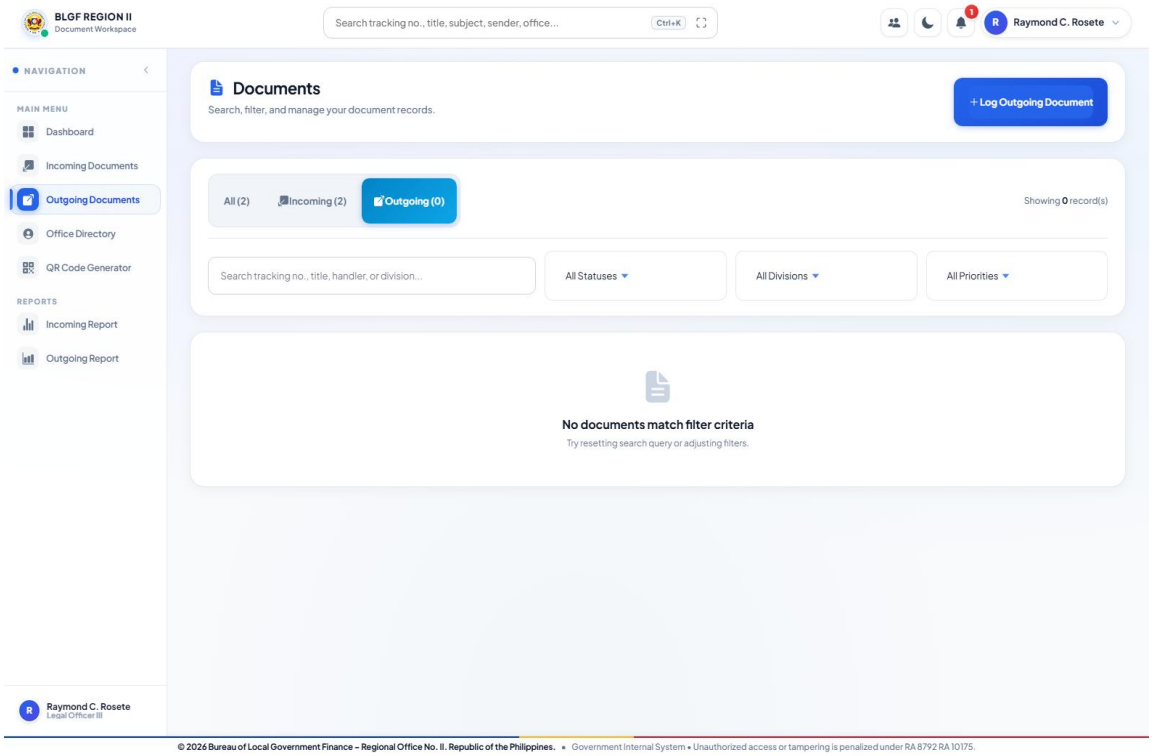


Figure 4.17. Outgoing Documents

Procedure:

Open Incoming or Outgoing Documents; use status, office, date, or keyword filters; inspect progress and handler information; select View or Quick View to open the access-authorized transaction.

c. Data Description

The monitoring module provides All, Incoming, and Outgoing views with route number, priority, title, subject, progress, status, division, handler, dates, and permitted actions. Administrator results support organization-wide monitoring; ordinary users receive only created, assigned, sent, or received transactions.

User's View 6: Document Transaction, Conversation, and Routing Module

a. Data Environment

Primary data: documents, document_routes, document_attachments, and users. One document may have many recipient branches; every route belongs to one document, and a route-scoped file remains tied to its intended conversation.

b. Data Entry

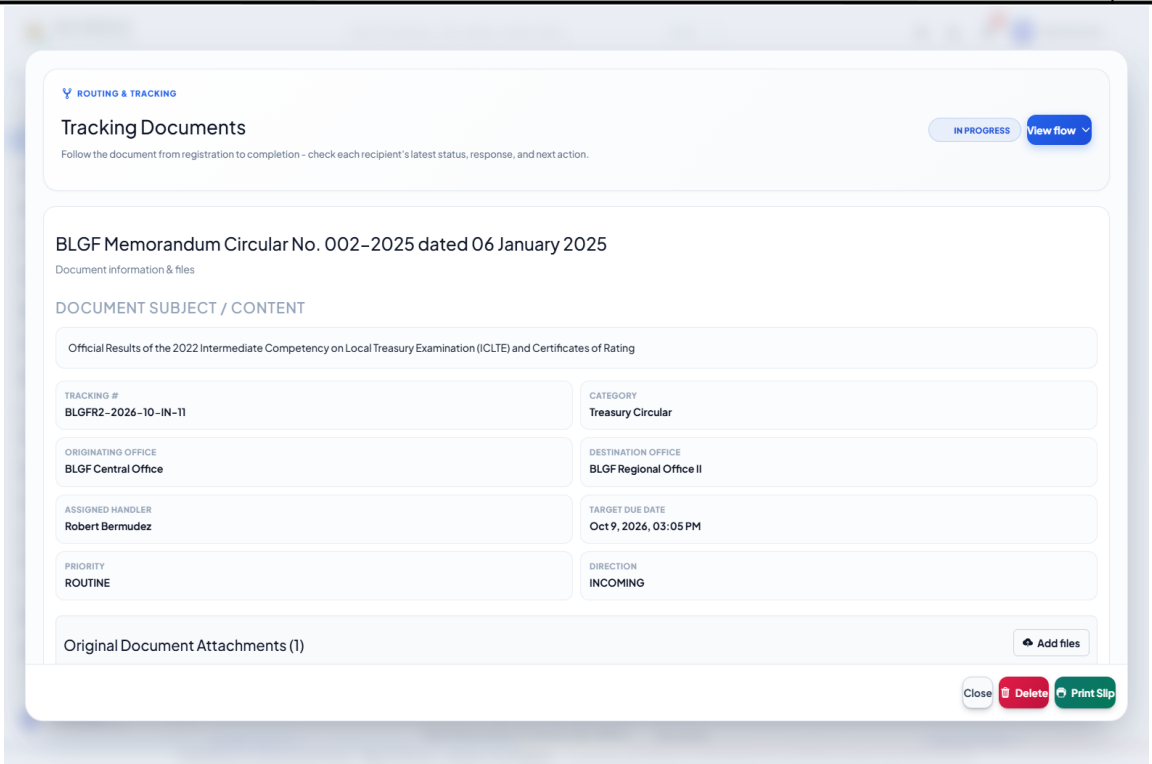


Figure 4.18. Document Details And Routing Flow

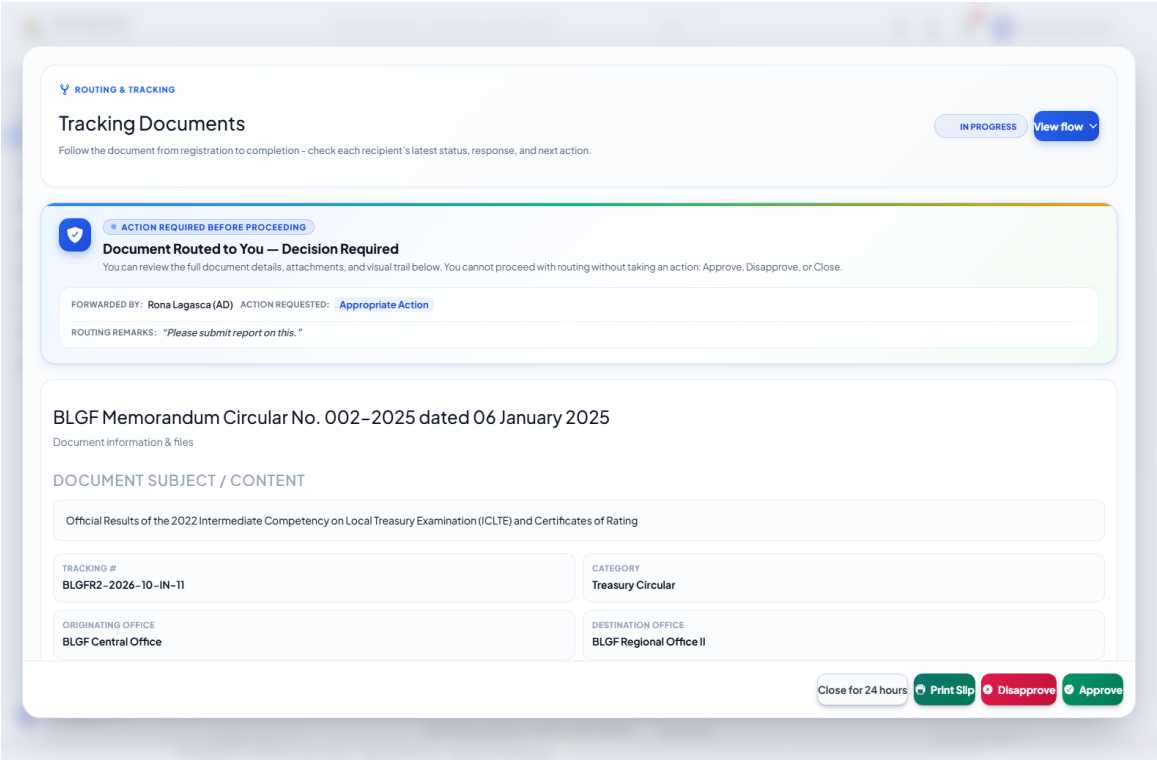


Figure 4.19. Document Details Conversation And Attachments

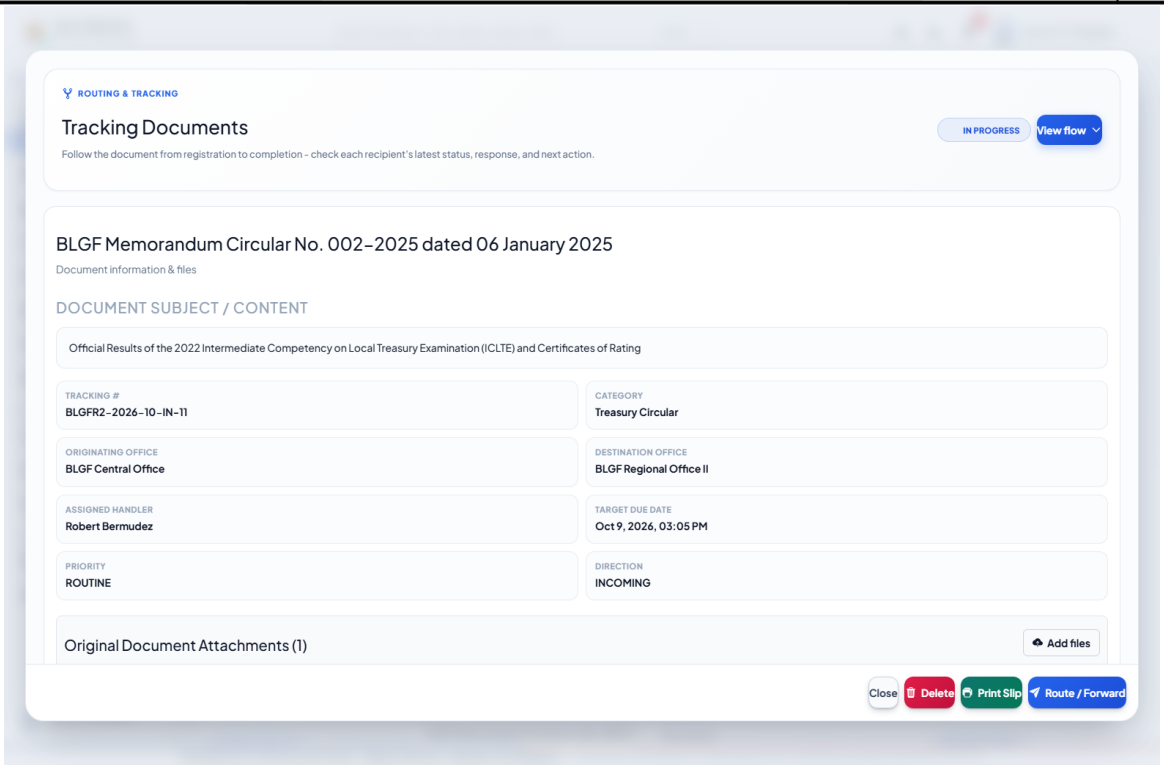


Figure 4.20. Document Details And Recipient Flow

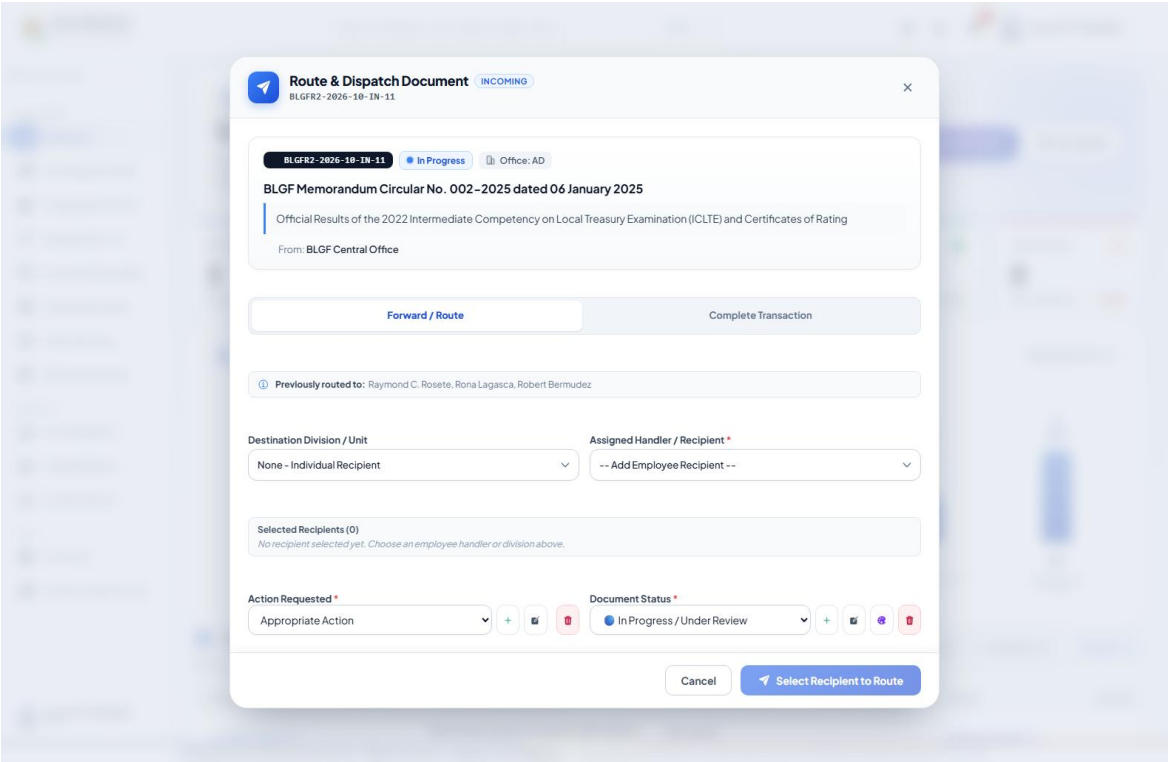


Figure 4.21. Route Forward And Reply Form

Procedure:

Open a document; select the intended routing branch; review its instruction, history, notes, and scoped files; when authorized, choose Route/Forward or Step Reply; select the recipient and action, enter the note, add an optional file, and submit.

c. Data Description

The transaction view presents document metadata, permitted original files, lifecycle progress, routing branches, decisions, instructions, conversation messages, and recipient-scoped attachments. Each recipient has an independent flow branch. The operational originator or assigned participant—not the System Administrator—uses the Route/Forward form to select recipients, action, status, note, and optional reply file. Completion and final instruction remain attached to the participant who actually ended that branch.

User's View 7: Routing Follow-up and Reminder Module

a. Data Environment

Primary data: documents, document_routes, users, and derived reminder notifications. Each unfinished recipient branch is monitored independently; sending a reminder does not change route status.

b. Data Entry

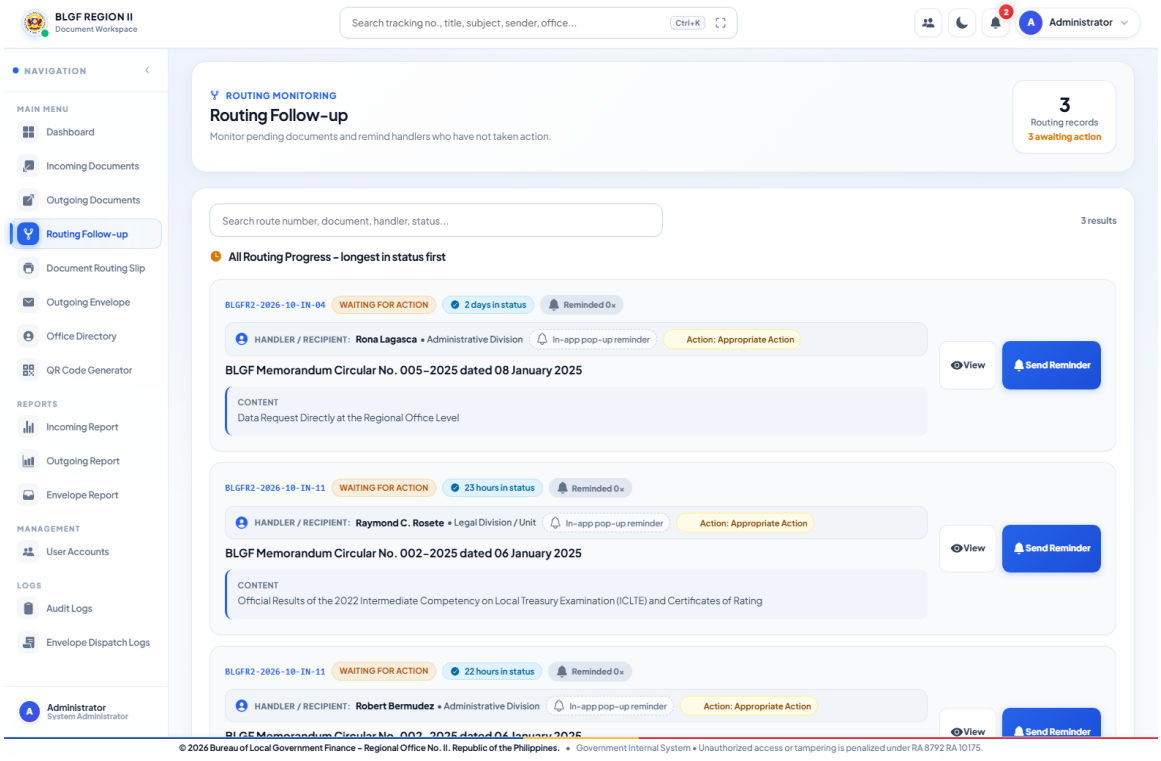


Figure 4.22. Routing Follow Up

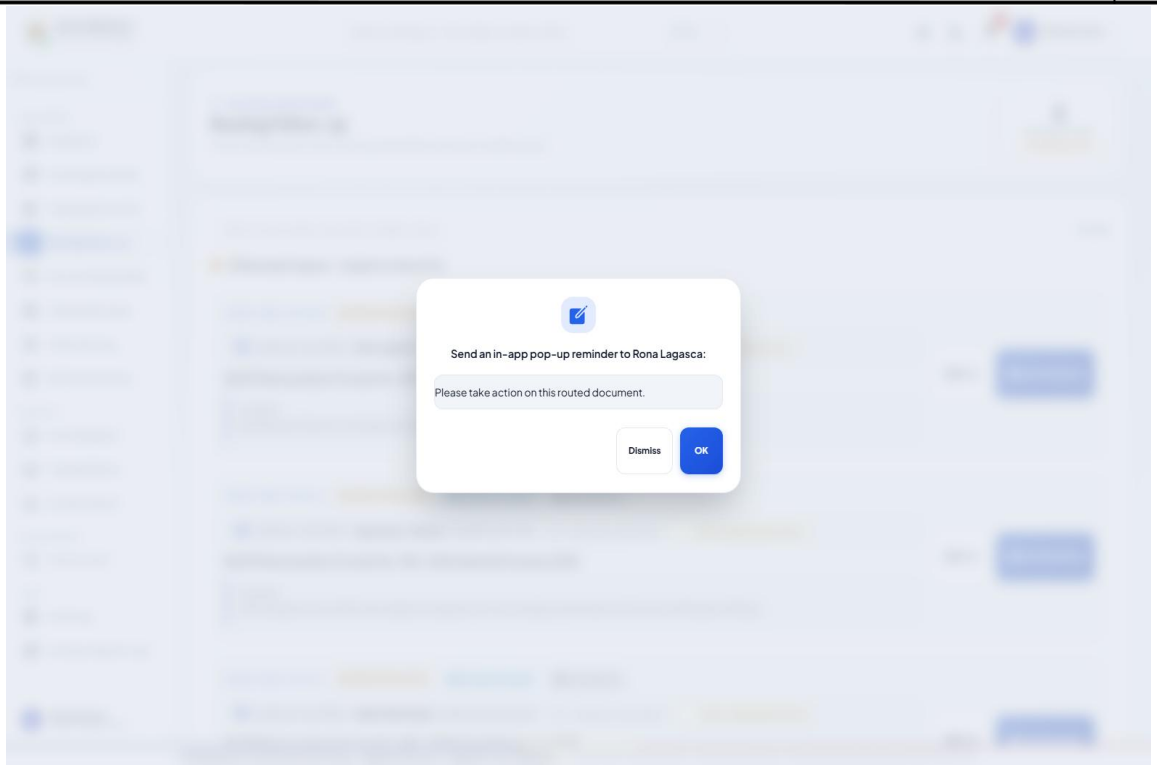


Figure 4.23. Send Reminder Dialog

Procedure:

Open Routing Follow-up; review all unfinished recipient branches; select View when document context is needed; select Send Reminder; edit the reminder message; submit the in-app alert. Do not use the reminder as an approval or routing action.

c. Data Description

Routing Follow-up lists every unfinished recipient separately and orders records by elapsed time in the current status. An authorized monitor may review the document and send an editable in-app reminder. A reminder is not an approval, disapproval, routing event, or completion action.

User's View 8: Document Routing Slip Module

a. Data Environment

Primary data: documents and document_routes. The printable routing slip reads the stored metadata and chronological route history without creating a second transaction record.

b. Data Entry

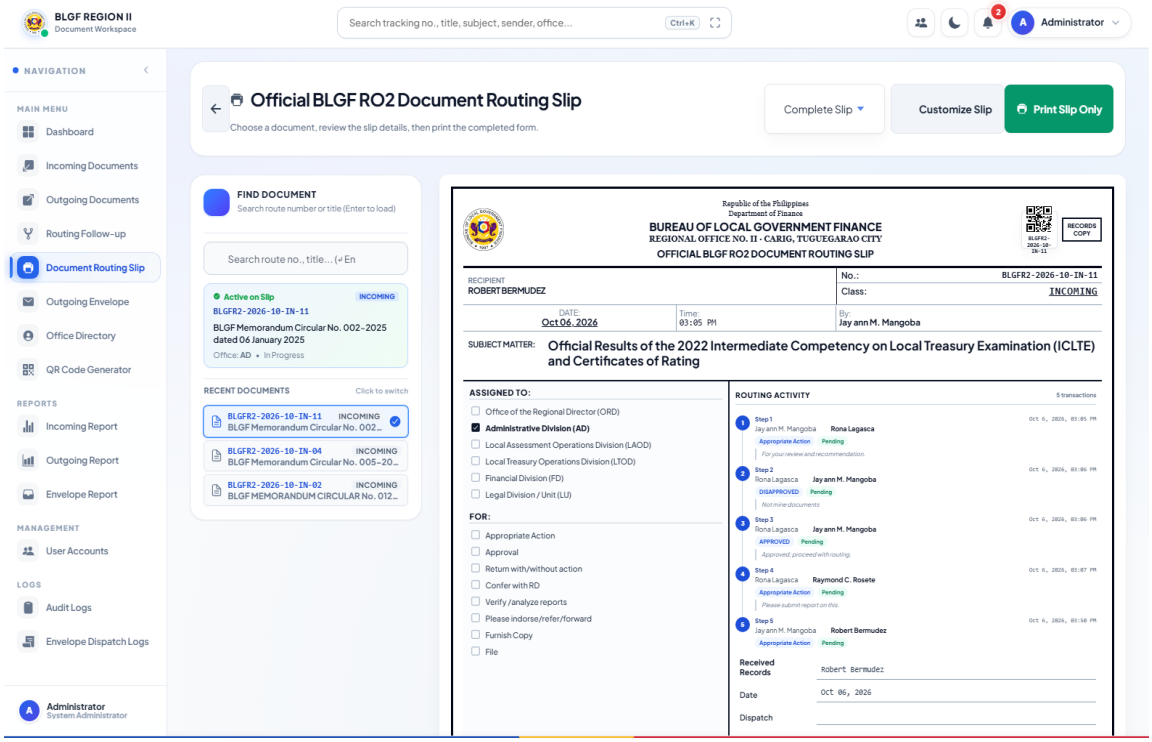


Figure 4.24. Document Routing Slip

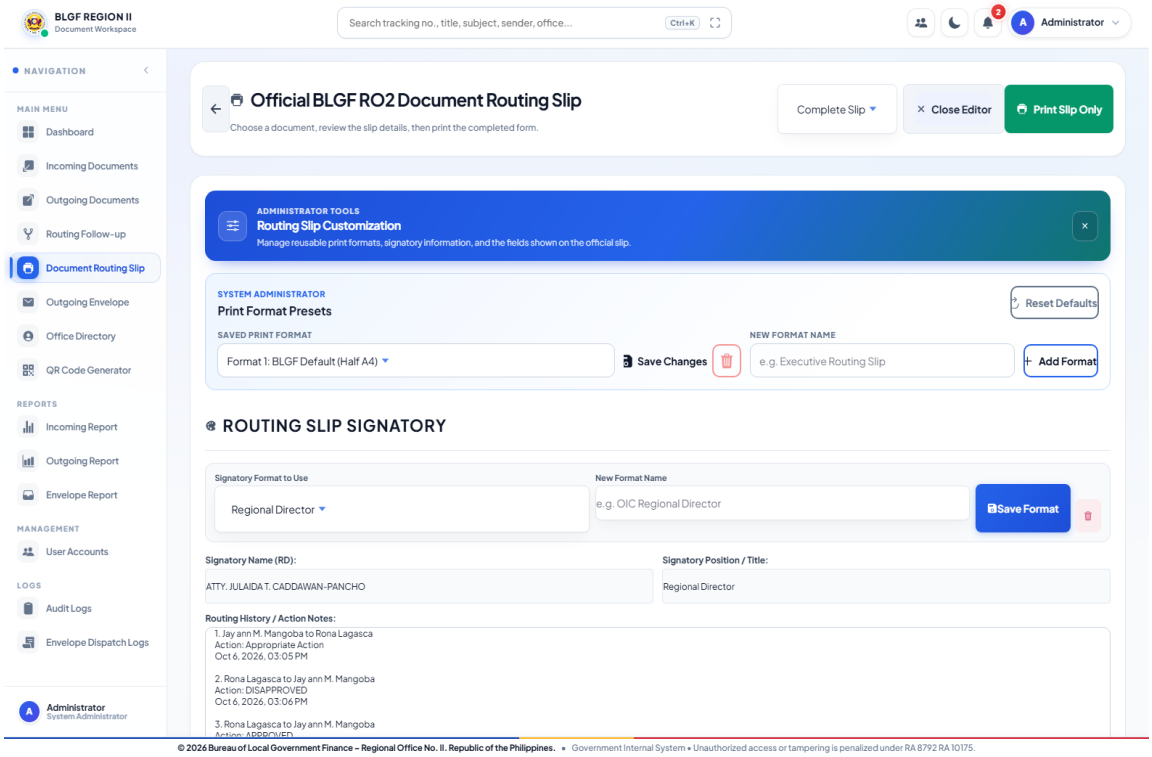


Figure 4.25. Routing Slip Customization

Procedure:

Open Document Routing Slip; select the registered document; verify route and recipient details; apply an authorized print preset or signatory format; preview the official layout; print or export as required.

c. Data Description

The routing-slip module selects a registered document and presents its route number, subject, origin, action, assignments, and routing activity in an official print layout. Authorized users may maintain print presets and signatory formats. This menu is hidden from a least-privilege STAFF account unless explicitly granted.

User's View 9: Outgoing Envelope Module

a. Data Environment

Primary data: documents, employee_profiles, and envelope_logs. Envelope preparation may use a linked document and directory recipient; dispatch is recorded separately from electronic routing.

b. Data Entry

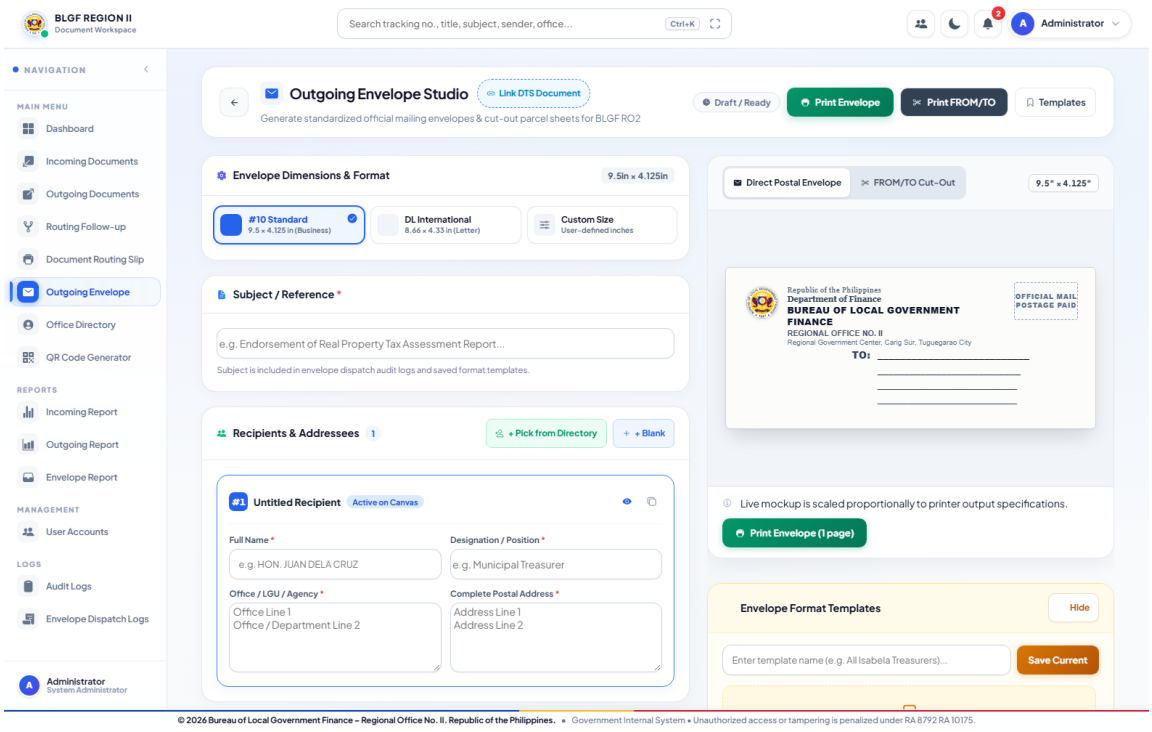


Figure 4.26. Outgoing Envelope

Procedure:

Open Outgoing Envelope; select or enter the document reference; supply recipient and return-address details; choose the envelope size or cut-out format; preview; print; save the template or dispatch record when permitted.

c. Data Description

The Outgoing Envelope Studio links a DTS document, records the subject/reference, prepares one or several recipients, imports directory entries, applies return-address information, previews standard or custom sizes, prints envelopes or FROM/TO cut-outs, saves templates, and records dispatch activity. This feature is permission-controlled.

User's View 10: Office Directory and Personnel Records Module

a. Data Environment

Primary data: employee_profiles, users, directory_sections, and divisions. A profile may exist without a login account; a login account may link to at most one employee profile.

b. Data Entry

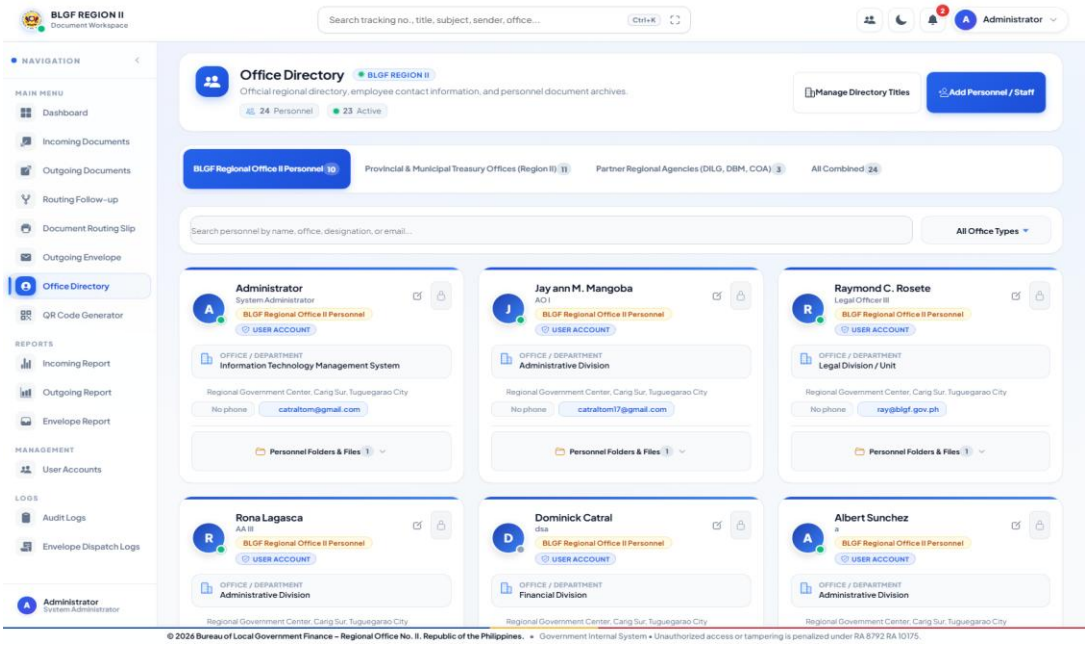


Figure 4.27. Office Directory

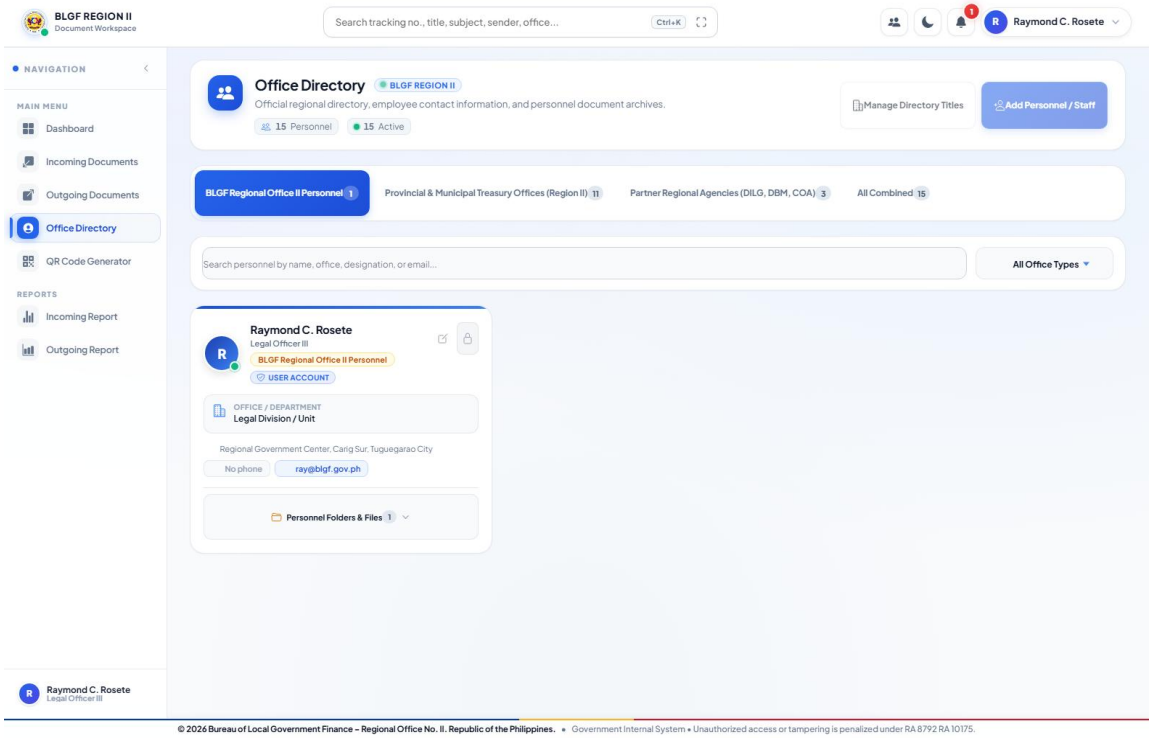


Figure 4.28. Office Directory

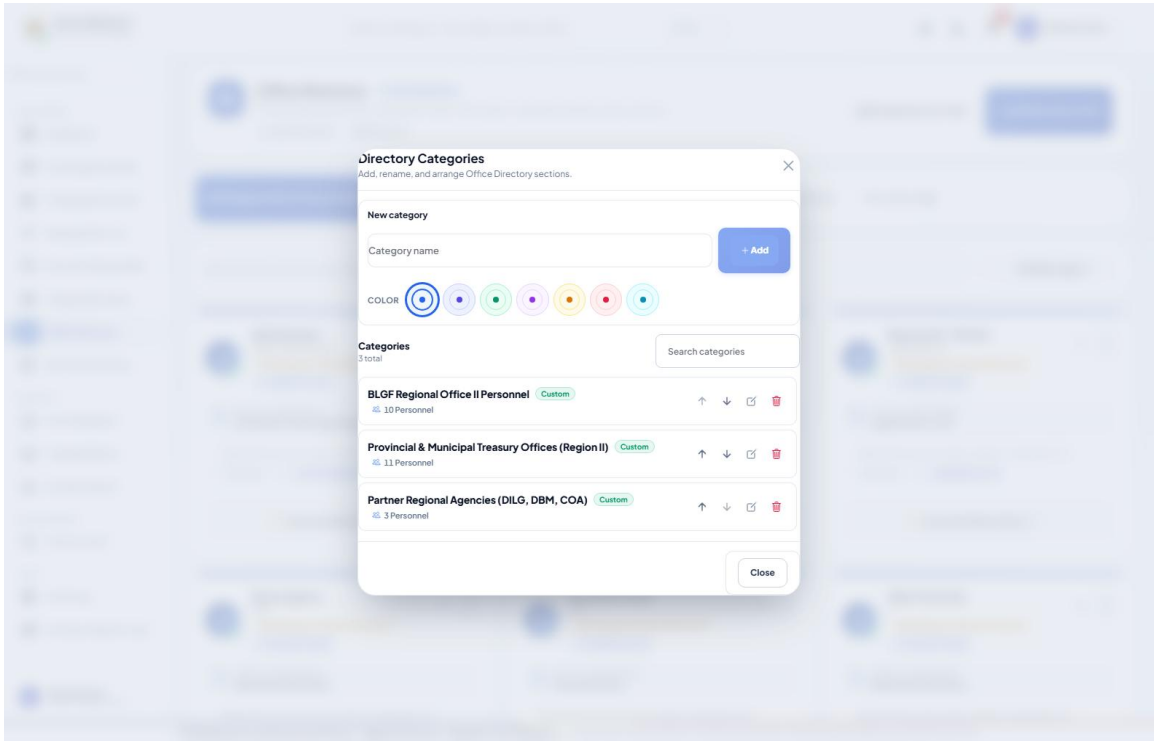


Figure 4.29. Directory Title Management

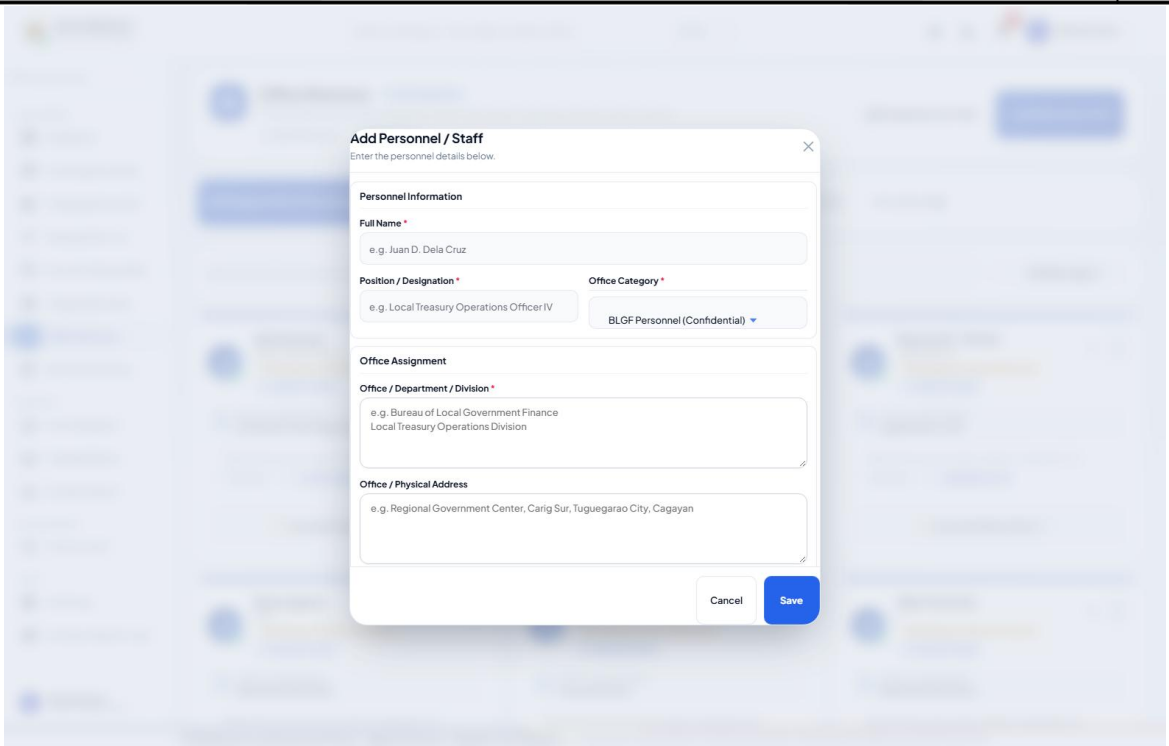


Figure 4.30. Add Personnel Form

Procedure:

Open Office Directory; select a directory section; search or inspect personnel; authorized administrators may add personnel, manage titles, and organize permitted profile folders; regular users remain in view-only scope unless granted otherwise.

c. Data Description

The Office Directory organizes BLGF personnel, treasury offices, LGUs, and partner agencies. Authorized administrators manage directory titles and personnel details; linked User Account profiles remain protected and synchronized. Personnel folders support controlled file organization and PDF preview. A regular user sees only directory sections and personnel allowed by permission.

User's View 11: QR Code Module

a. Data Environment

Primary data: documents and generated QR output. The QR image is derived from a verified tracking number, URL, or approved value and does not replace the document record.

b. Data Entry

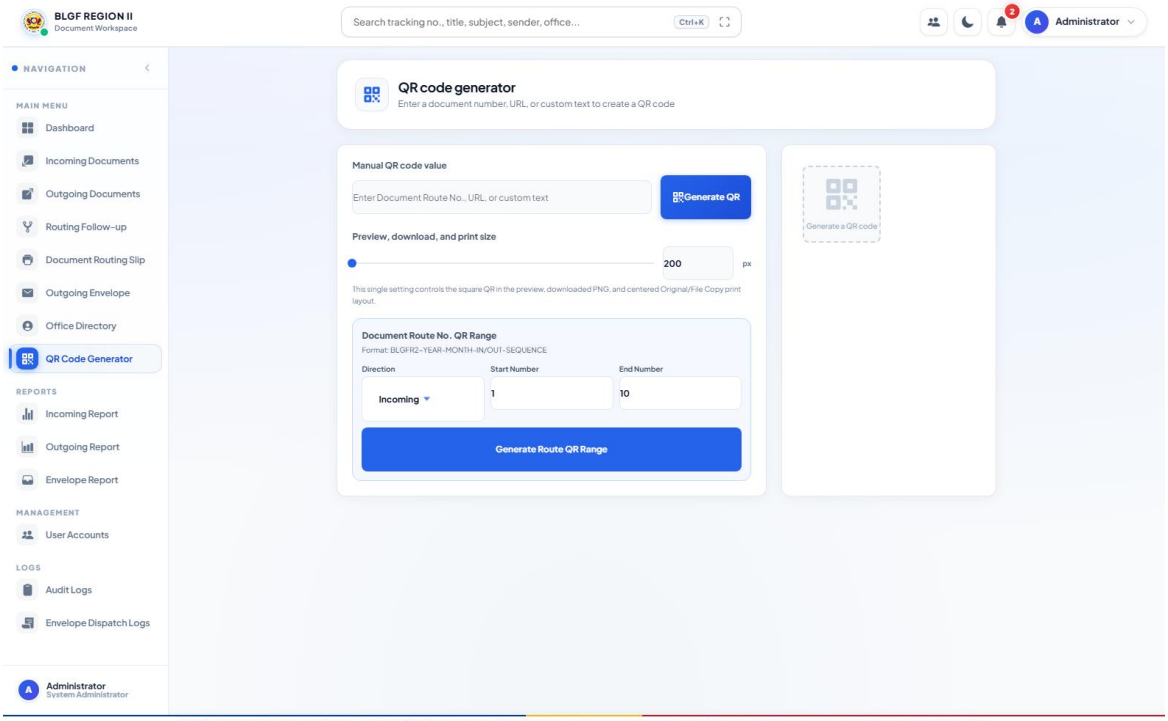


Figure 4.31. Qr Code Generator

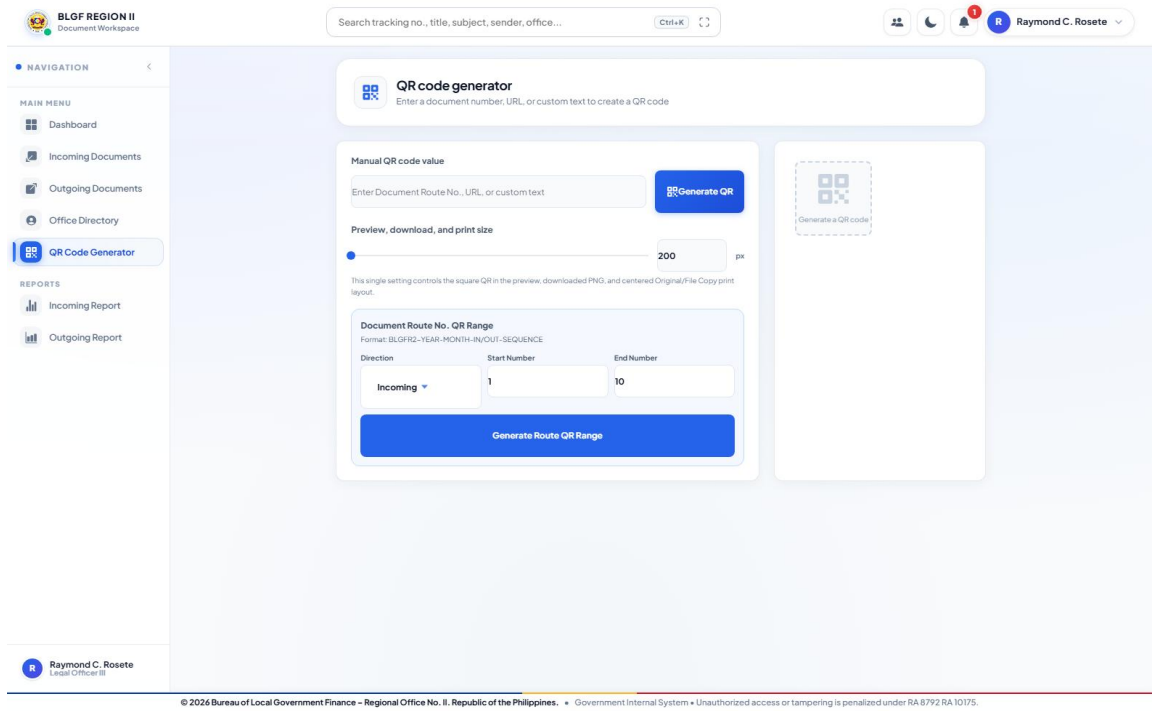


Figure 4.32. Qr Code Generator

Procedure:

Open QR Code Generator; enter a permitted route number, route range, URL, or approved value; choose the size; generate the preview; download, copy, or print the resulting code.

c. Data Description

The QR module generates a code from a document number, URL, or approved custom value. It supports route-number ranges, size control, preview, recent codes, download, copy, and print. The same functional screen may be available to Administrator and regular user when the QR menu is granted.

User's View 12: Incoming, Outgoing, and Envelope Reports Module

a. Data Environment

Primary data: documents, document_routes, and envelope_logs. Incoming and outgoing reports summarize transaction state and full route history; envelope reports summarize physical dispatch records.

b. Data Entry

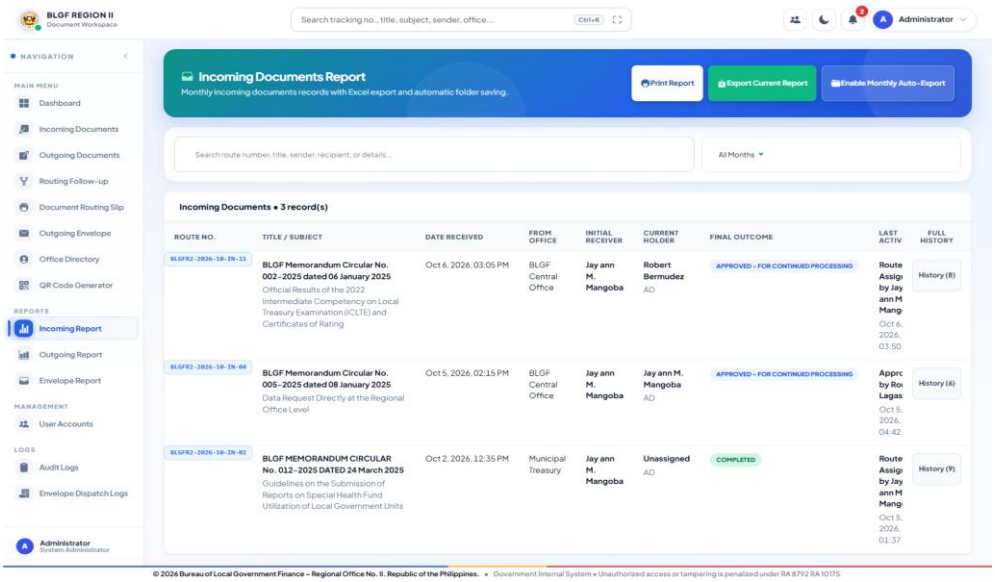


Figure 4.33. Incoming Report

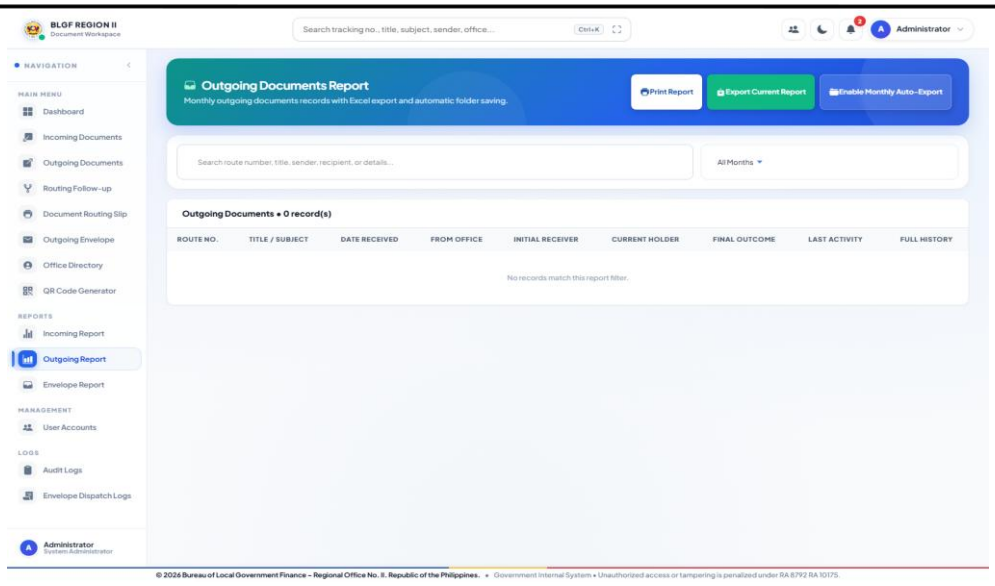


Figure 4.34. Outgoing Report

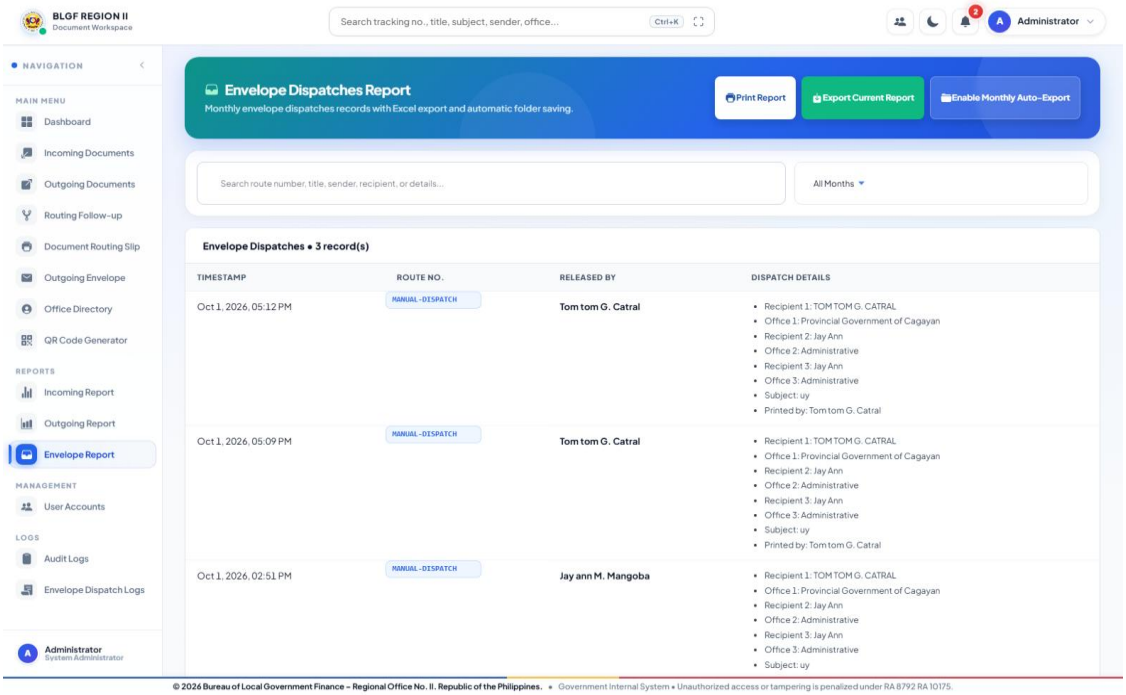


Figure 4.35. Envelope Report

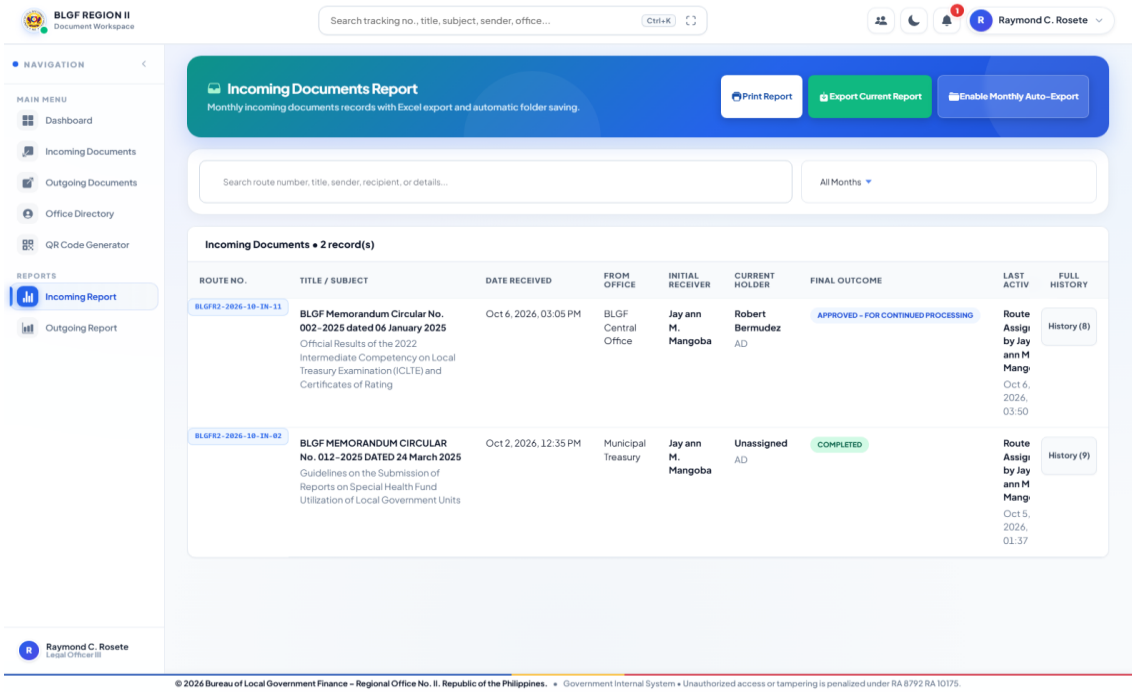


Figure 4.36. Incoming Report

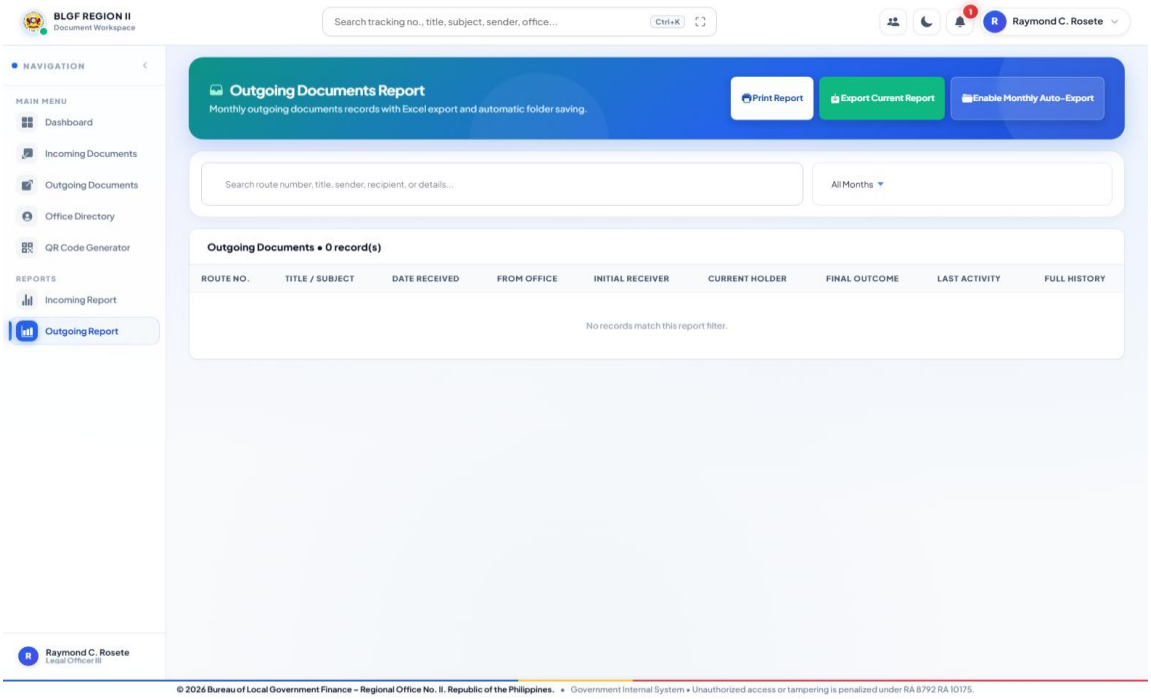


Figure 4.37. Outgoing Report

Procedure:

Open the required report; set the date and status filters; inspect expandable document history; print or export the permitted data; configure monthly auto-export only when the account has that administrative permission.

c. Data Description

Reports use live document and dispatch information. Incoming and outgoing reports include route number, subject, dates, office, initial receiver, current holder, outcome, last activity, and expandable history. Authorized users may print, export Excel workbooks, and select a supported monthly auto-export folder. Regular-user reports remain access-scoped.

User's View 13: User and Access Management Module

a. Data Environment

Primary data: users, divisions, and employee_profiles. Roles provide defaults, while account permissions determine the final allowed menus and actions.

b. Data Entry

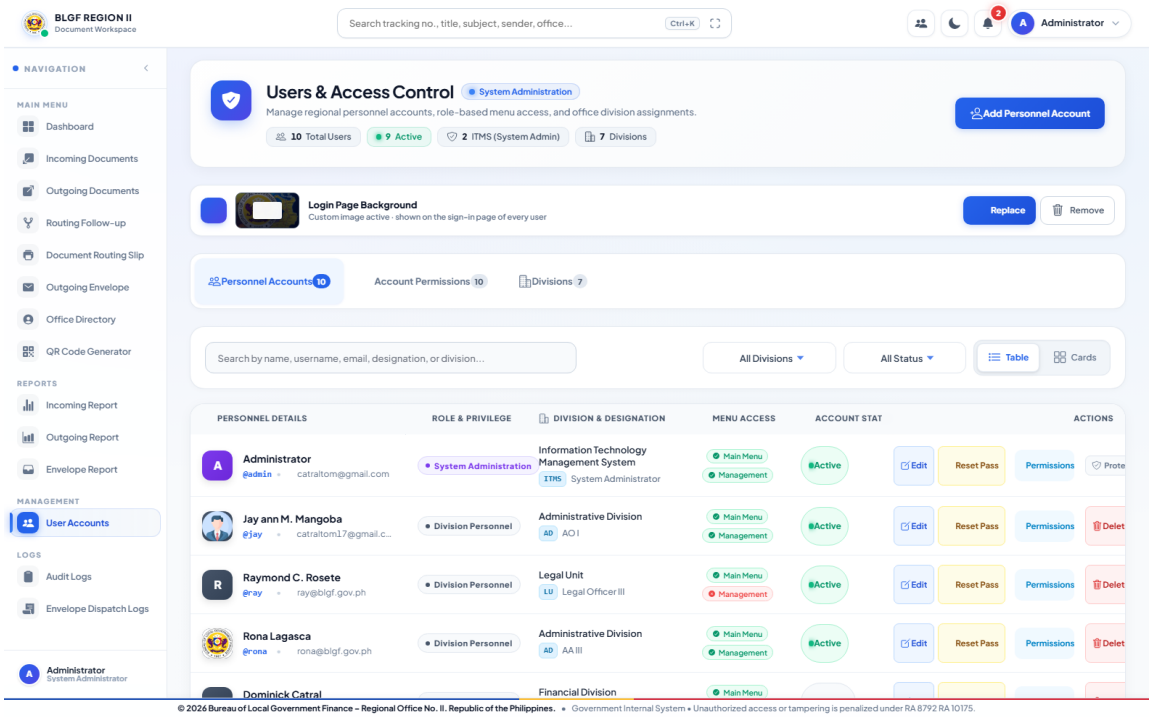


Figure 4.38. User Accounts

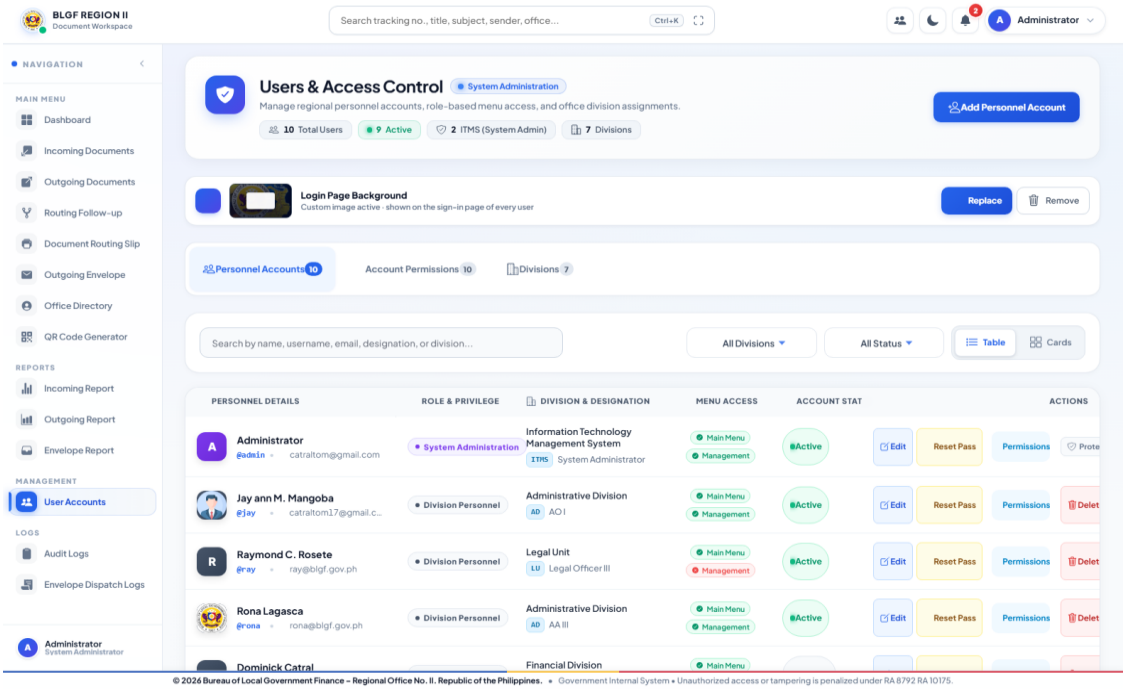


Figure 4.39. Personnel Accounts Table

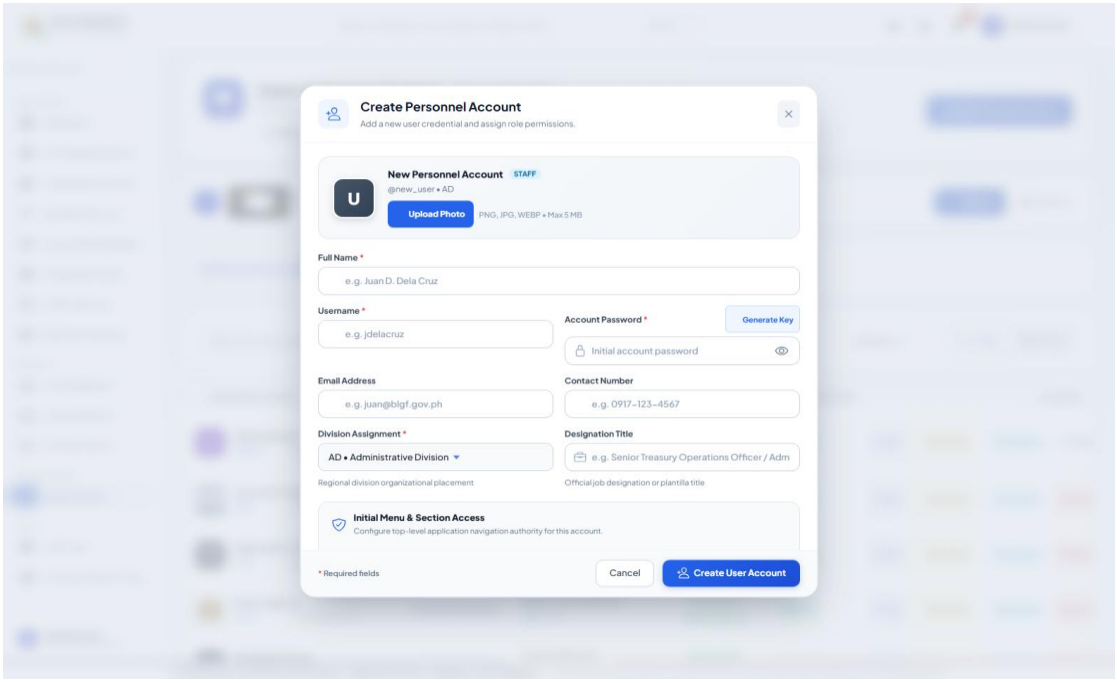


Figure 4.40. Create User Account Form

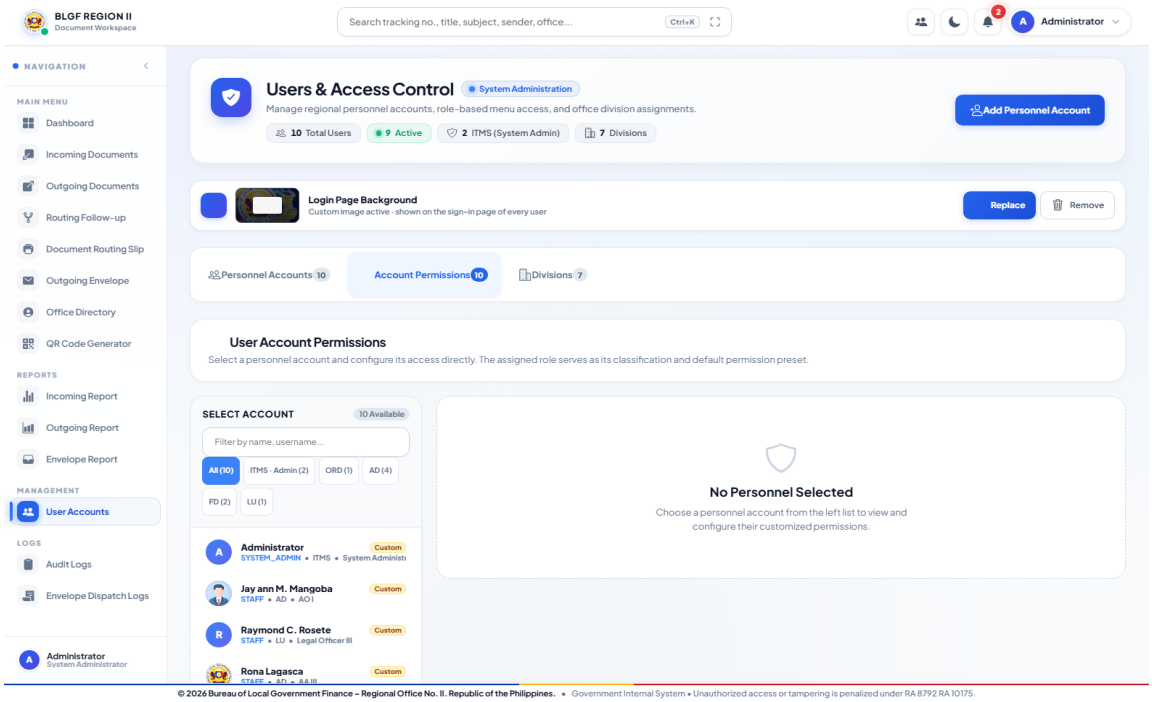


Figure 4.41. Role And Account Permissions

Procedure:

Open User Accounts; search or select a personnel account; create or edit identity details; assign the role and division; configure menu and granular action permissions; activate, deactivate, or reset credentials only when authorized.

c. Data Description

The Administrator-only access module creates and maintains personnel accounts, active state, password reset, role, division, designation, menu access, and granular actions. Protected System Administrator identities cannot be deleted or assigned by an unauthorized user. Role defaults may be customized per account through the permission matrix.

User's View 14: Division Management Module

a. Data Environment

Primary data: divisions, users, and documents. The unique division code is reused as an organizational reference for accounts, assignments, filters, and reports.

b. Data Entry

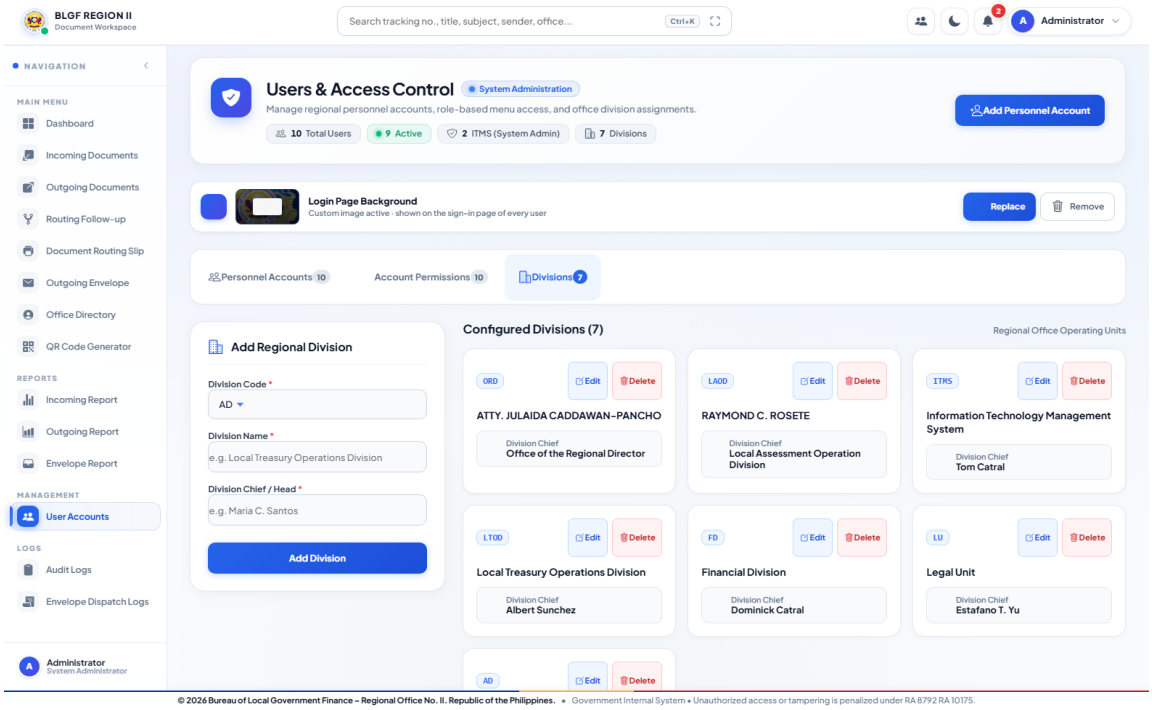


Figure 4.42. Division Management

Procedure:

Open Division Management; add or edit the official code, name, chief, and email; verify linked account and document references before saving; retain understandable historical values.

c. Data Description

Division Management maintains the official organizational codes, names, chiefs, and email addresses used by accounts, routing, filters, reports, and directory records. Changes require System Administrator authority and must preserve understandable historical document data.

User's View 15: Audit and Envelope Dispatch Logs Module

a. Data Environment

Primary data: audit_logs, envelope_logs, users, and document references. Logs preserve who performed an event, what happened, when it occurred, and the affected route number.

b. Data Entry

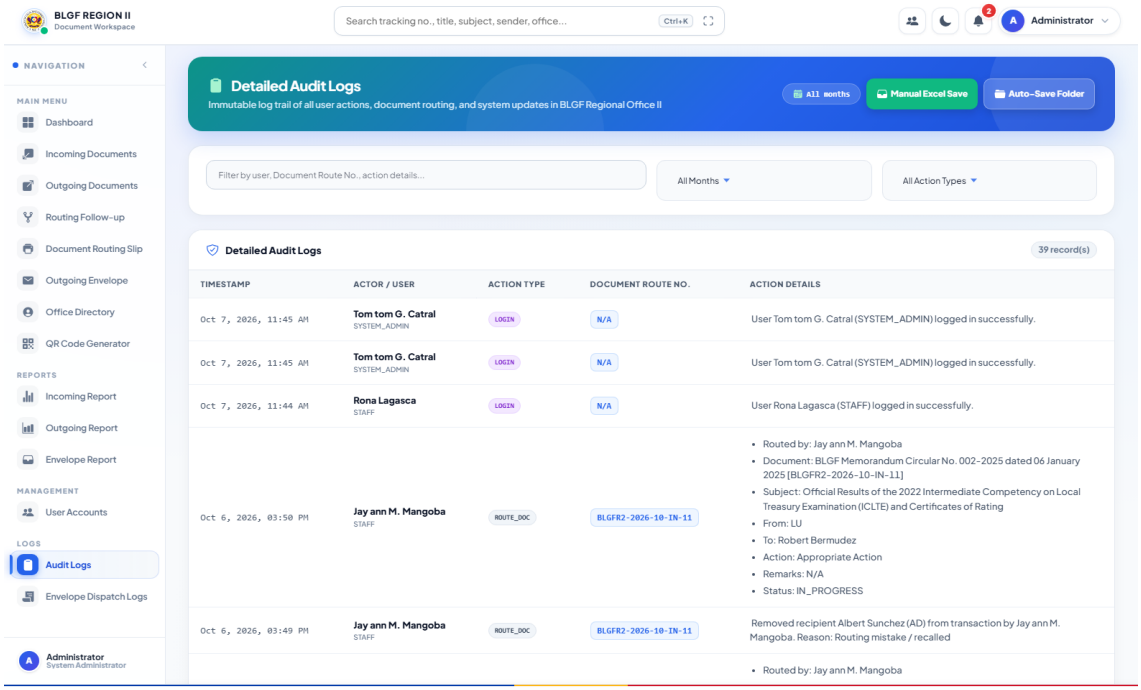


Figure 4.43. Audit Logs

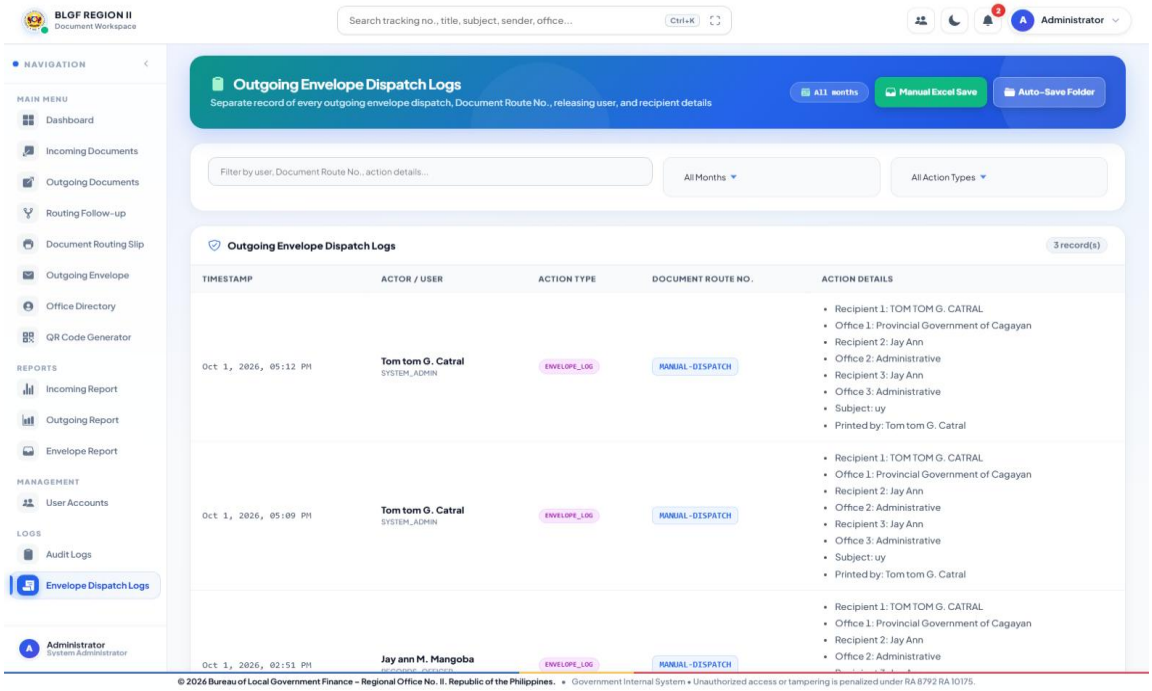


Figure 4.44. Envelope Dispatch Logs

Procedure:

Open Audit Logs or Envelope Dispatch Logs; filter by date, actor, action, or route number; inspect the recorded detail; export only when authorized. Log entries are monitoring evidence and are not operational routing controls.

c. Data Description

Audit Logs preserve actor, role, event, document route number, details, time, and network address for authentication, document, routing, decision, user, cancellation, and deletion activity. Envelope Dispatch Logs separately preserve physical-mail preparation and dispatch. Authorized views support filtering, pagination, Excel/CSV output, and monthly auto-save.

User's View 16: Account Settings Module

a. Data Environment

Primary data: users. The authenticated user may change permitted profile and password information but cannot elevate the assigned role or permissions.

b. Data Entry

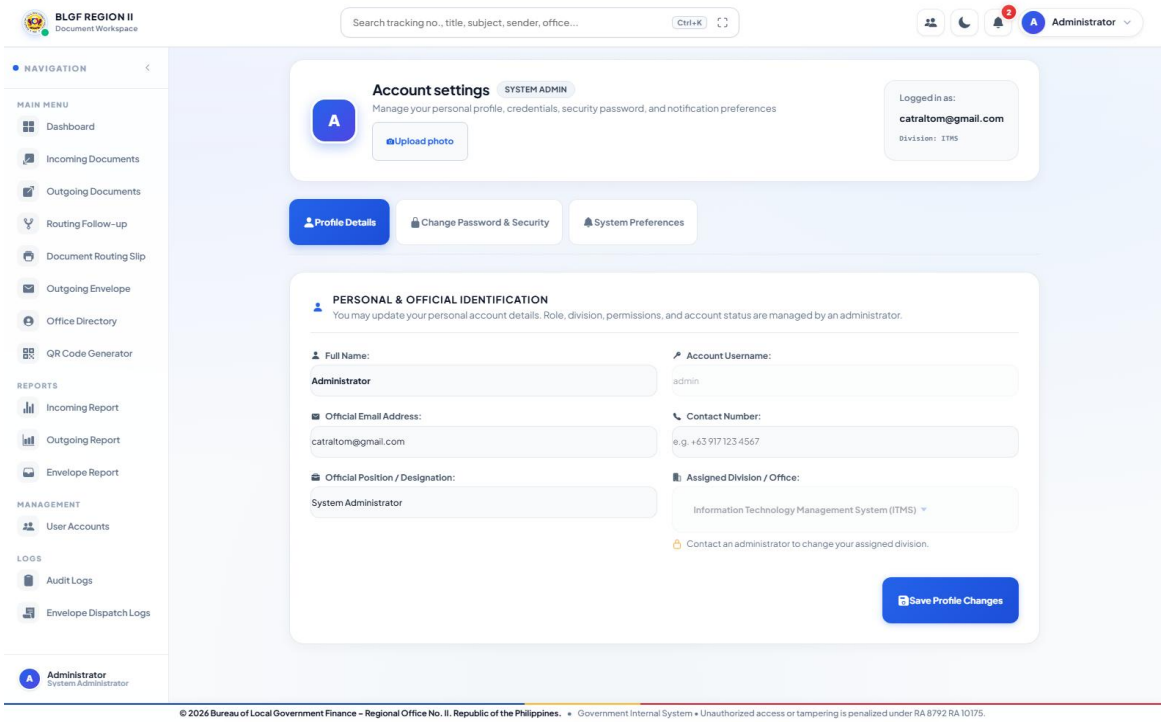


Figure 4.45. Account Settings

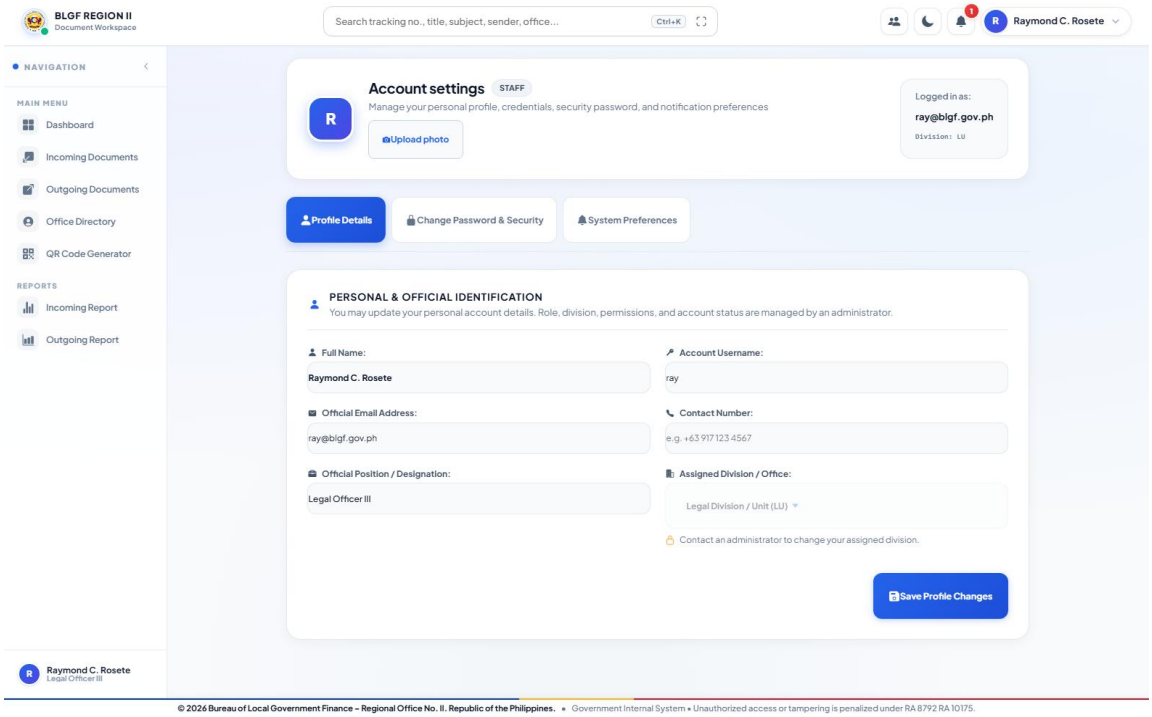


Figure 4.46. Account Settings

Procedure:

Open Account Settings; update permitted personal or official details and profile image; use the password section to change credentials; save; sign in again when required. Assigned role and privileges cannot be self-elevated.

c. Data Description

Account Settings allows the authenticated person to maintain permitted personal and official information, profile image, and password. The screen preserves the assigned role and prevents self-escalation. Administrator and regular-user presentations use the same identity controls within their respective permissions.

4.2.2 Data Model

The Data Model is presented in two levels. The simplified relationship view explains the principal business relationships and cardinalities. The detailed physical ERD then lists the implemented MySQL fields, primary keys, unique keys, enforced Prisma foreign keys, logical references, derived notification view, referential actions, and the relationship index. This two-level presentation keeps the explanation readable without omitting implementation detail.

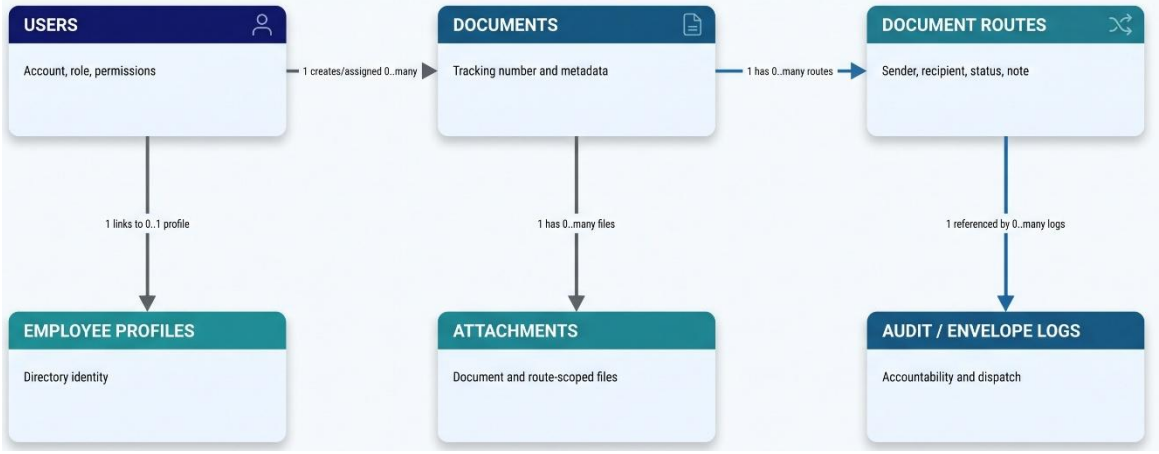


Figure 4.47. Simplified Relationship View of the WDTMP Data Model

Detailed Physical Data Model

In the detailed ERD, PK identifies a primary key, UQ a uniqueness constraint, FK an enforced Prisma relation, LR a logical stored reference without a foreign-key constraint, and DV a value derived at runtime. A 1 to 0..many relationship means one parent can have no child record or several child records; 1 to 0..1 means an optional one-to-one relationship. CASCADE applies only to document-owned routes and attachments, while user relations use controlled Set Null behavior to preserve historical names and transaction evidence.

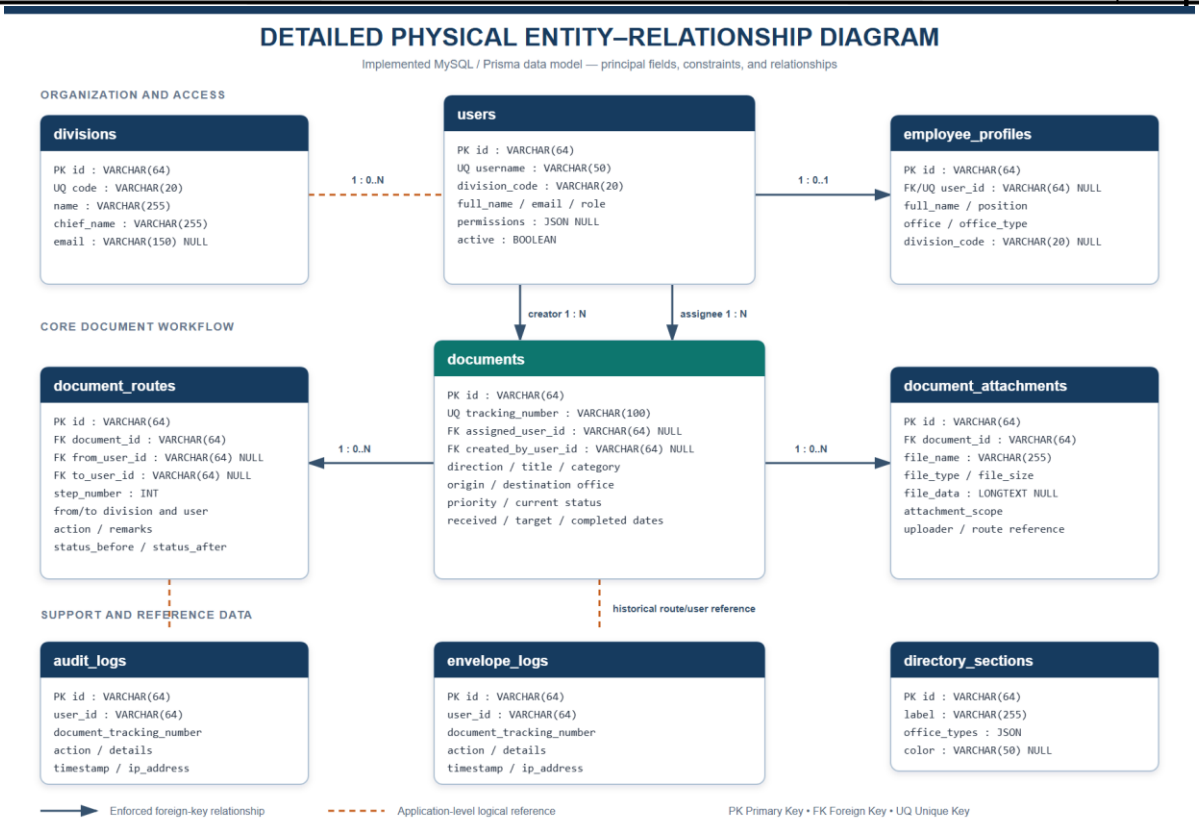


Figure 4.48. Detailed Physical Entity-Relationship Diagram of the Implemented WDTMP

Relationship Definitions

users (1) -> documents (0..many), created_by_user_id: one account may register many documents; each document retains at most one linked creator account.

users (1) -> documents (0..many), assigned_user_id: one account may be the current assignee of many documents; assignment may be null before routing or after controlled account changes.

documents (1) -> document_routes (0..many), document_id: every route belongs to exactly one document; deleting the parent document cascades to its owned route history.

users (1) -> document_routes (0..many), from_user_id and to_user_id: an account may send or receive many route steps; retained display names preserve history if the linked account becomes unavailable.

documents (1) -> document_attachments (0..many), document_id: every attachment belongs to one document; attachment_scope and uploaded_for_route_id restrict contextual visibility.

users (1) -> employee_profiles (0..1), user_id: an account may have one linked employee profile, while directory-only personnel may exist without a login account.

divisions -> users/documents and directory_sections -> employee_profiles are controlled code/type references used for organization, filtering, and reports.

audit_logs and envelope_logs retain user_id and document_tracking_number as historical logical references so accountability remains readable independently of current account or document state.

Entity Descriptions

Division: stores the official division identifier, unique code, name, chief, and email used by accounts, documents, routing filters, and reports.

User: stores credentials, identity, role, division code, permissions, active state, temporary-password expiration, and profile details. It is the parent identity for created documents, assigned documents, routes sent, routes received, and an optional employee profile.

Document: stores the unique tracking/route number and the complete incoming or outgoing metadata, current state, priority, office, assignment, registration dates, target date, creator, and completion date.

DocumentRoute: stores each independent workflow step, sender and recipient identities, divisions, requested action, recipient-specific remarks, status transition, step number, and creation time.

DocumentAttachment: stores document-owned file metadata and data together with attachment scope, uploader, and optional route-step reference for confidential or recipient-scoped visibility.

EmployeeProfile: stores directory identity, position, office type, contact information, active state, folder metadata, division code, and an optional one-to-one link to a login account.

DirectorySection: stores administrator-defined directory group labels, permitted office types, and display color.

AuditLog: stores authenticated actor, role, action, optional route number, detailed event narrative, time, and IP address for accountable system activity.

EnvelopeLog: stores the corresponding actor, document reference, event detail, time, and IP address for outgoing-envelope preparation and dispatch activity.

Table Design

The table design below is based on the implemented Prisma/MySQL schema and the synchronized application stores used by the running system. PK means primary key, FK means foreign key, and NULL means the field is optional. Identity relationships use stable user IDs so document ownership and routing remain accurate after an authorized name or username change.

Relational integrity rules include a unique tracking number, cascading removal of route and attachment children when their parent document is deleted, controlled nullification of user references to preserve historical display data, indexes on document and user relationships, and recipient-scoped attachment metadata. Database backup must be paired with referenced file and personnel-folder storage.

Table 4.1. Divisions Table Design

Stores the official organizational units used by accounts, routing, filters, reports, and directory records.

Field Name	Data Type	Key / Constraint	Description
id	VARCHAR(64)	PK, NOT NULL	Immutable division identifier
code	VARCHAR(20)	UNIQUE, NOT NULL	Official short code such as AD, ORD, FD, LU, LAOD, LTOD, or ITMS
name	VARCHAR(255)	NOT NULL	Official division name
chief_name	VARCHAR(255)	NOT NULL	Current division chief or responsible official
email	VARCHAR(150)	NULL	Official contact email

Table 4.2. Users Table Design

Stores authenticated personnel identities, credentials, roles, organizational assignment, and granular access configuration.

Field Name	Data Type	Key / Constraint	Description
id	VARCHAR(64)	PK, NOT NULL	Immutable user identifier used by document ownership and routing
username	VARCHAR(50)	UNIQUE, NOT NULL	Current sign-in name

source_username	VARCHAR(50)	NOT NULL	Original/source username retained for synchronization
password	TEXT	NULL	Protected authentication credential
temporary_password_expires_at	DATETIME(3)	NULL	Expiration of administrator-issued temporary password
full_name	VARCHAR(150)	NOT NULL	Personnel display name
email	VARCHAR(150)	NOT NULL	Official/recovery email
role	VARCHAR(50)	NOT NULL	SYSTEM_ADMIN, ORD, RECORDS_OFFICER, DIVISION_CHIEF, ACTION_OFFICER, ADMIN, or STAFF
division_code	VARCHAR(20)	Logical reference	Assigned organizational division
designation	VARCHAR(150)	NULL	Official position/designation
contact_no	VARCHAR(50)	NULL	Contact number
avatar_url	TEXT	NULL	Profile-image reference
permissions	JSON	NULL	Allowed views, actions, delete flag, and route/log visibility
active	BOOLEAN	NOT NULL, default TRUE	Controls ability to sign in
created_at	DATETIME(3)	NOT NULL	Account creation timestamp

Table 4.3. Documents Table Design

Stores the authoritative metadata and current lifecycle state of every incoming and outgoing transaction.

Field Name	Data Type	Key / Constraint	Description
id	VARCHAR(64)	PK, NOT NULL	Immutable internal document identifier
tracking_number	VARCHAR(100)	UNIQUE, NOT NULL	Generated BLGFR2 route/tracking number; issued sequences are not reused
direction	VARCHAR(20)	NOT NULL	INCOMING or OUTGOING
title	VARCHAR(255)	NOT NULL	Official document title
subject	TEXT	NULL	Subject matter or particulars
category	VARCHAR(100)	NOT NULL	Document classification/type
originating_office	VARCHAR(255)	NOT NULL	Office or agency of origin
destination_office	VARCHAR(255)	NOT NULL	Current/intended destination office
sender_name	VARCHAR(150)	NULL	Source sender
sender_position	VARCHAR(150)	NULL	Sender position
sender_address	TEXT	NULL	Sender/office address
recipient_name	VARCHAR(150)	NULL	External or document-level recipient
recipient_position	VARCHAR(150)	NULL	Recipient position
recipient_office	VARCHAR(255)	NULL	Recipient office
recipient_address	TEXT	NULL	Recipient address
route_no	VARCHAR(100)	NULL	User-facing route number reference
priority	VARCHAR(30)	NOT NULL	ROUTINE, URGENT, VERY_URGENT, or CONFIDENTIAL
current_status	VARCHAR(30)	NOT NULL	NOT_YET_ROUTED, PENDING, IN_PROGRESS, FOR_SIGNATURE, COMPLETED, RETURNED, or ON_HOLD
current_division	VARCHAR(20)	NOT NULL	Current holding/processing division code
assigned_user	VARCHAR(150)	NULL	Display snapshot of current handler

assigned_user_id	VARCHAR(64)	FK users.id, NULL	Stable current-handler identity; ON DELETE SET NULL
date_received	DATETIME(3)	NOT NULL	Date and time registered/received
target_completion_date	DATETIME(3)	NULL	Expected completion date
completed_date	DATETIME(3)	NULL	Actual completion timestamp
created_by	VARCHAR(150)	NOT NULL	Display snapshot of originator
created_by_user_id	VARCHAR(64)	FK users.id, NULL	Stable creator identity; ON DELETE SET NULL
created_at	DATETIME(3)	NOT NULL	Creation timestamp
updated_at	DATETIME(3)	NOT NULL	Most recent update timestamp

Table 4.4. Document Routes Table Design

Stores each handoff, recipient branch, requested action, instruction, and status transition in chronological order.

Field Name	**Data Type**	**Key / Constraint**	**Description**
id	VARCHAR(64)	PK, NOT NULL	Unique route-step identifier
document_id	VARCHAR(64)	FK documents.id, NOT NULL	Parent document; ON DELETE CASCADE
step_number	INT	NOT NULL	Ordered step/branch number
from_division	VARCHAR(20)	NOT NULL	Sending division
from_user_id	VARCHAR(64)	FK users.id, NULL	Stable sender identity; ON DELETE SET NULL
from_user	VARCHAR(150)	NOT NULL	Sender display snapshot
to_division	VARCHAR(20)	NOT NULL	Receiving division
to_user	VARCHAR(150)	NULL	Recipient display snapshot
to_user_id	VARCHAR(64)	FK users.id, NULL	Stable recipient identity; ON DELETE SET NULL
action_requested	VARCHAR(255)	NOT NULL	Requested document action

remarks	TEXT	NULL	Dispatch note, handoff instruction, return reason, or final instruction
status_before	VARCHAR(50)	NOT NULL	Lifecycle status before the step
status_after	VARCHAR(50)	NOT NULL	Lifecycle status resulting from the step
created_at	DATETIME(3)	NOT NULL	Route/action timestamp

Table 4.5. Document Attachments Table Design

Stores document- and recipient-scoped file metadata and optional persistent file data.

Field Name	**Data Type**	**Key / Constraint**	**Description**
id	VARCHAR(64)	PK, NOT NULL	Unique attachment identifier
document_id	VARCHAR(64)	FK documents.id, NOT NULL	Parent document; ON DELETE CASCADE
file_name	VARCHAR(255)	NOT NULL	Stored/original filename
file_size	VARCHAR(50)	NULL	Display file size
file_type	VARCHAR(100)	NULL	MIME or supported file type
upload_date	DATETIME(3)	NOT NULL	Upload timestamp
file_data	LONGTEXT	NULL	Base64/file data fallback for supported deployments
attachment_scope	VARCHAR(20)	NULL	DOCUMENT or RECIPIENT visibility scope
uploaded_by_user_id	VARCHAR(64)	Logical user reference, NULL	User who uploaded the file
uploaded_for_route_id	VARCHAR(64)	Indexed route reference, NULL	Route/recipient step for confidential visibility

Table 4.6. Audit Logs Table Design

Preserves accountable authentication, document, routing, user, and administrative activity.

Field Name	Data Type	Key / Constraint	Description
id	VARCHAR(64)	PK, NOT NULL	Unique audit record
timestamp	DATETIME(3)	NOT NULL	Date and time of event
user_id	VARCHAR(64)	NOT NULL	Actor identifier
user_name	VARCHAR(150)	NOT NULL	Actor display snapshot
user_role	VARCHAR(50)	NOT NULL	Actor role
action	VARCHAR(100)	NOT NULL	LOGIN, CREATE_DOC, ROUTE_DOC, UPDATE_STATUS, UPDATE_USER, DELETE_DOC, or related event
document_tracking_number	VARCHAR(100)	NULL	Affected route number
details	TEXT	NOT NULL	Human-readable event details
ip_address	VARCHAR(45)	NULL	Recorded IPv4/IPv6 address

Table 4.7. Envelope Logs Table Design

Stores preparation and dispatch events for physical outgoing envelopes separately from electronic routing history.

Field Name	Data Type	Key / Constraint	Description
id	VARCHAR(64)	PK, NOT NULL	Unique envelope-log record
timestamp	DATETIME(3)	NOT NULL	Preparation/dispatch time
user_id	VARCHAR(64)	NOT NULL	Responsible user identifier
user_name	VARCHAR(150)	NOT NULL	Responsible user display name
user_role	VARCHAR(50)	NOT NULL	Responsible user role
action	VARCHAR(100)	NOT NULL	Envelope preparation/dispatch action

document_tracking_number	VARCHAR(100)	NULL	Linked document route number
details	TEXT	NOT NULL	Recipient, subject, format, and dispatch context
ip_address	VARCHAR(45)	NULL	Recorded network address

Table 4.8. Employee Profiles Table Design

Stores BLGF, treasury-office, LGU, and partner-agency directory and personnel-record information.

Field Name	Data Type	Key / Constraint	Description
id	VARCHAR(64)	PK, NOT NULL	Unique personnel profile
user_id	VARCHAR(64)	UNIQUE FK users.id, NULL	Optional linked system account; ON DELETE SET NULL
full_name	VARCHAR(255)	NOT NULL	Personnel name
position	VARCHAR(255)	NOT NULL	Position/designation
office	VARCHAR(255)	NOT NULL	Office/agency
office_type	VARCHAR(50)	NOT NULL	BLGF, MUNICIPAL_TREASURER, PROVINCIAL_TREASURER, LGU, or OTHER_AGENCIES
division_code	VARCHAR(20)	NULL	BLGF division where applicable
email	VARCHAR(150)	NOT NULL	Contact email
contact_no	VARCHAR(50)	NULL	Contact number
address	TEXT	NULL	Office/postal address
active	BOOLEAN	NOT NULL, default TRUE	Directory visibility state
created_at	DATETIME(3)	NOT NULL	Profile creation timestamp
folders	JSON	NULL	Personnel folders, file metadata, and protected/static-folder flags

Table 4.9. Directory Sections Table Design

Stores configurable Office Directory group titles and their office-type membership.

Field Name	Data Type	Key / Constraint	Description
id	VARCHAR(64)	PK, NOT NULL	Directory-section identifier

label	VARCHAR(255)	NOT NULL	Displayed section title
office_types	JSON	NOT NULL	Office types included in the section
color	VARCHAR(50)	NULL	Interface accent/color setting

Table 4.10. System Settings Table Design

Stores portal-wide configuration values independently from user accounts.

Field Name	**Data Type**	**Key / Constraint**	**Description**
setting_key	VARCHAR(64)	PK, NOT NULL	Unique setting name
setting_value	TEXT	NULL	Serialized/configured value such as login background
updated_at	DATETIME(3)	NOT NULL	Last modification timestamp

Table 4.11. Notifications Application Store Design

Stores user-specific route alerts and reminder popups in synchronized application state/JSON storage.

Field Name	**Data Type**	**Key / Constraint**	**Description**
id	String	PK/logical identifier	Unique notification
userId	String	Required user reference	Intended recipient
title	String	Required	Notification heading
message	String	Required	Displayed route/reminder message
documentId	String	Optional document reference	Related transaction
trackingNumber	String	Optional	Displayed route number
type	Enum	Required	INFO, URGENT, ACTION_REQUIRED, or SUCCESS
requiresDecision	Boolean	Optional, default false for reminder	Separates reminder from approval/disapproval
decisionStatus	Enum	Optional	APPROVED or DISAPPROVED when applicable

reminderSenderName	String	Optional	Person who sent reminder
reminderActionRequested	String	Optional	Pending requested action
reminderHandlerName	String	Optional	Current responsible handler
createdAt	ISO date-time	Required	Creation timestamp

4.3 The Extent of Compliance of the Developed System to ISO 25010 Software Quality Standards

The numerical findings below are based on the digital questionnaire actually administered to the respondents and reproduced in Appendix D.2. Appendix D.1 preserves the originally supplied instrument without alteration for documentary reference.

Survey Results

The ISO/IEC 25010 evaluation was completed by twenty-three (23) respondents composed of thirteen (13) BLGF employees and ten (10) IT experts. All respondents voluntarily agreed to participate and rated all 27 individual quality indicators. The weighted mean was interpreted using the scale in Chapter III: 4.21–5.00, Very Great Extent; 3.41–4.20, Great Extent; 2.61–3.40, Moderate Extent; 1.81–2.60, Little Extent; and 1.00–1.80, No Extent.

Table 4.12. Distribution of ISO/IEC 25010 Evaluators

Respondent Type	Frequency	Percentage
BLGF Employee	13	56.52%
IT Expert	10	43.48%
Total	23	100.00%

The respondent group combined the operational perspective of BLGF employees with the technical assessment of IT experts. Of the 23 respondents, 13 (56.52%) were BLGF employees and 10 (43.48%) were IT experts.

Table 4.13. Functional Suitability Evaluation

Individual Indicator	5	4	Weighted Mean	Interpretation
The system provides the required registering, routing, monitoring, retrieving, and archiving functions.	23	0	5.00	Very Great Extent

The system produces correct and appropriate document-processing information.	22	1	4.96	Very Great Extent
The available functions help users complete tasks and address workflow problems.	23	0	5.00	Very Great Extent
The system applies appropriate functions and permissions for each authorized role.	22	1	4.96	Very Great Extent
Category Mean			4.98	Very Great Extent

The functional suitability category obtained a mean of 4.98, interpreted as Very Great Extent. Two indicators received perfect means of 5.00: provision of the required document-management functions and support for completing workflow tasks. Correct information output and role-appropriate functions both obtained 4.96.

Table 4.14. Reliability Evaluation

Individual Indicator	**5**	**4**	**Weighted Mean**	**Interpretation**
The system performs consistently during normal document-processing operations.	22	1	4.96	Very Great Extent
The system remains usable when recoverable errors occur.	22	1	4.96	Very Great Extent
The system preserves or restores data and transaction history after interruption or failure.	23	0	5.00	Very Great Extent
Category Mean			4.97	Very Great Extent

Reliability obtained a category mean of 4.97. Preservation or restoration of data and transaction history received a perfect mean of 5.00, while consistent operation and usability during recoverable errors both obtained 4.96.

Table 4.15. Usability Evaluation

Individual Indicator	**5**	**4**	**Weighted Mean**	**Interpretation**
The user interface is easy to understand and learn.	22	1	4.96	Very Great Extent
Labels, instructions, notifications, and generated information are clear.	22	1	4.96	Very Great Extent

The user documentation provides sufficient operating guidance.	23	0	5.00	Very Great Extent
Users can complete common tasks through straightforward steps.	23	0	5.00	Very Great Extent
Category Mean			4.98	Very Great Extent

Usability achieved a category mean of 4.98. Documentation sufficiency and straightforward task completion each obtained 5.00. Interface learnability and clarity of system information each obtained 4.96.

Table 4.16. Performance Efficiency Evaluation

Individual Indicator	**5**	**4**	**Weighted Mean**	**Interpretation**
The system responds promptly to commands, searches, and transactions.	22	1	4.96	Very Great Extent
The system uses available computing and network resources efficiently.	21	2	4.91	Very Great Extent
The system supports the expected users and transactions without unacceptable slowdown.	23	0	5.00	Very Great Extent
Category Mean			4.96	Very Great Extent

Performance efficiency obtained a category mean of 4.96. Capacity for the expected workload received 5.00. Efficient use of computing and network resources received 4.91, indicating a minor opportunity for continued performance monitoring.

Table 4.17. Compatibility Evaluation

Individual Indicator	**5**	**4**	**Weighted Mean**	**Interpretation**
The system performs correctly in supported browsers, devices, and environments.	23	0	5.00	Very Great Extent
The modules exchange and use document, routing, notification, and user information correctly.	21	2	4.91	Very Great Extent
Category Mean			4.96	Very Great Extent

Compatibility obtained a category mean of 4.96. Correct operation in supported environments received 5.00, while information exchange among modules received 4.91.

Table 4.18. Security Evaluation

Individual Indicator	5	4	Weighted Mean	Interpretation
The system restricts functions and records according to roles and permissions.	23	0	5.00	Very Great Extent
The system protects data, attachments, credentials, and records from unauthorized access or modification.	20	3	4.87	Very Great Extent
The system provides traceable records of significant user and document actions.	21	2	4.91	Very Great Extent
Category Mean			4.93	Very Great Extent

Security obtained a category mean of 4.93, the lowest among the eight characteristics but still interpreted as Very Great Extent. Role-based restriction received 5.00. Protection from unauthorized access or modification received 4.87, the lowest individual mean, indicating that security controls should continue to be reviewed, tested, and strengthened.

Table 4.19. Maintainability Evaluation

Individual Indicator	5	4	Weighted Mean	Interpretation
System components and errors can be analyzed and corrected efficiently.	23	0	5.00	Very Great Extent
The system can be modified without unnecessarily affecting unrelated functions.	23	0	5.00	Very Great Extent
The system continues to operate correctly after controlled changes.	23	0	5.00	Very Great Extent
System functions can be tested to verify requirements.	23	0	5.00	Very Great Extent
Category Mean			5.00	Very Great Extent

Maintainability ranked highest with a perfect category mean of 5.00. Every respondent assigned the highest rating to all four indicators, showing strong agreement regarding analyzability, modifiability, stability after controlled changes, and testability.

Table 4.20. Portability Evaluation

Individual Indicator	5	4	Weighted Mean	Interpretation
The system can operate in another compatible environment with minimal adjustment.	23	0	5.00	Very Great Extent
The system can be installed or deployed using manageable procedures.	21	2	4.91	Very Great Extent
The system can adapt to supported hardware, software, or configuration changes.	22	1	4.96	Very Great Extent
The system can replace relevant manual document-tracking activities within scope.	23	0	5.00	Very Great Extent
Category Mean			4.97	Very Great Extent

Portability obtained a category mean of 4.97. Operation in another compatible environment and replacement of relevant manual activities each received 5.00. Installation or deployment procedures obtained 4.91, suggesting that deployment documentation should remain current and easy to follow.

Table 4.21. Summary of ISO/IEC 25010 Compliance Evaluation

Quality Characteristic	Category Mean	Interpretation	Rank
Maintainability	5.00	Very Great Extent	1
Functional Suitability	4.98	Very Great Extent	2.5
Usability	4.98	Very Great Extent	2.5
Reliability	4.97	Very Great Extent	4.5
Portability	4.97	Very Great Extent	4.5
Performance Efficiency	4.96	Very Great Extent	6.5
Compatibility	4.96	Very Great Extent	6.5
Security	4.93	Very Great Extent	8
Grand Mean	4.97	Very Great Extent	

The overall ISO/IEC 25010 evaluation produced a grand mean of 4.97, interpreted as Very Great Extent. Maintainability ranked first with 5.00, followed by functional suitability and usability at 4.98. Security had the lowest category mean at 4.93; nevertheless, it remained within the Very Great Extent range. The findings answer the study objective by showing that the developed WDTMP complies with the evaluated ISO/IEC 25010 quality characteristics to a very great extent.

No qualitative comments, strengths, limitations, or recommendations were entered in the open-ended fields of the submitted response file. Consequently, the

qualitative discussion is limited to the numerical ratings and does not infer comments that were not provided.

CHAPTER V

SUMMARY OF FINDINGS AND RECOMMENDATIONS

5.1 Summary of Findings

The study found that the previous manual and fragmented workflow made it difficult to determine the current holder, required action, processing time, attachment context, and complete history of a document. Physical routing slips and separate records increased the risk of delayed follow-up, duplicate encoding, misplaced files, inconsistent route-number references, and limited accountability.

Functional verification of the developed system established that it provides the following implemented capabilities:

1. A secured authentication process with active-account checking, temporary-password setup, password recovery, role-based menus, and granular action permissions.

2. A dashboard with access-aware incoming, outgoing, pending, in-progress, completed, returned, workload, and recent-activity information.

3. Incoming and outgoing registration with an automatically generated unique route number, classification, sender and recipient information, priority, due date, attachments, Register Only, and Register and Route.

4. Immediate visibility of Register Only transactions under Recent Document Activity and the appropriate document list with Not Yet Routed status. Issued route-number sequences remain retired after cancellation or deletion and are not reused.

5. Searchable and filterable document monitoring with lifecycle progress, current status, division, handler, dates, and permission-controlled actions.

6. Single- and multiple-recipient routing, transfer, receipt/action timestamps, approval or disapproval where required, returned/on-hold/for-signature states, reply attachments, final instructions, and continued routing after an earlier completion.

7. A visual routing flow in which every recipient has a separate branch and aligned connector. Dispatch notes, replies, attachments, and completion belong to the proper user branch. End - Transaction Completed appears only for the participant who actually ended the transaction.

8. Recipient-specific privacy for notes and route attachments. The sender, intended recipient, and authorized System Administrator can inspect the relevant exchange; unrelated recipients do not receive confidential step files. Identical initial notes for several recipients are presented as one grouped dispatch note that identifies all intended recipients.

9. Step Conversation and Attachments showing the sender, recipient, time, instruction, final response, and permitted files as an understandable transaction conversation.

10. Routing Follow-up that lists every incomplete recipient separately, orders items by time in status, and sends an editable in-app reminder. A reminder does not approve, disapprove, route, or complete a document.

11. Official routing-slip selection, customization, signatory formats, print layouts, and document-linked routing activity.

12. An Outgoing Envelope Studio with document linking, individual or directory-based batch recipients, return-address presets, standard/custom dimensions, envelope or cut-out printing, reusable templates, and dispatch logging.

13. QR creation for custom values and route-number ranges, including preview, size, recent-code history, download, copy, and print functions.

14. Incoming, outgoing, and envelope reports with filtering, complete history, printing, Excel export, and supported monthly folder auto-export.

15. An Office Directory covering BLGF personnel, treasury offices, and partner agencies, together with configurable sections, linked/protected user profiles, personnel folders, uploads, title editing, and PDF preview.

16. Detailed audit and envelope-dispatch logs with actor, role, event, route number, details, timestamp, pagination, filtering, Excel/CSV export, and optional monthly auto-save.

17. Administrator management of personnel accounts, protected System Administrator identities, password reset, status, role/menu/action permissions, divisions, login background, and account settings.

18. Strict role separation verified in the running system: the System Administrator can monitor all transactions and perform authorized configuration or correction but cannot perform operational approval, transfer, multi-route, or completion. A least-privilege STAFF account sees only its assigned menus and related transactions.

The ISO/IEC 25010 evaluation was completed by 23 respondents: 13 BLGF employees and 10 IT experts. The individual-indicator results produced category means of 4.98 for functional suitability, 4.97 for reliability, 4.98 for usability, 4.96 for performance efficiency, 4.96 for compatibility, 4.93 for security, 5.00 for maintainability, and 4.97 for portability. The grand mean was 4.97, interpreted as

Very Great Extent. Maintainability ranked highest, while security ranked lowest but remained within the highest interpretation range. These findings establish that the WDTMP complied with all eight evaluated ISO/IEC 25010 characteristics to a very great extent.

5.2 Conclusions

The developed Web-Based Document Tracking and Records Management System satisfies the system-development objective by centralizing BLGF Region II document registration, routing, monitoring, attachments, reminders, reporting, personnel records, and accountability controls in one role-aware application.

The system improves transaction clarity because every handoff is connected to an identifiable sender, recipient, requested action, instruction, status transition, and timestamp. Separate recipient branches, scoped notes and attachments, final instructions, and accurate completion markers make the routing history easier to understand while protecting information from unrelated users.

The permanent route-number sequence, immutable user-ID relationships, chronological routing history, audit logs, and controlled continuation after completion strengthen traceability. Dashboard summaries, follow-up reminders, searchable lists, reports, routing slips, envelopes, and QR tools reduce dependence on handwritten logs and personal follow-up.

Role-based access successfully differentiates System Administrator monitoring and configuration from the operational responsibilities of regular users. Based on the grand mean of 4.97, the developed system complies with the evaluated ISO/IEC 25010 quality characteristics to a Very Great Extent and is technically capable of supporting the intended BLGF Region II workflow. Continued security review, deployment testing, user orientation, backup validation, and monitoring remain necessary to preserve this level of quality during actual organizational use.

5.3 Recommendations

1. Sustain the system qualities rated highly by the BLGF employees and IT experts, and periodically repeat the ISO/IEC 25010 evaluation after significant functional, security, infrastructure, or deployment changes.

2. Adopt written BLGF Region II procedures for Register Only, Register and Route, multi-recipient routing, recipient-specific notes and files, transfer, return, final instruction, completion, continued routing, reminder use, correction, and deletion.

3. Retain the separation of duties in which the System Administrator monitors, configures, and performs authorized corrections but does not participate in operational approval or routing through the administrator account.

4. Apply least privilege when assigning menus and granular actions. Review permissions after personnel transfer, reassignment, separation, or change in responsibility, and deactivate obsolete access promptly.

5. Preserve permanent route-number progression and audit history. Incorrect transactions should be cancelled or corrected through traceable actions rather than by reusing a previously issued number.

6. Require concise recipient-specific instructions and proper attachment classification. Confidential or branch-specific files should remain within the intended conversation and must not be duplicated in broad completion summaries.

7. Review Routing Follow-up regularly for overdue and incomplete recipient actions. Use reminders only as follow-up notices and retain their separation from approval, disapproval, and completion functions.

8. Back up the MySQL database together with document attachments, personnel folders, and relevant application configuration. Perform and document restoration tests at scheduled intervals.

9. Provide role-specific user orientation covering registration, routing, action recording, privacy, completion, reports, slips, envelopes, QR tools, directory records, and incident reporting.

10. Before production deployment and after material updates, conduct functional regression, security, upload, compatibility, performance, accessibility, backup/recovery, and permission-boundary tests.

11. Future enhancements may include approved digital signatures, retention/disposition scheduling, encrypted off-site backup, management analytics, and integration with other authorized government systems, subject to privacy, security, records-management, and budget requirements.

BIBLIOGRAPHY

Express.js. (n.d.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. https://expressjs.com/

International Organization for Standardization. (2011). *Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models* (ISO/IEC Standard No. 25010:2011). https://www.iso.org/standard/35733.html

International Organization for Standardization. (2016). *Information and documentation—Records management—Part 1: Concepts and principles* (ISO Standard No. 15489-1:2016). https://www.iso.org/standard/62542.html

National Archives of the Philippines. (n.d.). *Frequently asked questions*. https://nationalarchives.gov.ph/frequently-asked-question

OpenJS Foundation. (n.d.). *Node.js documentation*. https://nodejs.org/docs/latest/api/

Oracle Corporation. (n.d.). *MySQL reference manual*. https://dev.mysql.com/doc/

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw Hill.

Angular Team. (n.d.). Angular documentation. https://angular.dev/

Republic Act No. 8792. (2000). *Electronic Commerce Act of 2000*. Official Gazette of the Republic of the Philippines. https://www.officialgazette.gov.ph/2000/06/14/republic-act-no-8792-s-2000/

Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. Official Gazette of the Republic of the Philippines. https://www.officialgazette.gov.ph/2012/08/15/republic-act-no-10173/

Republic Act No. 11032. (2018). *Ease of Doing Business and Efficient Government Service Delivery Act of 2018*. Official Gazette of the Republic of the Philippines. https://www.officialgazette.gov.ph/2018/05/28/republic-act-no-11032/

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

APPENDICES

Appendix A. Request Letter for Gathering Data

Republic of the Philippines
Cagayan State University
CARIG CAMPUS
Carig Sur, Tuguegarao City
Tel. No. (078) 304 – 0818

Graduate School

February 11, 2026

ATTY. JULAIDA T. CADDAWAN-PANCHO
Regional Director
Bureau of Local Government Finance
Tuguegarao City, Cagayan

RECEIVED
11 FEB 2026

Ma'am:

The undersigned is a student of the Graduate School taking up Master of Science in Information Technology at Cagayan State University, Carig Campus.

As a requirement of the course the undersigned would like to request permission to conduct my study entitled **"Web-Based Document Tracking System for Monitoring, Routing, and Managing Organizational Documents"** in your organization.

Rest assured that the data gathered will be solely used for my thesis. I would be very grateful for your favorable action on this matter

Thank you very much and more power.

Respectfully yours,

Tom-tom G. Catral
Student, MSIT

Noted by:

Edison D. Bravo
Adviser

Further Noted by:

MANNY R. CAPILI
Program Coordinator

Recommending Approval:

DENNIS BACUYAG, Ph.D
Dean – Graduate School

Appendix B. Request for Pilot Testing

August _, 2026

May I respectfully request permission to conduct pilot testing of the Web-Based Document Tracking and Records Management System with selected authorized personnel. The activity will involve task-based use of a controlled test environment and completion of an ISO/IEC 25010 questionnaire. Testing will use non-sensitive or expressly authorized sample data. Participation will be voluntary, and responses will be summarized without unnecessary personal identifiers.

Proposed schedule: August _, 2026, 8:00 o'clock in the morning at the Bureau of Local Government Finance Regional Office 02, Tuguegarao City, Cagayan,

Respectfully yours,

TOM TOM G. CATRAL
Researcher

Appendix C. User's Manual

User Manual for the Web-Based Document Tracking and Management Platform

1. Introduction

1.1 Purpose of the Manual

This manual provides concise procedures for the main WDTMP functions. Screenshots are paired with the corresponding instructions for quick reference.

1.2 Intended Users

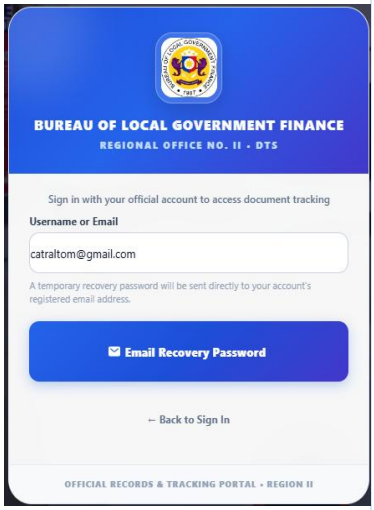
This manual is intended for administrators, records personnel, document originators, recipients, approvers, and other authorized BLGF employees. Available functions depend on account permissions.

1.3 Scope of the System

It covers access, account settings, registration, monitoring, routing, reminders, QR and print outputs, reports, directory records, administration, and troubleshooting.

C.1 Accessing the System

Figure C.1. User Manual - Login And Password Recovery



Login And Password Recovery Procedure

1. Enter the registered username and password, then select Log In.
2. If access is lost, select Forgot Password, submit the registered account, verify the recovery credential, and set a new password.

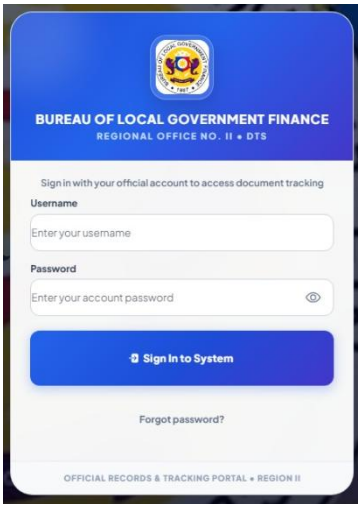


Figure C.2. User Manual - Account Menu

Account Menu
Procedure

1. Open the account menu.
2. Select Install Application when available; otherwise use the browser Install App or Add to Home Screen command.
3. Confirm the installation on desktop or mobile, then sign in normally.

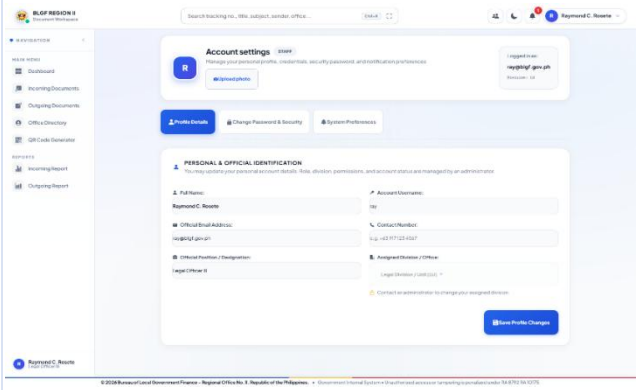


Figure C.3. User Manual - Account Settings

Account Settings
Procedure

1. Open Account Settings, edit only the permitted profile or password fields, and select Save.
2. Role and permissions cannot be elevated from this screen.

C.2 Registering a Document

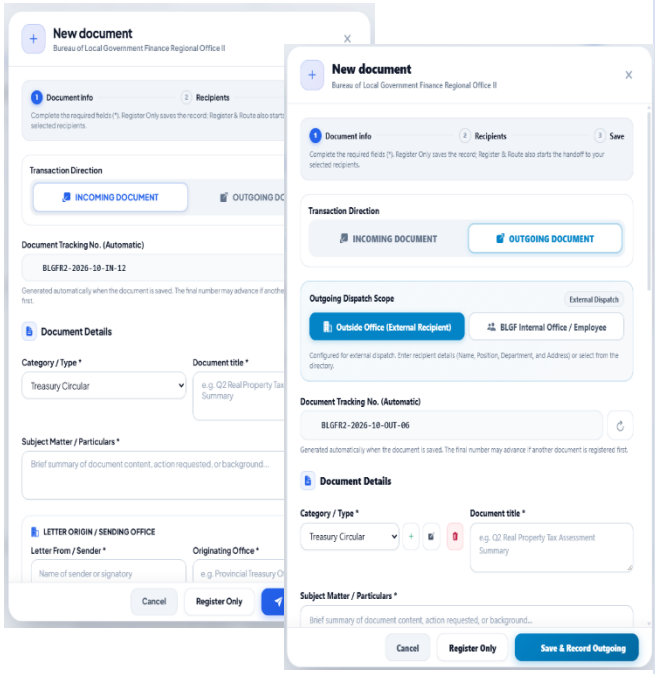


Figure C.4. User Manual - Log New Document Form

Log New Document Form Procedure

1. Select Log New Document and choose Incoming or Outgoing.
2. Complete the required document information, recipient, requested action, instruction, target date, and permitted file.
3. Select Register Only to save as Not Yet Routed, or Register and Route to start the workflow.

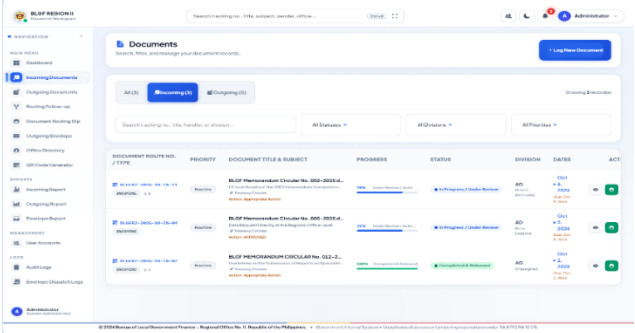


Figure C.5. User Manual - Incoming Documents

Incoming Documents Procedure

1. Open Incoming Documents.
2. Search or filter the list, then select View for the required transaction.
3. The Administrator can see all records for monitoring.

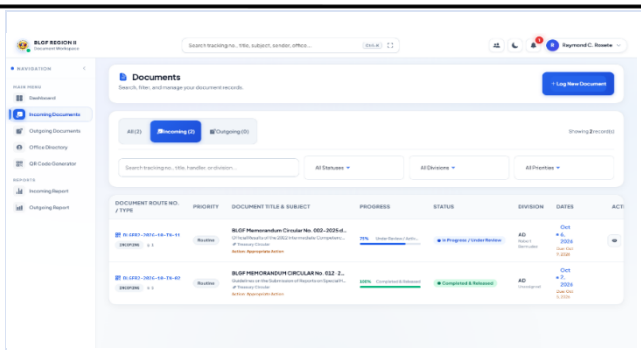


Figure C.6. User Manual - Incoming Documents

Incoming Documents Procedure

1. Open Incoming Documents to see transactions received by or otherwise related to the user.
2. Search or filter, then select View.

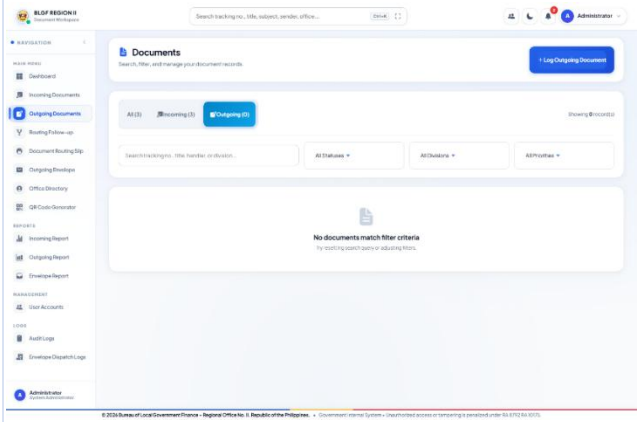


Figure C.7. User Manual - Outgoing Documents

Outgoing Documents Procedure

Open Outgoing Documents to monitor registered outgoing records, including internal BLGF employee and external recipients.

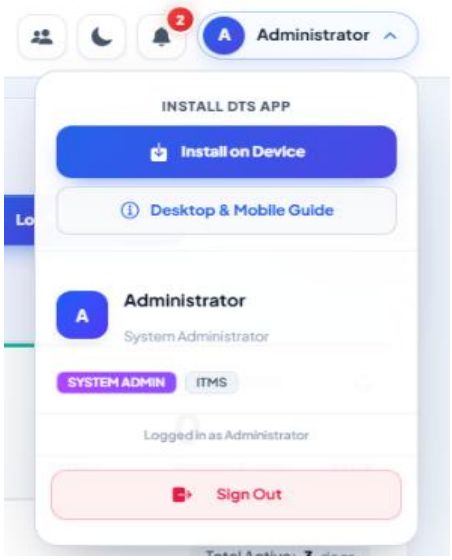
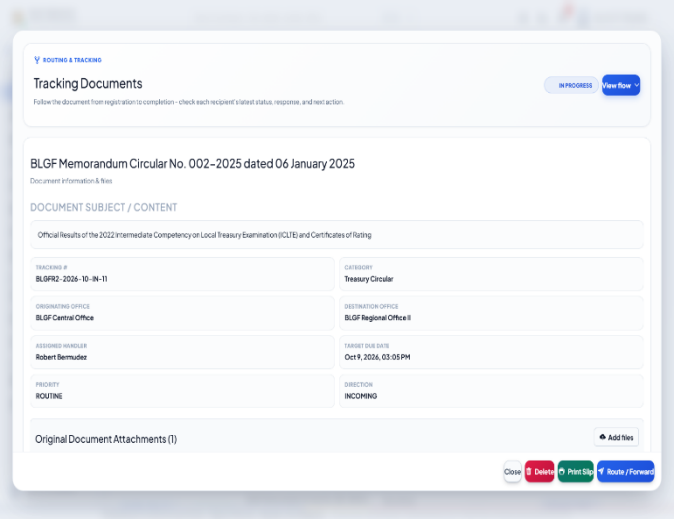


Figure C.8. User Manual - Outgoing Documents

Outgoing Documents Procedure

1. Open Outgoing Documents to see records created, sent, received, or assigned to the user.
2. Select the required route number.



C.3 Routing and Acting on a Document

Refer to the document details screen.

Figure C.9. User Manual - Document Details And Recipient Flow

Document Details And Recipient Flow

Procedure

1. Open a route number and review the document summary.
2. Select one recipient branch to inspect its sender, instruction, status, conversation, and allowed files.

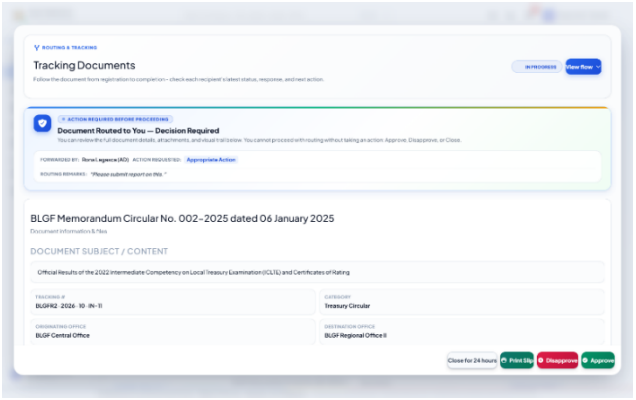


Figure C.10. User Manual - Document Details Conversation And Attachments

Document Details Conversation And Attachments Procedure

1. In Step Conversation and Attachments, read the chronological notes and files for the selected branch.
2. Attachments remain visible only to the participants allowed by their scope.

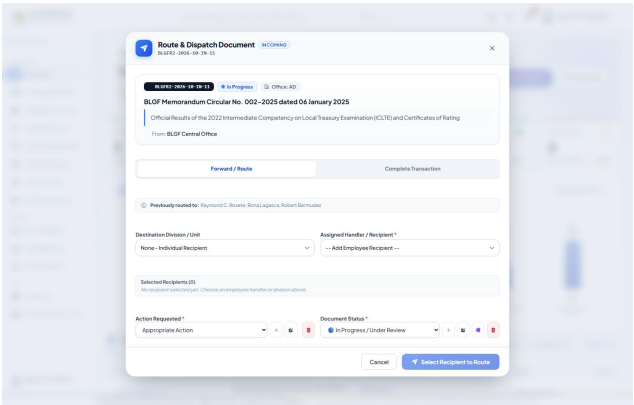


Figure C.11. User Manual - Route Forward And Reply Form

Route Forward And Reply Form Procedure

1. Select Route/Forward or Step Reply.
2. Choose the recipient, requested action, and status; enter the recipient-specific note; optionally attach a permitted file; then submit.

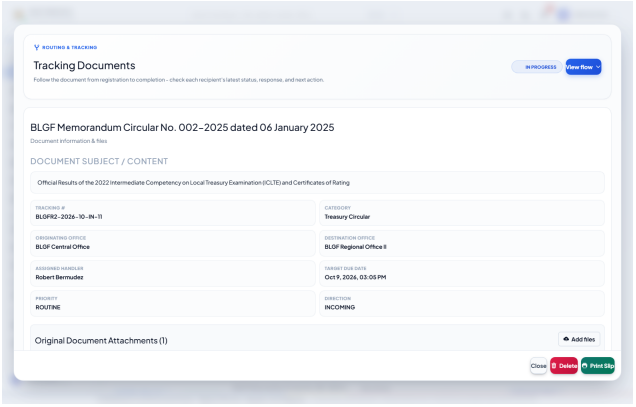


Figure C.12. User Manual - Document Details And Routing Flow

Document Details And Routing Flow

Procedure

1. Use the routing flow to monitor each independent recipient branch.
2. Final Instruction and End - Transaction Completed appear only under the user who completed that branch.
3. Select Add Route or Continue Routing when another endorsement is required.

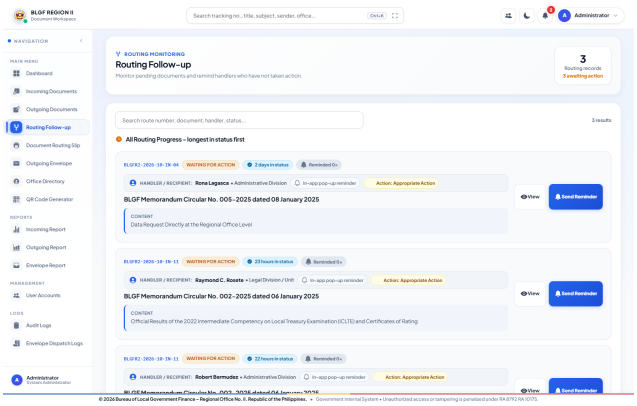


Figure C.13. User Manual - Routing Follow Up

Routing Follow Up

Procedure

1. Open Routing Follow-up to see every incomplete recipient branch, ordered by time in status.
2. Select View to inspect the transaction or Send Reminder for follow-up.

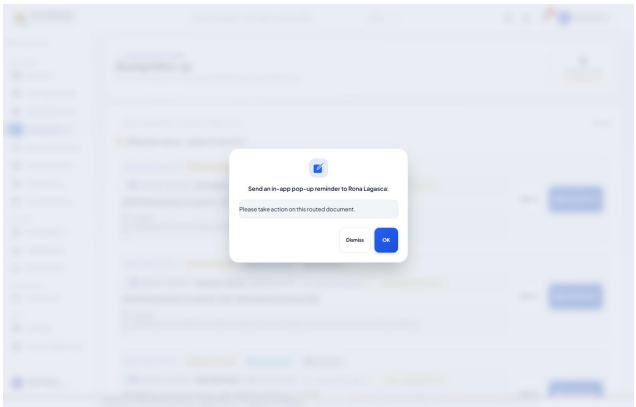


Figure C.14. User Manual - Send Reminder Dialog

Send Reminder Dialog

Procedure

1. Edit the reminder note, confirm the intended recipient and route number, and select Send Reminder.
2. The message appears as an in-app alert only and does not perform an approval action.

C.4 Search, Notifications, QR, and Slips

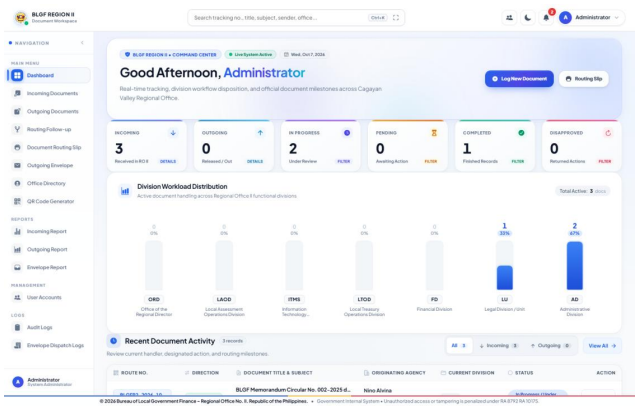


Figure C.15. User Manual - Dashboard

Dashboard
Procedure
1. Review organization-wide totals, workload, status summaries, and Recent Document Activity.
2. Use monitoring and edit tools only; the Administrator does not route, approve, or complete a transaction.

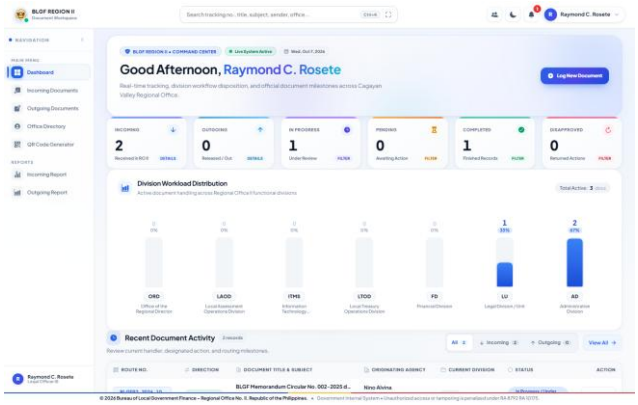


Figure C.16. User Manual - Dashboard

Dashboard
Procedure
1. Review the documents and recent activity related to the signed-in user.
2. Select a permitted route number to open its details.

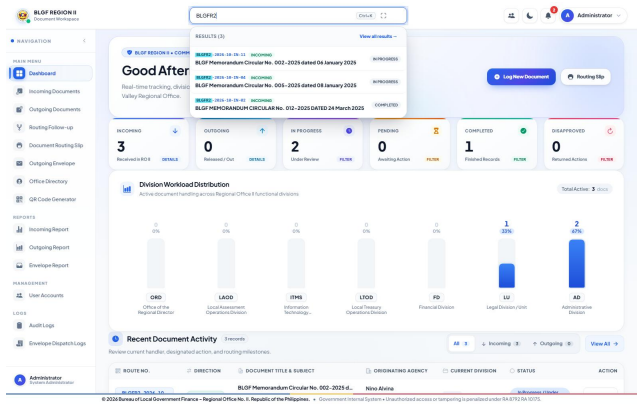


Figure C.17. User Manual - Global Document Search

Global Document Search
Procedure
1. Enter a route number, title, subject, sender, or office.
2. Select the matching permitted result to view the document.

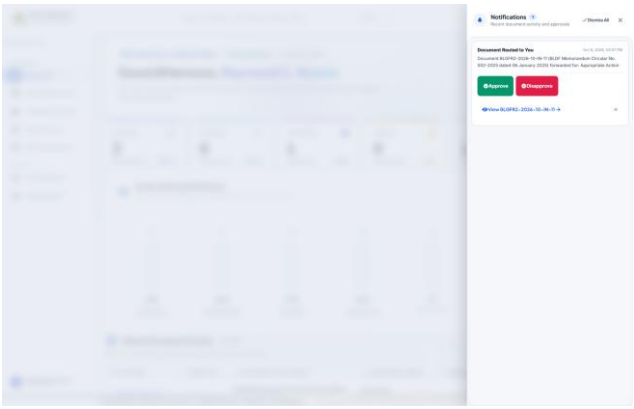


Figure C.18. User Manual - Notification Drawer

Notification Drawer Procedure

1. Open Notifications and select a routing or reminder alert.
2. A reminder only displays the sender message and does not approve or complete the document.

3.4 QR Code and Official Print Outputs

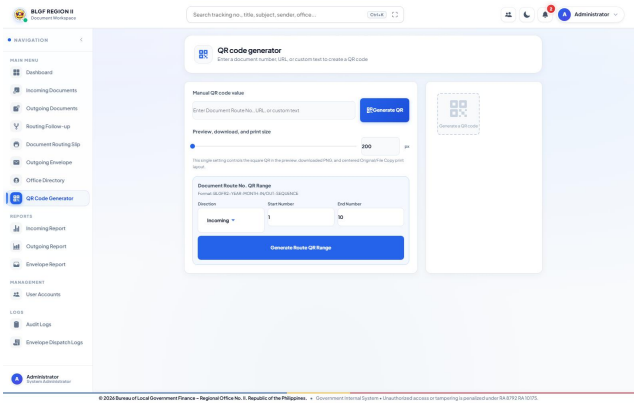


Figure C.19. User Manual - Qr Code Generator

Qr Code Generator Procedure

Enter or select the verified route number, choose the QR size, generate the preview, and use Download, Copy, or Print.

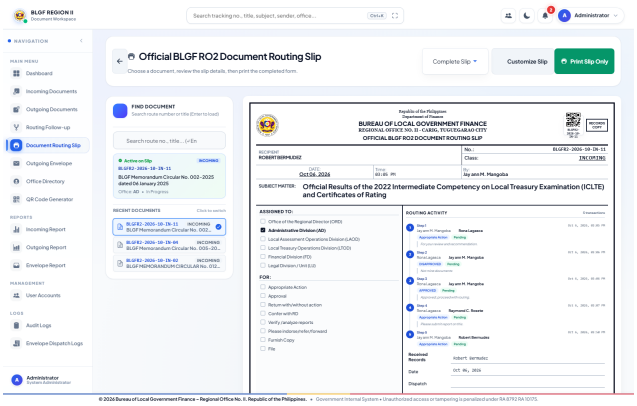


Figure C.20. User Manual - Document Routing Slip

Document Routing Slip Procedure

Select the document, verify the metadata and complete routing history, then preview and print the routing slip.

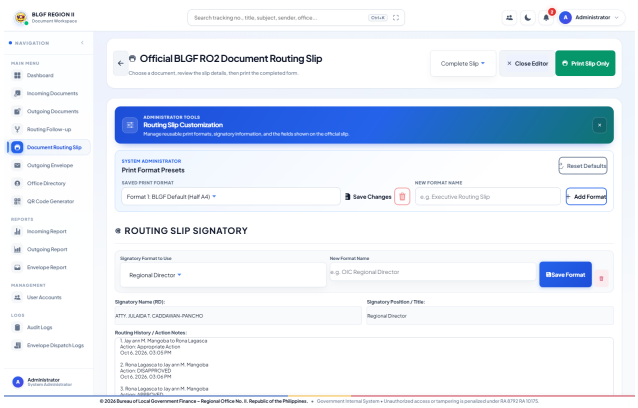


Figure C.21. User Manual - Routing Slip Customization

Routing Slip Customization Procedure

Choose the approved layout or signatory preset, verify the preview, and save the print settings before printing.

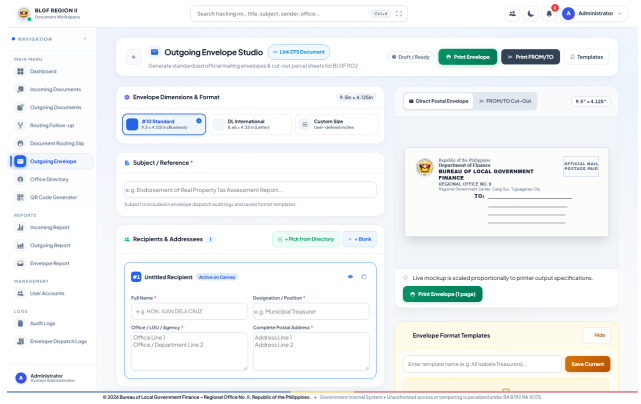


Figure C.22. User Manual - Outgoing Envelope

Outgoing Envelope Procedure

Link the outgoing document, verify the recipient and return address, select the envelope or FROM/TO cut-out format, preview, print, and save the dispatch record.

3.5 Reports and Completed Transactions

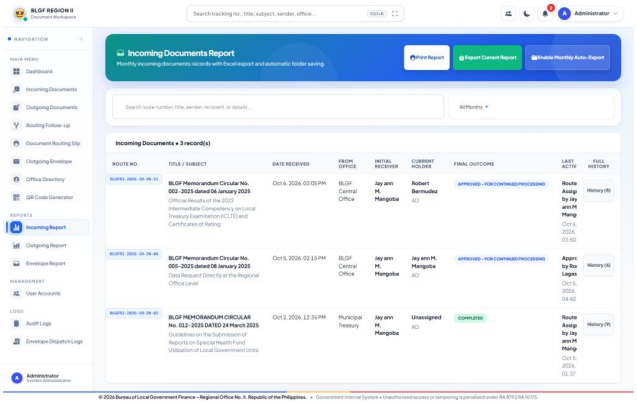


Figure C.23. User Manual - Incoming Report

Incoming Report Procedure

1. Open Incoming Report, set the date and status filters, and select Completed when completed transactions are required.
2. Expand route history before printing or exporting.

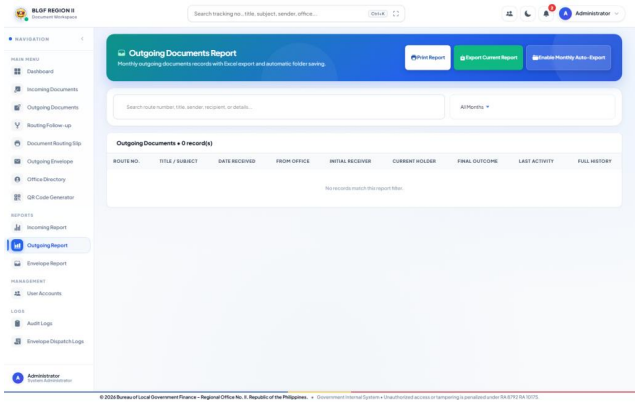


Figure C.24. User Manual - Outgoing Report

Outgoing Report Procedure

Open Outgoing Report, apply the required date/status filters, review each outcome and route history, then print or export.

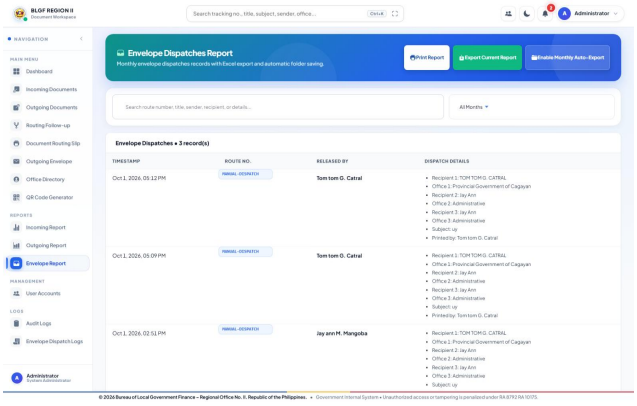


Figure C.25. User Manual - Envelope Report

Envelope Report Procedure

1. Open Envelope Report to review saved physical-dispatch records separately from electronic routing.
2. Filter, print, or export as required.

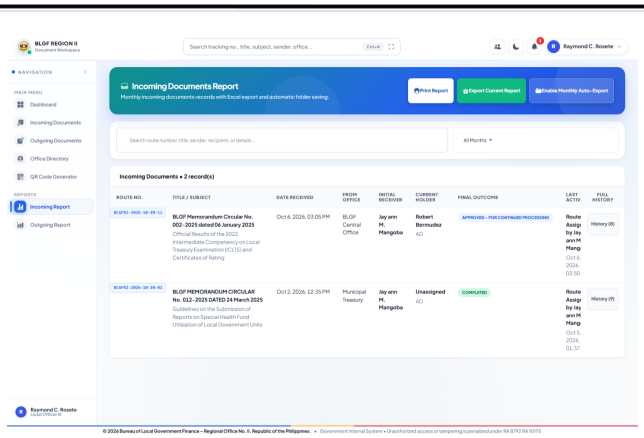


Figure C.26. User Manual - Incoming Report

Incoming Report Procedure

Open Incoming Report to review and export only the incoming records visible to the signed-in user.

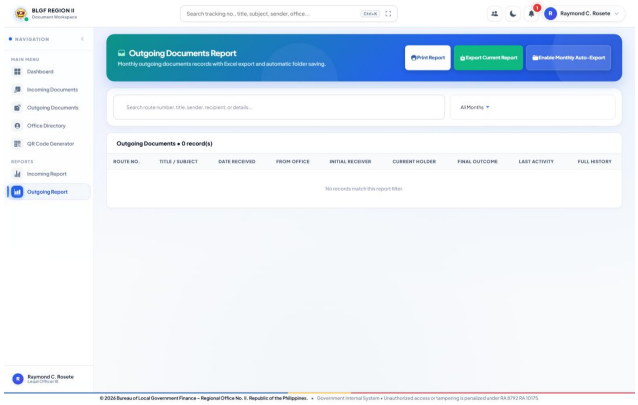


Figure C.27. User Manual - Outgoing Report

Outgoing Report Procedure

Open Outgoing Report to review and export only the outgoing records visible to the signed-in user.

C.5 Administrative Functions

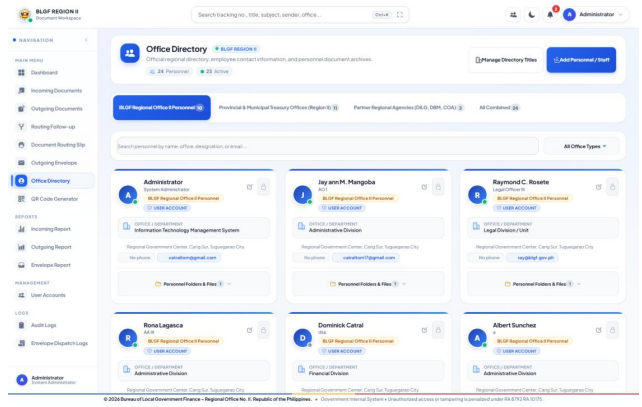


Figure C.28. User Manual - Office Directory

Office Directory Procedure

1. Open Office Directory to search personnel and organizations.
2. Authorized administrators may add or edit directory records; other users receive view-only access when granted.

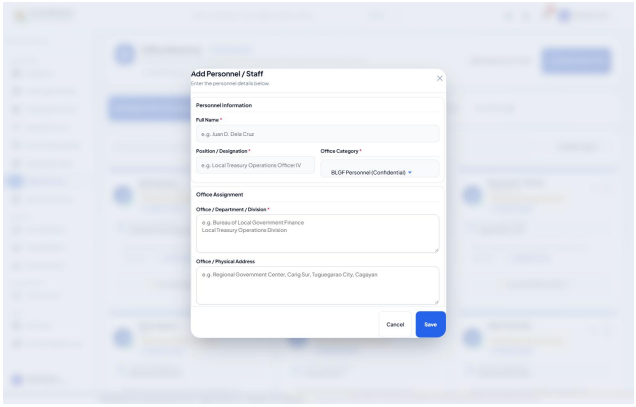


Figure C.29. User Manual - Add Personnel Form

Add Personnel Form Procedure

Select Add Personnel, complete the official identity, office, position, contact, division, and account-link fields, then save.

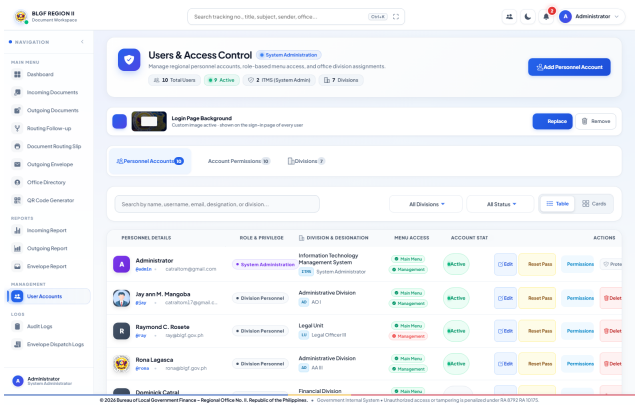


Figure C.30. User Manual - User Accounts

User Accounts Procedure

Open User Accounts to search, create, edit, activate, deactivate, or reset permitted accounts.

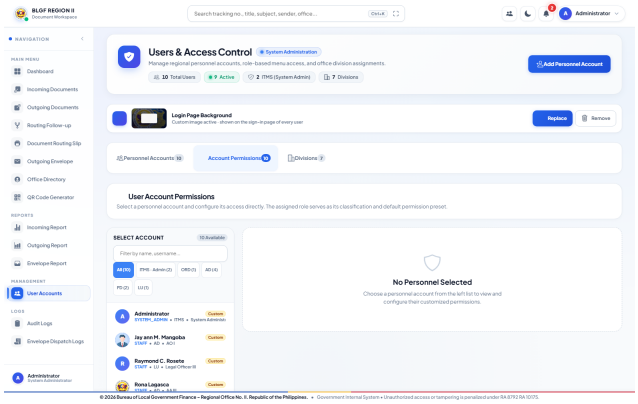


Figure C.31. User Manual - Role And Account Permissions

Role And Account Permissions Procedure

1. Select an account, assign the proper role defaults, review each menu/action permission, and save.
2. Use least privilege.

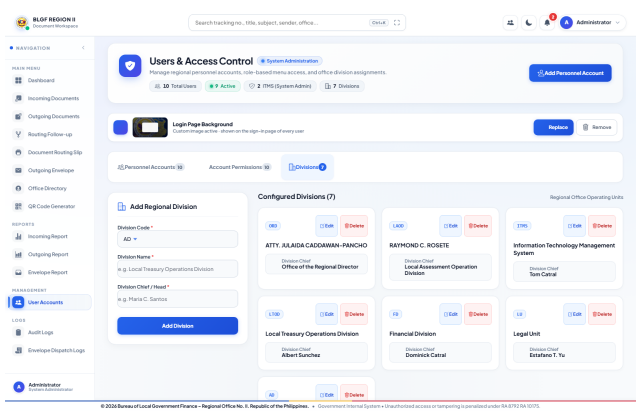


Figure C.32. User Manual - Division Management

Division Management Procedure

Open Division Management, add or edit the official code, name, chief, and email, then save.

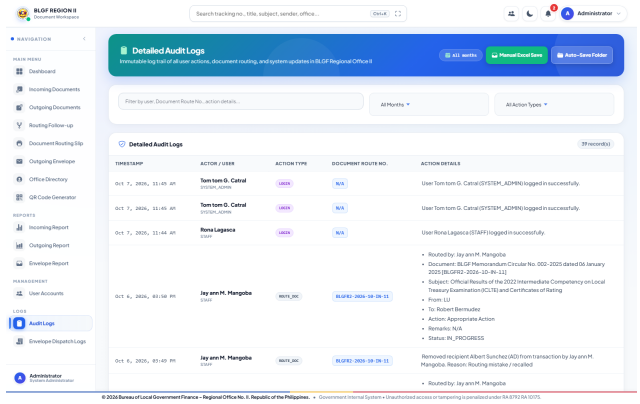


Figure C.33. User Manual - Audit Logs

Audit Logs Procedure

1. Open Audit Logs, apply user, action, document, or date filters, and export when required.
2. Logs are read-only accountability records.

C.6 Basic Troubleshooting

Steps to follow:

1. If a permitted record is missing, refresh once and verify direction, status, and date filters.
2. Confirm that the account is a creator, sender, recipient, assignee, or authorized monitor.
3. If an action is unavailable, check the branch status and permission before attempting again.
4. For upload or print issues, verify the file type, file size, route number, paper size, and browser print destination.
5. Record the route number, time, screen, and attempted action before reporting a persistent problem.

Appendix D.1 Original ISO Questionnaire Provided by the Adviser (Verbatim)

The following questionnaire is reproduced exactly as provided. Its wording, sequence, scale, repeated item, and formatting content have not been substantively changed.

Survey-Questionnaire

Software Effectiveness Questionnaire of the System using ISO/IEC 25010:2011 or ISO 9126 Quality Standards

(This survey-questionnaire is designed to determine the efficiency of the _____).

Part I. Profile of the Respondents
Kindly fill-in the blanks with required information or by simply putting a (√) check mark.

Age: __ below 30 years old
__ 31 – 40 years old
__ 41 – 50 years old
__ 51 – 60 years old
__ above 60 years old

Sex: ___ Male ___ Female
Highest Educational Attainment:
College ___
Post-Graduate ___

Part II. Assessment of the Extent of Compliance to ISO 25010:2011 Software Quality

Instruction: Kindly indicate by putting a check mark (√) in the appropriate box your assessment on the extent of compliance to ISO 25010:2011 of the developed *web-based performance measurement system*in terms of Functionality, Reliability, Usability, Efficiency, Maintainability, Portability and security using the following scale:

5 – Compliant to a very high extent
4 – Compliant to a high extent
3 – Compliant to a moderate extent
2 – Compliant to a low extent
- – Compliant to a very low extent

• **Functionality**	**5**	**4**	**3**	**2**	**1**	**Comments**
1. The required task is performed by the system satisfactorily						
2. The output generated by the system is correct.						

3. The system helps the user overcome any problem encountered while using it.						
4. The system regulates user access.						
• **Reliability**	**5**	**4**	**3**	**2**	**1**	**Comments**
5. Most errors have been corrected over time.						
6. The system is capable of maintaining its performance despite the presence of errors during operations.						
7. The system provides restore operation of lost data after a failure.						
• **Usability**	**5**	**4**	**3**	**2**	**1**	**Comments**
8. The user interface is intuitive and receptive to learning.						
9. The information generated by the system is easy to understand and act upon.						
10. The documentation is very informative.						
11. The system performs tasks in a straightforward manner.						
• **Efficiency**	**5**	**4**	**3**	**2**	**1**	**Comments**
12. The system responds immediately to user commands other input.						
13. The system maximizes the utilization of available resources (i.e. people, data, machines, and materials) efficiently.						
14. The system performs efficiently on intended tasks with ease and scalability.						

• **Maintainability**	**5**	**4**	**3**	**2**	**1**	**Comments**
15. Deficiencies can be easily diagnosed and modified from the system.						
16. The system can be easily customized and modified.						
17. The system continues functioning if changes are made.						
18. The system can easily be tested.						

F. Portability	**5**	**4**	**3**	**2**	**1**	**Comments**
19. The system performs its functions efficiently even it is moved to another environment.						
20. The system can be installed easily.						
21. The system can be installed easily.						
22. The system complies with the portability standards.						
23. The system can take the place of other software.						
• **Compatibility**	**5**	**4**	**3**	**2**	**1**	**Comments**
24. The system can perform its required functions efficiently						
25. The degree to which the system can exchange information and use the information that has been exchange,						
• **Security**	**5**	**4**	**3**	**2**	**1**	**Comments**
26. The system has its security level access with pre-determined action.						
27. The data is secured as well as its associated records.						
28. Access to top-level management data is topped with necessary security credentials.						

Source: ISO 25010:2011 Software Quality Model

Kindly list below YOUR PERCEIVED "BEST FEATURES OR STRENGTH" of the developed system.

- _____
- _____
- _____
- _____
- _____

Kindly list below YOUR PERCEIVED "LIMITATIONS OF THE SYSTEM NEEDED TO BE ADDRESSED".

- _____
- _____
- _____
- _____
- _____

Suggestions:

Appendix D.2 Digital Questionnaire Actually Administered

SOFTWARE QUALITY EVALUATION QUESTIONNAIRE
Web-Based Document Tracking and Management Platform (WDTMP)
Bureau of Local Government Finance – Regional Office No. 2

Purpose: This questionnaire evaluates the extent to which the developed WDTMP complies with the ISO/IEC 25010:2011 software product quality characteristics. Please use the system before answering. Responses will be summarized for academic purposes and treated confidentially.

PART I. RESPONDENT PROFILE
Respondent Type: ☐ BLGF Employee ☐ IT Expert
Age: ☐ 30 or below ☐ 31–40 ☐ 41–50 ☐ 51–60 ☐ Above 60
Sex: ☐ Male ☐ Female ☐ Prefer not to say
Highest Educational Attainment: ☐ College ☐ Postgraduate ☐ Other: _____
Years of Relevant Work/IT Experience: ☐ Below 1 year ☐ 1–5 years ☐ 6–10 years ☐ More than 10 years

PART II. ISO/IEC 25010 SOFTWARE QUALITY ASSESSMENT
Instruction: Place a check mark (✓) under the rating that best represents your assessment of each statement.
5 – Very Great Extent 4 – Great Extent 3 – Moderate Extent 2 – Little Extent 1 – No Extent

No.	Quality Characteristic / Evaluation Statement	5	4	3	2	1	Comments
1	A. Functional Suitability The system provides the functions required for registering, routing, monitoring, retrieving, and archiving documents.	☐	☐	☐	☐	☐	
2	The system produces correct and appropriate information for document-processing tasks.	☐	☐	☐	☐	☐	

3	The available functions help users complete document-related tasks and address workflow problems.	☐	☐	☐	☐	☐	
4	The system applies the appropriate functions and access permissions for each authorized user role.	☐	☐	☐	☐	☐	
5	B. Reliability The system performs consistently during normal document-processing operations.	☐	☐	☐	☐	☐	
6	The system remains usable when recoverable errors occur.	☐	☐	☐	☐	☐	
7	The system preserves or restores document data and transaction history after an interruption or failure.	☐	☐	☐	☐	☐	
8	C. Usability The user interface is easy to understand and learn.	☐	☐	☐	☐	☐	
9	Labels, instructions, notifications, and system-generated information are clear.	☐	☐	☐	☐	☐	
10	The user documentation provides sufficient guidance for operating the system.	☐	☐	☐	☐	☐	
11	Users can complete common tasks through a straightforward sequence of steps.	☐	☐	☐	☐	☐	
12	D. Performance Efficiency The system responds promptly to user commands, searches, and document transactions.	☐	☐	☐	☐	☐	
13	The system uses available computing and network resources efficiently.	☐	☐	☐	☐	☐	
14	The system can support the expected number of users and document transactions without unacceptable slowdown.	☐	☐	☐	☐	☐	
15	E. Compatibility The system performs correctly in its supported browsers, devices, and operating environment.	☐	☐	☐	☐	☐	
16	The system modules exchange and use document, routing, notification, and user information correctly.	☐	☐	☐	☐	☐	

17	F. Security The system restricts functions and records according to authorized user roles and permissions.	☐	☐	☐	☐	☐	
18	The system protects document data, attachments, credentials, and associated records from unauthorized access or modification.	☐	☐	☐	☐	☐	
19	The system provides traceable records of significant user and document-processing actions.	☐	☐	☐	☐	☐	
20	G. Maintainability System components and errors can be analyzed and corrected efficiently.	☐	☐	☐	☐	☐	
21	The system can be modified or enhanced without unnecessarily affecting unrelated functions.	☐	☐	☐	☐	☐	
22	The system continues to operate correctly after controlled changes are introduced.	☐	☐	☐	☐	☐	
23	System functions can be tested to verify that requirements are satisfied.	☐	☐	☐	☐	☐	
24	H. Portability The system can operate in another compatible deployment environment with minimal adjustment.	☐	☐	☐	☐	☐	
25	The system can be installed or deployed using clear and manageable procedures.	☐	☐	☐	☐	☐	
26	The system can adapt to supported changes in hardware, software, or configuration.	☐	☐	☐	☐	☐	
27	The system can replace the relevant manual document-tracking activities within the defined scope of the study.	☐	☐	☐	☐	☐	

PART III. OPEN-ENDED ASSESSMENT

1. Best features or strengths of the system:

_____

_____

2. Limitations or areas that need improvement:

_____

_____

3. Suggestions and recommendations:

_____

Source: Adapted from the ISO/IEC 25010:2011 software product quality model.

Appendix E. Source Code

The following listings are copied from the source tree that passed npm run lint and npm run build. Line ranges are stated for long feature files. The complete electronic repository remains the authoritative executable source.

E.1 Build Configuration and Dependencies

Source file: package.json

// ----- lines 1-0 -----
// ----- lines 55-0 -----

E.2 Angular Client Dependencies

Source file: frontend/package.json

// ----- lines 1-0 -----
// ----- lines 40-0 -----

E.3 Prisma/MySQL Data Model

Source file: prisma/mysql/schema.prisma

// ----- lines 1-0 -----
// ----- lines 182-0 -----

E.4 Session and Identity Management

Source file: frontend/src/app/services/session.service.ts

// ----- lines 1-0 -----
// ----- lines 78-0 -----

E.5 API Service for Documents, Routes, Notifications, Reports, and Administration

Source file: frontend/src/app/services/api.service.ts

// ----- lines 1-220 -----

```
  1: import { Injectable, inject } from '@angular/core';
  2: import { HttpClient } from '@angular/common/http';
  3: import { Observable } from 'rxjs';
  4: import { map } from 'rxjs/operators';
  5: import {
  6:   DashboardStats,
  7:   Division,
  8:   DocumentAttachment,
  9:   DocumentRecord,
 10:   EmployeeProfile,
 11:   NotificationItem,
```

```
  12:   User,
  13: } from '../types';
  14:
  15: export type StorageKind =
  16:   | 'documentAttachments'
  17:   | 'profilePictures'
  18:   | 'loginBackground';
  19:
  20: const configuredApiBase = String(
  21:   (window as unknown as { __BLGF_API_BASE__?: string }).__BLGF_API_BASE__
?? '',
  22: ).replace(/\/$/, '');
  23: const API_BASE = configuredApiBase || '/api';
  24:
  25: @Injectable({ providedIn: 'root' })
  26: export class ApiService {
  27:   private http = inject(HttpClient);
  28:
  29:   private get<T>(url: string, params?: Record<string, string | number>):
Observable<T> {
  30:     return this.http.get<T>(`${API_BASE}${url}`, {
  31:       params: params as Record<string, string>,
  32:     });
  33:   }
  34:
  35:   private post<T>(url: string, body?: unknown, _params?: Record<string,
string>, headers?: Record<string, string>): Observable<T> {
  36:     return this.http.post<T>(`${API_BASE}${url}`, body, headers ? { headers } :
undefined);
  37:   }
  38:
  39:   private put<T>(url: string, body?: unknown, headers?: Record<string,
string>): Observable<T> {
  40:     return this.http.put<T>(`${API_BASE}${url}`, body, headers ? { headers } :
undefined);
  41:   }
  42:
  43:   private patch<T>(url: string, body?: unknown): Observable<T> {
  44:     return this.http.patch<T>(`${API_BASE}${url}`, body);
  45:   }
  46:
```

```
  private delete<T>(url: string, headers?: Record<string, string>): Observable<T> {
    return this.http.delete<T>(`${API_BASE}${url}`, headers ? { headers } : undefined);
  }

  readonly apiBase = API_BASE;

  // ---------------------------------------------------------------
  // Storage
  // ---------------------------------------------------------------
  uploadToStorage(kind: StorageKind, file: File): Observable<{ fileName: string; url: string; fileData?: string }> {
    const formData = new FormData();
    formData.append('file', file);
    return this.http.post<{ fileName: string; url: string; fileData?: string }>(
      `${API_BASE}/storage/upload/${kind}`,
      formData,
    );
  }

  getLoginBackground(): Observable<{ url: string | null }> {
    return this.get<{ url: string | null }>('/settings/login-background');
  }

  saveLoginBackground(url: string | null): Observable<{ url: string | null }> {
    return this.put<{ url: string | null }>('/settings/login-background', { url });
  }

  // ---------------------------------------------------------------
  // Auth
  // ---------------------------------------------------------------
  login(username: string, password: string): Observable<{
    user: User;
    mustChangePassword?: boolean;
  }> {
    return this.post<{ user: User; mustChangePassword?: boolean }>('/auth/login', {
      username,
      password,
    });
  }
```

```
85:
86:   // Finishes a temporary-password login by storing the user's own password.
87:   completePasswordSetup(
88:     username: string,
89:     temporaryPassword: string,
90:     newPassword: string,
91:   ): Observable<{ user: User }> {
92:     return this.post<{ user: User }>('/auth/complete-password-setup', {
93:       username,
94:       temporaryPassword,
95:       newPassword,
96:     });
97:   }
98:
99:   forgotPassword(identifier: string): Observable<{
100:     message: string;
101:     temporaryPassword?: string;
102:     username?: string;
103:     targetEmail?: string;
104:     emailSent?: boolean;
105:     success?: boolean;
106:     notice?: string;
107:     expiresInMinutes?: number;
108:   }> {
109:     return this.post('/auth/forgot-password', { identifier });
110:   }
111:
112:   forgotAdminPassword(identifier: string): Observable<{
113:     message: string;
114:     temporaryPassword?: string;
115:     username?: string;
116:     targetEmail?: string;
117:     emailSent?: boolean;
118:     success?: boolean;
119:     notice?: string;
120:     expiresInMinutes?: number;
121:   }> {
122:     return this.forgotPassword(identifier);
123:   }
124:
125:   // ---------------------------------------------------------------
126:   // Stats
```

```
  // ---------------------------------------------------------------
  getStats(): Observable<DashboardStats> {
    return this.get<DashboardStats>('/stats');
  }

  // ---------------------------------------------------------------
  // Documents
  // ---------------------------------------------------------------
  getDocuments(params?: {
    search?: string;
    direction?: string;
    status?: string;
    division?: string;
    priority?: string;
  }): Observable<DocumentRecord[]> {
    return this.get<DocumentRecord[]>('/documents', params);
  }

  getDocumentById(id: string): Observable<DocumentRecord> {
    return this.get<DocumentRecord>(`/documents/${id}`);
  }

  getNextRouteNumber(direction: string): Observable<{ routeNo: string }> {
    return this.get<{ routeNo: string }>('/documents/next-route-number', {
direction });
  }

  createDocument(data: Partial<DocumentRecord> & { userId?: string;
userRole?: string }): Observable<DocumentRecord> {
    return this.post<DocumentRecord>('/documents', data);
  }

  routeDocument(
    documentId: string,
    data: {
      fromDivision: string;
      fromUser: string;
      toDivision: string;
      toUser?: string;
      toUserId?: string;
      actionRequested: string;
      remarks?: string;
```

```
    newStatus?: string;
    actingUserId?: string;
    actingUserName?: string;
    actingUserRole?: string;
  },
): Observable<DocumentRecord> {
  return this.post<DocumentRecord>(`/documents/${documentId}/route`, data);
}

decideDocumentRoute(
  documentId: string,
  decision: 'APPROVED' | 'DISAPPROVED',
  remarks?: string,
  actingUserId?: string,
): Observable<DocumentRecord> {
  const headers = actingUserId ? { 'X-User-Id': actingUserId } : undefined;
  return this.post<DocumentRecord>(
    `/documents/${documentId}/decision`,
    { decision, remarks },
    {},
    headers,
  );
}

transferDocument(
  documentId: string,
  data: {
    fromDivision: string;
    fromUser: string;
    toDivision: string;
    toUser?: string;
    toUserId?: string;
    transferReason: string;
    actingUserId?: string;
    actingUserName?: string;
    actingUserRole?: string;
  },
): Observable<DocumentRecord> {
  return this.post<DocumentRecord>(`/documents/${documentId}/transfer`, data);
}
```

```
  207:
  208:   multiRouteDocument(
  209:     documentId: string,
  210:     data: {
  211:       fromDivision: string;
  212:       fromUser: string;
  213:       targetDivisions: string[];
  214:       recipients?: { toDivision: string; toUser: string; toUserId: string }[];
  215:       actionRequested: string;
  216:       remarks?: string;
  217:       newStatus?: string;
  218:       actingUserId?: string;
  219:       actingUserName?: string;
  220:       actingUserRole?: string;
 // ----- lines 300-436 -----
  300:   }
  301:
  302:   deleteUser(id: string): Observable<{ success: boolean }> {
  303:     return this.delete<{ success: boolean }>(`/users/${id}`);
  304:   }
  305:
  306:   resetUserPassword(
  307:     id: string,
  308:     actingUserId?: string,
  309:   ): Observable<{
  310:     success: boolean;
  311:     message: string;
  312:     targetEmail?: string;
  313:     username?: string;
  314:   }> {
  315:     const headers = actingUserId ? { 'X-User-Id': actingUserId } : undefined;
  316:     return this.post<{
  317:       success: boolean;
  318:       message: string;
  319:       targetEmail?: string;
  320:       username?: string;
  321:     }>(`/users/${id}/reset-password`, {}, headers);
  322:   }
  323:
  324:   // ---------------------------------------------------------
  325:   // Divisions
  326:   // ---------------------------------------------------------
```

```
  getDivisions(): Observable<Division[]> {
    return this.get<Division[]>('/divisions');
  }

  createDivision(data: Omit<Division, 'id'>): Observable<Division> {
    return this.post<Division>('/divisions', data);
  }

  updateDivision(id: string, data: Partial<Division>): Observable<Division> {
    return this.put<Division>(`/divisions/${id}`, data);
  }

  deleteDivision(id: string): Observable<{ success: boolean }> {
    return this.delete<{ success: boolean }>(`/divisions/${id}`);
  }

  // ---------------------------------------------------------
  // Audit logs
  // ---------------------------------------------------------
  getAuditLogs(): Observable<import('../types').AuditLog[]> {
    return this.get<import('../types').AuditLog[]>('/audit-logs');
  }

  createAuditLog(data: Partial<import('../types').AuditLog>):
Observable<import('../types').AuditLog> {
    return this.post<import('../types').AuditLog>('/audit-logs', data);
  }

  getEnvelopeLogs(): Observable<import('../types').AuditLog[]> {
    return this.get<import('../types').AuditLog[]>('/envelope-logs');
  }

  createEnvelopeLog(data: Partial<import('../types').AuditLog>):
Observable<import('../types').AuditLog> {
    return this.post<import('../types').AuditLog>('/envelope-logs', data);
  }

  // ---------------------------------------------------------
  // Notifications
  // ---------------------------------------------------------
  getNotifications(userId?: string): Observable<NotificationItem[]> {
    return this.get<NotificationItem[]>(
```

```
    userId ? `/notifications?userId=${encodeURIComponent(userId)}` : '/notifications',
    );
  }

  deleteNotification(id: string): Observable<{ success: boolean }> {
    return this.delete<{ success: boolean }>(`/notifications/${id}`);
  }

  sendRoutingReminder(data: {
    documentId: string;
    recipientUserId: string;
    message: string;
    actionRequested?: string;
  }): Observable<
    NotificationItem & {
      recipientName?: string;
    }
  > {
    return this.post<
      NotificationItem & {
        recipientName?: string;
      }
    >('/notifications/reminder', data);
  }

  // ---------------------------------------------------------
  // Employees
  // ---------------------------------------------------------
  getEmployees(): Observable<EmployeeProfile[]> {
    return this.get<EmployeeProfile[]>('/employees');
  }

  createEmployee(data: Partial<EmployeeProfile>): Observable<EmployeeProfile> {
    return this.post<EmployeeProfile>('/employees', data);
  }

  updateEmployee(id: string, data: Partial<EmployeeProfile>): Observable<EmployeeProfile> {
    return this.put<EmployeeProfile>(`/employees/${id}`, data);
  }
```

```
  406:
  407:   deleteEmployee(id: string): Observable<{ success: boolean }> {
  408:     return this.delete<{ success: boolean }>(`/employees/${id}`);
  409:   }
  410:
  411:   // ---------------------------------------------------------------
  412:   // Directory Sections & Titles
  413:   // ---------------------------------------------------------------
  414:   getDirectorySections(): Observable<any[]> {
  415:     return this.get<any[]>('/directory-sections');
  416:   }
  417:
  418:   saveDirectorySections(sections: any[]): Observable<{ success: boolean;
directorySections: any[] }> {
  419:     return this.post<{ success: boolean; directorySections: any[] }>('/directory-
sections', sections);
  420:   }
  421:
  422:   // ---------------------------------------------------------------
  423:   // Record storage (document-level attachments carried as base64)
  424:   // ---------------------------------------------------------------
  425:   attachmentPayload(file: File): Observable<Pick<DocumentAttachment,
'fileName' | 'fileSize' | 'fileType' | 'url' | 'fileData'>> {
  426:     return this.uploadToStorage('documentAttachments', file).pipe(
  427:       map((result) => ({
  428:         fileName: result.fileName,
  429:         fileSize: (file.size / (1024 * 1024)).toFixed(2),
  430:         fileType: file.type,
  431:         url: result.url || '',
  432:         fileData: result.fileData,
  433:       })),
  434:     );
  435:   }
  436: }
```

E.6 Document and Attachment Visibility Rules

Source file: frontend/src/app/utils/document-visibility.ts

```
// ----- lines 1-0 -----
// ----- lines 67-0 -----
```

E.7 Recipient-Scoped Attachment and Flow Rules

Source file: frontend/src/app/utils/attachment-visibility.ts

// ----- lines 1-0 -----
// ----- lines 11-0 -----

E.8 Recipient Flow Construction

Source file: frontend/src/app/utils/recipient-flow.ts

// ----- lines 1-0 -----
// ----- lines 30-0 -----

E.9 Registration Form Submission Logic

Source file: frontend/src/app/components/modals/create-document/create-document.component.ts

// ----- lines 820-1040 -----

```
820:         .map((profile) => (profile.position || '').trim())
821:         .filter(Boolean),
822:     ),
823:   ]);
824:   recipientOfficeSuggestions = computed(() => [
825:     ...new Set(
826:       this.directoryProfiles()
827:         .map((profile) => (profile.office || '').trim())
828:         .filter(Boolean),
829:     ),
830:   ]);
831:   recipientAddressSuggestions = computed(() => [
832:     ...new Set(
833:       this.directoryProfiles()
834:         .map((profile) => (profile.address || '').trim())
835:         .filter(Boolean),
836:     ),
837:   ]);
838:
839:   private loadDocumentCategories(): ManagedOption[] {
840:     const DEFAULTS = [
841:       'Treasury Circular',
842:       'Real Property Tax Assessment',
843:       'Financial Report',
844:       'Legal Opinion',
845:       'Personnel Memo',
846:       'General Correspondence',
847:     ].map((label) => ({ value: label, label }));
848:     try {
```

```
    const raw = localStorage.getItem('blgf_document_categories');
    if (!raw) return DEFAULTS;
    const parsed: unknown = JSON.parse(raw);
    if (!Array.isArray(parsed)) return DEFAULTS;
    const cats = parsed
      .map((item) => {
        if (typeof item === 'string') {
          const label = item.trim();
          return label ? { value: label, label } : null;
        }
        if (!item || typeof item !== 'object') return null;
        const opt = item as { label?: unknown; value?: unknown };
        const label = typeof opt.label === 'string' ? opt.label.trim() : '';
        const value = typeof opt.value === 'string' ? opt.value.trim() : '';
        return label || value ? { value: value || label, label: label || value } :
null;
      })
      .filter((item): item is ManagedOption => item !== null)
      .filter((item, index, all) => all.findIndex((candidate) => candidate.value
=== item.value) === index);
    return cats.length ? cats : DEFAULTS;
  } catch {
    return DEFAULTS;
  }
}

getEmployeeName(id: string): string {
  return (
    this.allEmployees().find((u) => u.id === id)?.fullName || id
  );
}

onTargetDateChange(event: Event): void {
  this.targetCompletionDate.set(
    (event.target as HTMLInputElement).value,
  );
}

onPriorityChange(value: string): void {
  this.priority.set(value as PriorityLevel);
}
```

```
  priorityBadgeClasses(value: string): string {
    switch (value) {
      case 'VERY_URGENT':
        return 'priority-very-urgent';
      case 'URGENT':
        return 'priority-urgent';
      case 'CONFIDENTIAL':
        return 'priority-confidential';
      default:
        return 'priority-routine';
    }
  }

  onDivisionChange(event: Event): void {
    const division = (event.target as HTMLSelectElement).value as
      | DivisionCode
      | 'ALL'
      | '';
    this.currentDivision.set(division);
    if (division === 'ALL' || !division) {
      this.additionalDivisions.set([]);
    } else {
      this.additionalDivisions.set(
        this.additionalDivisions().filter((d) => d !== division),
      );
    }
    this.excludedRecipientIds.set([]);
    if (division) {
      this.selectedRecipientIds.set([]);
      this.recipientName.set('');
      this.recipientUserId.set('');
      this.recipientPosition.set('');
      this.recipientOffice.set('');
      this.recipientAddress.set('');
    }
  }

  onAddDivision(event: Event): void {
    const division = (event.target as HTMLSelectElement)
      .value as DivisionCode;
    if (
      division &&
```

```
931:       division !== this.currentDivision() &&
932:       !this.additionalDivisions().includes(division)
933:     ) {
934:       this.additionalDivisions.set([
935:         ...this.additionalDivisions(),
936:         division,
937:       ]);
938:       this.excludedRecipientIds.set([]);
939:     }
940:     this.showAddDivision.set(false);
941:   }
942:
943:   onRemoveDivision(division: DivisionCode | 'ALL', index: number): void {
944:     if (this.currentDivision() === 'ALL') {
945:       this.currentDivision.set('');
946:       this.additionalDivisions.set([]);
947:     } else if (index === 0) {
948:       const [nextPrimary, ...remaining] = this.additionalDivisions();
949:       this.currentDivision.set(nextPrimary || '');
950:       this.additionalDivisions.set(remaining);
951:     } else {
952:       this.additionalDivisions.set(
953:         this.additionalDivisions().filter((d) => d !== division),
954:       );
955:     }
956:     this.excludedRecipientIds.set([]);
957:   }
958:
959:   onRecipientSelect(event: Event): void {
960:     const userId = (event.target as HTMLSelectElement).value;
961:     const account = this.allEmployees().find((u) => u.id === userId);
962:     if (!account) {
963:       this.recipientName.set('');
964:       this.recipientUserId.set('');
965:       this.recipientPosition.set('');
966:       this.recipientOffice.set('');
967:       this.recipientAddress.set('');
968:       this.currentDivision.set('');
969:       return;
970:     }
971:     this.recipientName.set(account.fullName);
972:     this.recipientUserId.set(account.id);
```

```
    this.recipientPosition.set(account.designation || account.role);
    this.recipientOffice.set('BLGF Regional Office II');
    this.currentDivision.set(account.divisionCode);
    this.recipientAddress.set(
      'Regional Government Center, Carig Sur, Tuguegarao City',
    );
  }

  addRecipient(): void {
    const id = this.recipientUserId();
    if (!id) return;
    this.selectedRecipientIds.set([
      ...new Set([...this.selectedRecipientIds(), id]),
    ]);
    this.recipientName.set('');
    this.recipientUserId.set('');
    this.recipientPosition.set('');
    this.recipientOffice.set('');
    this.recipientAddress.set('');
  }

  removeRecipient(id: string): void {
    this.selectedRecipientIds.set(
      this.selectedRecipientIds().filter((uid) => uid !== id),
    );
  }

  excludeRecipient(id: string): void {
    this.excludedRecipientIds.set([...this.excludedRecipientIds(), id]);
  }

  async onFilePicked(event: Event): Promise<void> {
    const input = event.target as HTMLInputElement;
    const files = input.files;
    if (!files || files.length === 0) return;
    const newAtts: DocumentAttachment[] = [];
    for (const f of Array.from(files) as File[]) {
      const storedFile = await firstValueFrom(
        this.api.uploadToStorage('documentAttachments', f),
      );
      const sizeMb = (f.size / (1024 * 1024)).toFixed(2);
      newAtts.push({
```

```
1015:         id: `att-${Date.now()}-${Math.random().toString(36).substr(2, 5)}`,
1016:         fileName: f.name,
1017:         fileSize: `${sizeMb} MB`,
1018:         fileType: documentFileType(f),
1019:         uploadDate: new Date().toISOString(),
1020:         url: storedFile.url,
1021:       });
1022:     }
1023:     this.attachments.set([...this.attachments(), ...newAtts]);
1024:     input.value = '';
1025:   }
1026:
1027:   removeAttachment(id: string): void {
1028:     this.attachments.set(
1029:       this.attachments().filter((a) => a.id !== id),
1030:     );
1031:   }
1032:
1033:   private buildFormData(forceRegisterOnly = false): any {
1034:     const isOutside = this.isOutsideOffice();
1035:     const div = isOutside ? (this.currentUser.divisionCode || 'ORD') :
this.currentDivision();
1036:     const addl = isOutside ? [] : this.additionalDivisions();
1037:     const selIds = isOutside ? [] : this.selectedRecipientIds();
1038:     const holdRecips = isOutside ? [] : this.holdingDivisionRecipients();
1039:     const displayed = this.displayedRouteNo();
1040:
// ----- lines 1100-1206 -----
1100:       initialTargetDivisions:
1101:         isOutside || div === 'ALL' || isNotYetRouted
1102:           ? []
1103:           : [div, ...addl].filter(Boolean),
1104:       initialRecipientIds: isOutside || isNotYetRouted
1105:         ? []
1106:         : [
1107:             ...new Set([
1108:               ...selIds,
1109:               ...holdRecips.map((u) => u.id),
1110:             ]),
1111:           ],
1112:       targetCompletionDate: (isOutside || !this.targetCompletionDate())
1113:         ? new Date(Date.now() + 3 * 86400000).toISOString()
```

```
1114:         : new Date(this.targetCompletionDate()).toISOString(),
1115:       initialAction: isOutside
1116:         ? 'Dispatched / Recorded (Outside Office)'
1117:         : (isNotYetRouted ? (this.initialAction() || 'Awaiting Initial Routing') :
(this.initialAction() || 'Appropriate Action')),
1118:       remarks: isOutside
1119:         ? (this.remarks().trim() || 'Completed - Recorded as Outside Office
Outgoing Dispatch')
1120:         : (isNotYetRouted ? (this.remarks().trim() || 'Document registered. Not
yet routed.') : this.remarks()),
1121:       attachments: this.attachments(),
1122:       createdBy: this.currentUser.fullName,
1123:       userId: this.currentUser.id,
1124:       userRole: this.currentUser.role,
1125:       shouldPrintSlip: false,
1126:       shouldRouteModal: false,
1127:       shouldOpenEnvelope: false,
1128:       excludedRecipientIds: isOutside ? [] : this.excludedRecipientIds(),
1129:       isNotYetRouted,
1130:     };
1131:   }
1132:
1133:   handleSubmit(
1134:     event: Event,
1135:     actionType: 'none' | 'route' = 'none',
1136:   ): void {
1137:     event.preventDefault();
1138:     if (this.submitting()) return;
1139:
1140:     const isOutside = this.isOutsideOffice();
1141:     const outsideList = isOutside ? this.outsideDepartmentRecipients() : [];
1142:     const isRouting = actionType === 'route';
1143:
1144:     const hasAssignment =
1145:       Boolean(this.currentDivision()) ||
1146:       Boolean(this.recipientUserId()) ||
1147:       this.selectedRecipientIds().length > 0 ||
1148:       this.holdingDivisionRecipients().length > 0;
1149:
1150:     // If user clicked "Register & Route" without any initial recipient or division
assignment, show warning!
1151:     if (!isOutside && isRouting && !hasAssignment) {
```

```
1152:       this.routeAssignmentWarning.set(
1153:         'Please select an Initial Division or Assigned Recipient before routing.
(Or choose "Register Only" to save this document as Not Yet Routed).'
1154:       );
1155:       const assignmentPanel = document.getElementById('initial-assignment-
panel');
1156:       if (assignmentPanel) {
1157:         assignmentPanel.scrollIntoView({ behavior: 'smooth', block: 'center' });
1158:       }
1159:       return;
1160:     }
1161:
1162:     this.routeAssignmentWarning.set('');
1163:
1164:     const missingFields: string[] = [
1165:       !this.title().trim() && 'Document Title',
1166:       !this.category().trim() && 'Category / Type',
1167:       !this.subject().trim() && 'Subject Matter / Particulars',
1168:       !this.senderName().trim() && 'Letter From / Sender',
1169:       !this.originatingOffice().trim() && 'Originating Office',
1170:       !this.senderAddress().trim() && 'Office Address',
1171:       // If Outside Office: require manual recipient fields or outside chips
1172:       isOutside && !this.recipientName().trim() && outsideList.length === 0 &&
'Recipient Name / Addressee',
1173:       isOutside && !this.recipientOffice().trim() && outsideList.length === 0 &&
'Department / Office',
1174:       isOutside && !this.recipientAddress().trim() && outsideList.length === 0
&& 'Recipient Address',
1175:       // If Internal BLGF and ROUTING: require division or user assignment
1176:       !isOutside && isRouting && !hasAssignment && 'Initial Division or
Assigned Personnel',
1177:       !isOutside && !this.initialAction() && 'Action Requested',
1178:       !isOutside && !this.priority() && 'Priority Level',
1179:     ].filter(Boolean) as string[];
1180:
1181:     if (missingFields.length > 0) {
1182:       this.validationErrors.set(missingFields);
1183:       alert(
1184:         `Please complete all required fields before
proceeding:\n\n${missingFields.map((f) => `- ${f}`).join('\n')}`,
1185:       );
1186:       return;
```

```
 1187:     }
 1188:     this.validationErrors.set([]);
 1189:     this.submitting.set(true);
 1190:     const formData = this.buildFormData(!isOutside && actionType ===
'none');
 1191:     formData.shouldRouteModal =
 1192:       !isOutside &&
 1193:       actionType === 'route' &&
 1194:       !this.recipientUserId() &&
 1195:       !this.currentDivision();
 1196:     this.onSubmit(formData)
 1197:       .then(() => {
 1198:         this.validationErrors.set([]);
 1199:         this.onClose();
 1200:       })
 1201:       .catch((err) => {
 1202:         console.warn('Document registration submission error:', err);
 1203:       })
 1204:       .finally(() => this.submitting.set(false));
 1205:   }
 1206: }
```

E.10 Route, Reply, Completion, and Attachment Form Logic

Source file: frontend/src/app/components/modals/route-document/route-document.component.ts

```
// ----- lines 200-0 -----
// ----- lines 429-0 -----
```

E.11 Backend Document Registration and Route APIs

Source file: backend/documents.ts

```
// ----- lines 238-600 -----
  238:   router.get('/', async (req, res) => {
  239:     try {
  240:       const liveDocuments = await loadLiveDocuments();
  241:       if (liveDocuments) setDocumentsState(liveDocuments as
DocumentRecord[]);
  242:     } catch (error) {
  243:       console.error('[GET /api/documents] Live MySQL read failed:', error);
  244:       return res
  245:         .status(503)
  246:         .json({ error: 'Documents are temporarily unavailable.' });
```

```
  }
  let list = [...getDocumentsState()];
  const actingUser = getActingUser(req);
  if (!actingUser) {
    return res.status(401).json({ error: 'Active database user required.' });
  }
  list = list
    .filter((document) => canViewDocument(actingUser, document))
    .map(withRecoveredAuditRoutes)
    .map(hideSystemAdministratorFromTransactions);
  const { search, direction, status, division, priority } = req.query;

  if (direction) {
    list = list.filter((d) => d.direction === direction);
  }
  if (status) {
    list = list.filter((d) => d.currentStatus === status);
  }
  if (division) {
    list = list.filter((d) => d.currentDivision === division);
  }
  if (priority) {
    list = list.filter((d) => d.priority === priority);
  }
  if (search) {
    const q = (search as string).toLowerCase();
    list = list.filter(
      (d) =>
        d.routeNo?.toLowerCase().includes(q) ||
        d.title.toLowerCase().includes(q) ||
        d.subject.toLowerCase().includes(q) ||
        d.originatingOffice.toLowerCase().includes(q) ||
        d.category.toLowerCase().includes(q),
    );
  }

  res.json(list);
});

router.get('/next-route-number', async (req, res) => {
  const actingUser = getActingUser(req);
  if (!actingUser) {
```

```
289:       return res.status(401).json({ error: 'Active database user required.' });
290:     }
291:     try {
292:       const liveDocuments = await loadLiveDocuments();
293:       if (liveDocuments) setDocumentsState(liveDocuments as
DocumentRecord[]);
294:     } catch (error) {
295:       console.error('[GET next route number] Live read failed:', error);
296:       return res.status(503).json({ error: 'Unable to determine the next
Document Route No.' });
297:     }
298:     res.json({ routeNo: getNextRouteNumber(String(req.query.direction ||
'INCOMING')) });
299:   });
300:
301:   // GET Single Document
302:   router.get('/:id', async (req, res) => {
303:     const actingUser = getActingUser(req);
304:     if (!actingUser) {
305:       return res.status(401).json({ error: 'Active database user required.' });
306:     }
307:     try {
308:       const liveDocuments = await loadLiveDocuments();
309:       if (liveDocuments) setDocumentsState(liveDocuments as
DocumentRecord[]);
310:     } catch {
311:       // Fallback to in-memory documents if MySQL live read is unavailable
312:     }
313:     const doc = getDocumentsState().find(
314:       (d) => d.id === req.params.id || d.routeNo === req.params.id,
315:     );
316:     if (!doc || !canViewDocument(actingUser, doc)) {
317:       return res.status(404).json({ error: 'Document not found' });
318:     }
319:
res.json(hideSystemAdministratorFromTransactions(withRecoveredAuditRoutes(do
c)));
320:   });
321:
322:   // POST Create Document
323:   router.post('/', async (req, res) => {
324:     const body = req.body;
```

```
  const now = new Date().toISOString();
  let documentsState = getDocumentsState();
  const usersState = getUsersState();
  const actingUser = getActingUser(req);
  if (!actingUser) {
    return res.status(401).json({ error: 'Active database user required.' });
  }
  try {
    const liveDocuments = await loadLiveDocuments();
    if (liveDocuments) {
      documentsState = liveDocuments as DocumentRecord[];
      setDocumentsState(documentsState);
    }
  } catch (error) {
    console.error('[POST /api/documents] Live duplicate check failed:',
error);
    return res.status(503).json({ error: 'Unable to verify the next Document
Route No.' });
  }

  const trackingNumber = String(body.routeNo || '').trim();
  const nextRouteNumber = getNextRouteNumber(String(body.direction
|| 'INCOMING'));
  const isDuplicate = documentsState.some(
    (document) =>
      document.trackingNumber === trackingNumber ||
document.routeNo === trackingNumber,
  );
  if (!trackingNumber || isDuplicate || trackingNumber !==
nextRouteNumber) {
    return res.status(409).json({
      error: isDuplicate
        ? `Duplicate Document Route No.: ${trackingNumber} is already
assigned. Nothing was saved.`
        : `Document Route No. is no longer available. The next number is
${nextRouteNumber}. Nothing was saved.`,
      nextRouteNo: nextRouteNumber,
    });
  }

  const newDocumentId = nextDocumentId();
  const isMultiDivisionInitialRoute =
```

```
360:       body.routeAllDivisions || body.routeMultipleDivisions;
361:     const initialRecipients = usersState.filter(
362:       (user) =>
363:         user.active &&
364:         user.role !== 'SYSTEM_ADMIN' &&
365:         user.divisionCode !== 'ITMS' &&
366:         Array.isArray(body.initialRecipientIds) &&
367:         body.initialRecipientIds.includes(user.id),
368:     );
369:     const directlyAssignedUser = body.assignedUserId
370:       ? findActiveDatabaseUser(String(body.assignedUserId))
371:       : undefined;
372:     if (
373:       directlyAssignedUser?.role === 'SYSTEM_ADMIN' ||
374:       directlyAssignedUser?.divisionCode === 'ITMS'
375:     ) {
376:       return res.status(400).json({
377:         error: 'System administrators cannot be document recipients or
handlers.',
378:       });
379:     }
380:     const isOutside = Boolean(body.isOutside);
381:     const isNotYetRouted =
382:       !isOutside &&
383:       (body.currentStatus === 'NOT_YET_ROUTED' ||
384:         body.registerOnly === true ||
385:         body.isNotYetRouted ||
386:         (!body.currentDivision &&
387:           !body.assignedUserId &&
388:           (!body.initialRecipientIds || body.initialRecipientIds.length === 0)));
389:
390:     const completedRemarks =
391:       body.remarks && String(body.remarks).trim() !== '' &&
String(body.remarks).trim() !== 'N/A'
392:         ? String(body.remarks).trim()
393:         : 'Completed - Recorded as Outside Office Outgoing Dispatch';
394:
395:     const newDoc: DocumentRecord = {
396:       id: newDocumentId,
397:       trackingNumber,
398:       routeNo: trackingNumber,
399:       direction: body.direction || 'INCOMING',
```

```
400:       title: body.title,
401:       subject: body.subject || '',
402:       category: body.category || 'General Correspondence',
403:       originatingOffice: body.originatingOffice || 'External Office',
404:       destinationOffice: body.destinationOffice || (isOutside ? 'External
Office' : 'BLGF Region II'),
405:       senderName: body.senderName || '',
406:       senderPosition: body.senderPosition || '',
407:       senderAddress: body.senderAddress || '',
408:       recipientName: isOutside ? (body.recipientName || '') :
(isNotYetRouted ? 'Not Yet Routed' : (body.recipientName || '')),
409:       recipientPosition: body.recipientPosition || '',
410:       recipientOffice: body.recipientOffice || '',
411:       recipientAddress: body.recipientAddress || '',
412:       priority: isOutside ? 'ROUTINE' : (body.priority || 'ROUTINE'),
413:       currentStatus: isOutside
414:         ? 'COMPLETED'
415:         : (isNotYetRouted ? 'NOT_YET_ROUTED' : (body.currentStatus ||
'PENDING')),
416:       currentDivision: isNotYetRouted
417:         ? (actingUser.divisionCode || 'AD')
418:         : (body.currentDivision || 'AD'),
419:       assignedUser: isOutside
420:         ? (body.recipientName || 'External Recipient')
421:         : (isNotYetRouted ? 'Not Yet Routed' : (directlyAssignedUser?.fullName
|| '')),
422:       assignedUserId: isOutside || isNotYetRouted ? undefined :
directlyAssignedUser?.id,
423:       dateReceived: body.dateReceived || now,
424:       targetCompletionDate:
425:         body.targetCompletionDate ||
426:         new Date(Date.now() + 3 * 86400000).toISOString(),
427:       tags: body.tags || [],
428:       attachments: (body.attachments || []).map((file: any) => ({
429:         ...file,
430:         id: nextAttachmentId(),
431:         attachmentScope: 'DOCUMENT',
432:         uploadedByUserId: actingUser.id,
433:       })),
434:       actionRequested: isOutside
435:         ? 'Dispatched / Recorded (Outside Office)'
```

```
436:         : (isNotYetRouted ? (body.initialAction || 'Awaiting Initial Routing') :
(body.initialAction || 'Appropriate Action')),
437:       routes: isOutside
438:         ? [
439:             {
440:               id: nextRouteId(),
441:               documentId: newDocumentId,
442:               stepNumber: 1,
443:               routeNo: trackingNumber,
444:               fromDivision: actingUser.divisionCode || body.currentDivision ||
'AD',
445:               fromUserId: actingUser.id,
446:               fromUser: actingUser.fullName,
447:               toDivision: undefined,
448:               toUser: body.recipientName || 'External Recipient',
449:               toUserId: undefined,
450:               actionRequested: 'Dispatched / Recorded (Outside Office)',
451:               remarks: completedRemarks,
452:               statusBefore: 'PENDING' as DocumentStatus,
453:               statusAfter: 'COMPLETED' as DocumentStatus,
454:               receivedAt: now,
455:               createdAt: now,
456:             },
457:           ]
458:         : (isNotYetRouted
459:           ? []
460:           : (initialRecipients.length > 0
461:             ? initialRecipients.map((recipient, index) => ({
462:                 id: nextRouteId(),
463:                 documentId: newDocumentId,
464:                 stepNumber: index + 1,
465:                 routeNo: trackingNumber,
466:                 fromDivision: body.currentDivision || 'AD',
467:                 fromUserId: actingUser.id,
468:                 fromUser: actingUser.fullName,
469:                 toDivision: recipient.divisionCode,
470:                 toUser: recipient.fullName,
471:                 toUserId: recipient.id,
472:                 actionRequested: body.initialAction || 'Initial Entry & Routing',
473:                 remarks: body.remarks || 'N/A',
474:                 statusBefore: 'PENDING' as DocumentStatus,
```

```
475:             statusAfter: (body.currentStatus || 'PENDING') as
DocumentStatus,
476:             isMultiRoute: initialRecipients.length > 1,
477:             receivedAt: now,
478:             createdAt: now,
479:           }))
480:         : [
481:             {
482:               id: nextRouteId(),
483:               documentId: newDocumentId,
484:               stepNumber: 1,
485:               routeNo: trackingNumber,
486:               fromDivision: body.currentDivision || 'AD',
487:               fromUserId: actingUser.id,
488:               fromUser: actingUser.fullName,
489:               toDivision: body.currentDivision || 'AD',
490:               toUser: directlyAssignedUser?.fullName,
491:               toUserId: directlyAssignedUser?.id,
492:               actionRequested:
493:                 body.initialAction || 'Initial Entry & Routing',
494:               remarks: body.remarks || 'N/A',
495:               statusBefore: 'PENDING',
496:               statusAfter: body.currentStatus || 'PENDING',
497:               receivedAt: now,
498:               createdAt: now,
499:             },
500:           ])),
501:     createdBy: actingUser.fullName,
502:     createdByUserId: actingUser.id,
503:     createdAt: now,
504:     updatedAt: now,
505:   };
506:
507:   setDocumentsState([newDoc, ...documentsState]);
508:   saveDatabaseToFile(false);
509:
510:   const creationAuditLog = addAuditLog(
511:     getAuditLogsState(),
512:     {
513:       userId: actingUser.id,
514:       userName: actingUser.fullName,
515:       userRole: actingUser.role,
```

```
        action: 'CREATE_DOC',
        documentTrackingNumber: trackingNumber,
        details: [
          `Created by: ${actingUser.fullName}`,
          `Document: ${body.title || 'Untitled'} [${trackingNumber}]`,
          `Direction: ${body.direction || 'INCOMING'}`,
          `Status: ${isOutside ? 'COMPLETED' : (body.currentStatus ||
'PENDING')}`,
          `Subject: ${body.subject || 'N/A'}`,
          `Sender: ${body.senderName || 'N/A'}`,
          `Position: ${body.senderPosition || 'N/A'}`,
          `Originating office: ${body.originatingOffice || 'N/A'}`,
          `Address: ${body.senderAddress || 'N/A'}`,
          body.recipientName ? `Recipient: ${body.recipientName}
(${body.recipientOffice || 'External Office'})` : '',
          body.recipientAddress ? `Recipient Address:
${body.recipientAddress}` : '',
          `Action: ${isOutside ? 'Dispatched / Recorded (Outside Office)' :
(body.initialAction || 'Appropriate Action')}`,
          `Remarks: ${completedRemarks}`,
        ].filter(Boolean).join(' | '),
        ipAddress: req.ip || '127.0.0.1',
      },
      false,
    );

    const createdNotifications: NotificationItem[] = [];
    if (!isOutside && !isNotYetRouted && body.assignedUserId) {
      const assignedRecipient = usersState.find(
        (user) =>
          user.id === body.assignedUserId &&
          user.active &&
          user.role !== 'SYSTEM_ADMIN' &&
          user.divisionCode !== 'ITMS',
      );
      if (assignedRecipient) {
        createdNotifications.push(
          createNotification(
            {
              userId: assignedRecipient.id,
              title: 'New Document Routed to You',
```

```
553:             message: `Document ${trackingNumber} (${newDoc.title}) was
assigned directly to you.`,
554:             documentId: newDoc.id,
555:             trackingNumber,
556:             type: 'ACTION_REQUIRED',
557:           },
558:           getNotificationsState(),
559:         ),
560:       );
561:     }
562:   } else if (!isOutside && !isNotYetRouted && body.currentDivision) {
563:     const excludedRecipientIds = Array.isArray(body.excludedRecipientIds)
564:       ? body.excludedRecipientIds
565:       : [];
566:     usersState
567:       .filter(
568:         (user) =>
569:           user.active &&
570:           (isMultiDivisionInitialRoute
571:             ? Array.isArray(body.initialRecipientIds) &&
572:               body.initialRecipientIds.includes(user.id)
573:             : user.divisionCode === body.currentDivision) &&
574:           !excludedRecipientIds.includes(user.id),
575:       )
576:       .forEach((divisionUser) => {
577:         createdNotifications.push(
578:           createNotification(
579:             {
580:               userId: divisionUser.id,
581:               title: `New Document for ${isMultiDivisionInitialRoute ? 'Multiple
Divisions' : body.currentDivision}`,
582:               message: `Document ${trackingNumber} (${newDoc.title}) was
routed to ${isMultiDivisionInitialRoute ? 'multiple divisions' : 'your division'}.`,
583:               documentId: newDoc.id,
584:               trackingNumber,
585:               type: 'ACTION_REQUIRED',
586:             },
587:             getNotificationsState(),
588:           ),
589:         );
590:       });
591:   }
```

```
  592:     if (createdNotifications.length > 0) {
  593:       setNotificationsState([
  594:         ...createdNotifications,
  595:         ...getNotificationsState(),
  596:       ]);
  597:     }
  598:
  599:     await Promise.all([
  600:       saveDocumentDirect(newDoc),
// ----- lines 606-970 -----
  606:   // POST Approve / Disapprove the latest route assigned to the acting user
  607:   router.post('/:id/decision', async (req, res) => {
  608:     const documentsState = getDocumentsState();
  609:     const docIndex = documentsState.findIndex((document) =>
document.id === req.params.id);
  610:     const actingUser = getActingUser(req);
  611:     if (!actingUser) {
  612:       return res.status(401).json({ error: 'Active database user required.' });
  613:     }
  614:     if (actingUser.role === 'SYSTEM_ADMIN') {
  615:       return res.status(403).json({ error: 'System Administrators cannot approve
or disapprove routing transactions.' });
  616:     }
  617:     if (docIndex === -1) return res.status(404).json({ error: 'Document not
found.' });
  618:
  619:     const doc = documentsState[docIndex];
  620:     if (doc.currentStatus === 'COMPLETED' || hasCompletedPart(doc,
actingUser)) {
  621:       return res.status(409).json({ error: 'This transaction has ended.
Completed documents cannot be approved or disapproved.' });
  622:     }
  623:     const decision = String(req.body.decision || '').toUpperCase();
  624:     const remarks = String(req.body.remarks || '').trim();
  625:     if (decision !== 'APPROVED' && decision !== 'DISAPPROVED') {
  626:       return res.status(400).json({ error: 'Select Approved or Disapproved.' });
  627:     }
  628:     if (decision === 'DISAPPROVED' && !remarks) {
  629:       return res.status(400).json({ error: 'A disapproval remark is required.' });
  630:     }
  631:
  632:     let assignedRoute = [...(doc.routes || [])]
```

```
633:       .reverse()
634:       .find(
635:         (route) =>
636:           !['APPROVED', 'DISAPPROVED'].includes(
637:             route.actionRequested.toUpperCase(),
638:           ) &&
639:           (route.toUserId === actingUser.id ||
640:             (!route.toUserId &&
641:               route.toUser?.trim().toLowerCase() ===
642:                 actingUser.fullName.trim().toLowerCase())),
643:       );
644:     if (!assignedRoute && decision === 'APPROVED') {
645:       assignedRoute = [...(doc.routes || [])]
646:         .reverse()
647:         .find(
648:           (route) =>
649:             route.toUserId === actingUser.id ||
650:             (!route.toUserId &&
651:               route.toUser?.trim().toLowerCase() ===
652:                 actingUser.fullName.trim().toLowerCase()) ||
653:             route.fromUserId === actingUser.id ||
654:             (!route.fromUserId &&
655:               route.fromUser?.trim().toLowerCase() ===
656:                 actingUser.fullName.trim().toLowerCase()) ||
657:             doc.assignedUserId === actingUser.id ||
658:             doc.createdByUserId === actingUser.id ||
659:             ['ADMIN', 'SUPER_ADMIN'].includes(actingUser.role),
660:         );
661:     }
662:     if (!assignedRoute) {
663:       const recoveredAudit = [...getAuditLogsState()]
664:         .filter(
665:           (log) =>
666:             log.documentTrackingNumber === doc.trackingNumber &&
667:             new RegExp(
668:               `\\bTo:\\s*${actingUser.fullName.replace(/[.*+?^${}()|[\]\\]/g,
'\\$&')}(?:\\s*\\||$)`,
669:               'i',
670:             ).test(log.details),
671:         )
672:         .sort(
673:           (a, b) =>
```

```
674:           new Date(b.timestamp).getTime() - new
Date(a.timestamp).getTime(),
675:         )[0];
676:       if (recoveredAudit) {
677:         const recoveredSender =
findActiveDatabaseUser(recoveredAudit.userId);
678:         assignedRoute = {
679:           id: nextRouteId(),
680:           documentId: doc.id,
681:           stepNumber: (doc.routes || []).length + 1,
682:           routeNo: doc.routeNo || doc.trackingNumber,
683:           fromDivision: recoveredSender?.divisionCode || doc.currentDivision,
684:           fromUserId: recoveredSender?.id,
685:           fromUser: recoveredAudit.userName,
686:           toDivision: actingUser.divisionCode,
687:           toUserId: actingUser.id,
688:           toUser: actingUser.fullName,
689:           actionRequested:
690:
recoveredAudit.details.match(/Action:\s*(.*?)(?:\s*\|\s*Remarks:|$)/i)?.[1] ||
691:             'Appropriate Action',
692:           remarks:
693:
recoveredAudit.details.match(/Remarks:\s*(.*?)(?:\s*\|\s*Status:|$)/i)?.[1] ||
694:             '',
695:           statusBefore: doc.currentStatus,
696:           statusAfter: doc.currentStatus,
697:           createdAt: recoveredAudit.timestamp,
698:         };
699:         doc.routes.push(assignedRoute);
700:       }
701:     }
702:     if (!assignedRoute) {
703:       return res.status(403).json({ error: 'This document is not awaiting your
decision.' });
704:     }
705:     const latestDecision = [...(doc.routes || [])]
706:       .reverse()
707:       .find(
708:         (route) =>
709:           (route.fromUserId === actingUser.id ||
710:             (!route.fromUserId &&
```

```
711:             route.fromUser?.trim().toLowerCase() ===
712:               actingUser.fullName.trim().toLowerCase())) &&
713:           route.createdAt > assignedRoute.createdAt &&
714:           ['APPROVED', 'DISAPPROVED'].includes(route.actionRequested),
715:       );
716:     const hasDisapprovalInTrail =
717:       doc.currentStatus === 'RETURNED' ||
718:       (doc.routes || []).some(
719:         (r) => r.actionRequested?.toUpperCase() === 'DISAPPROVED',
720:       );
721:     const isReapproval =
722:       decision === 'APPROVED' &&
723:       (latestDecision?.actionRequested === 'DISAPPROVED' ||
hasDisapprovalInTrail);
724:     if (latestDecision && !isReapproval) {
725:       return res.status(409).json({ error: 'You have already acted on this
document route.' });
726:     }
727:
728:     const now = new Date().toISOString();
729:     assignedRoute.actionTaken = decision;
730:     assignedRoute.processedAt = now;
731:     const legacySenderRoute = (doc.routes || []).find(
732:       (route) =>
733:         Boolean(route.toUserId) &&
734:         route.toUser?.trim().toLowerCase() ===
735:           assignedRoute.fromUser?.trim().toLowerCase(),
736:     );
737:     const sender =
738:       (assignedRoute.fromUserId
739:         ? findActiveDatabaseUser(assignedRoute.fromUserId)
740:         : undefined) ||
741:       (legacySenderRoute?.toUserId
742:         ? findActiveDatabaseUser(legacySenderRoute.toUserId)
743:         : undefined) ||
744:       (doc.createdByUserId
745:         ? findActiveDatabaseUser(doc.createdByUserId)
746:         : undefined) ||
747:       getUsersState().find(
748:         (user) =>
749:           user.active &&
750:           (user.fullName.trim().toLowerCase() ===
```

```
751:           assignedRoute.fromUser?.trim().toLowerCase() ||
752:           user.fullName.trim().toLowerCase() ===
753:             doc.createdBy?.trim().toLowerCase()),
754:       );
755:     const decisionRoute: DocumentRouteStep = {
756:       id: nextRouteId(),
757:       documentId: doc.id,
758:       stepNumber: (doc.routes || []).length + 1,
759:       routeNo: doc.routeNo || doc.trackingNumber,
760:       fromDivision: actingUser.divisionCode,
761:       fromUserId: actingUser.id,
762:       fromUser: actingUser.fullName,
763:       toDivision: decision === 'DISAPPROVED' && sender
764:         ? sender.divisionCode
765:         : actingUser.divisionCode,
766:       toUser: sender?.fullName,
767:       toUserId: sender?.id,
768:       actionRequested: decision,
769:       remarks: remarks || 'Approved; proceed with routing.',
770:       statusBefore: doc.currentStatus,
771:       statusAfter: decision === 'DISAPPROVED' ? 'RETURNED' : 'IN_PROGRESS',
772:       receivedAt: now,
773:       createdAt: now,
774:     };
775:     doc.routes.push(decisionRoute);
776:     doc.currentStatus = decisionRoute.statusAfter;
777:     doc.updatedAt = now;
778:     if (decision === 'DISAPPROVED' && sender) {
779:       doc.currentDivision = sender.divisionCode;
780:       doc.assignedUser = sender.fullName;
781:       doc.assignedUserId = sender.id;
782:     } else if (decision === 'APPROVED') {
783:       doc.currentDivision = actingUser.divisionCode;
784:       doc.assignedUser = actingUser.fullName;
785:       doc.assignedUserId = actingUser.id;
786:     }
787:     documentsState[docIndex] = doc;
788:     setDocumentsState(documentsState);
789:     saveDatabaseToFile(false);
790:
791:     const auditLog = addAuditLog(getAuditLogsState(), {
792:       userId: actingUser.id,
```

```
  793:       userName: actingUser.fullName,
  794:       userRole: actingUser.role,
  795:       action: 'UPDATE_STATUS',
  796:       documentTrackingNumber: doc.trackingNumber,
  797:       details: `${decision} by ${actingUser.fullName} | Remarks: ${remarks ||
'Approved; proceed with routing.'}`,
  798:       ipAddress: req.ip || '127.0.0.1',
  799:     }, false);
  800:     await Promise.all([saveDocumentDirect(doc),
saveAuditLogDirect(auditLog)]);
  801:     res.json(withRecoveredAuditRoutes(doc));
  802:   });
  803:
  804:   // POST Route/Forward Document
  805:   router.post('/:id/route', async (req, res) => {
  806:     const documentsState = getDocumentsState();
  807:     const docIndex = documentsState.findIndex((d) => d.id ===
req.params.id);
  808:     if (docIndex === -1) {
  809:       return res.status(404).json({ error: 'Document not found' });
  810:     }
  811:
  812:     const doc = documentsState[docIndex];
  813:     const {
  814:       fromDivision,
  815:       toDivision,
  816:       toUser,
  817:       toUserId,
  818:       actionRequested,
  819:       remarks,
  820:       replyAttachments,
  821:       newStatus,
  822:       routeNo,
  823:     } = req.body;
  824:     const actingUser = getActingUser(req);
  825:     if (!actingUser) {
  826:       return res.status(401).json({ error: 'Active database user required.' });
  827:     }
  828:     if (actingUser.role === 'SYSTEM_ADMIN') {
  829:       return res.status(403).json({ error: 'System Administrators cannot route
documents.' });
  830:     }
```

```
831:    if (!canViewDocument(actingUser, doc)) {
832:      return res.status(404).json({ error: 'Document not found' });
833:    }
834:    const wasCompleted = doc.currentStatus === 'COMPLETED';
835:    if (
836:      wasCompleted &&
837:      !hasCompletedPart(doc, actingUser) &&
838:      !['ADMIN', 'ORD', 'RECORDS_OFFICER'].includes(actingUser.role)
839:    ) {
840:      return res.status(403).json({ error: 'Only the completing user or an
authorized records officer can continue this completed transaction.' });
841:    }
842:    if (hasOpenDisapproval(doc)) {
843:      return res.status(409).json({
844:        error: 'This document is disapproved. Re-approve it before routing.',
845:      });
846:    }
847:    if (doc.currentStatus === 'RETURNED') {
848:      await healStaleReturnedStatus(req, doc, documentsState, docIndex,
actingUser);
849:    }
850:    if (getPendingDecisionRoute(actingUser, doc)) {
851:      return res.status(409).json({
852:        error: 'Action required: Approve or Disapprove this document before
routing or forwarding it.',
853:      });
854:    }
855:
856:    if (
857:      routeNo &&
858:      documentsState.some(
859:        (document) =>
860:          document.id !== doc.id &&
861:          (document.routeNo?.trim().toLowerCase() ===
862:            String(routeNo).trim().toLowerCase() ||
863:            document.routes?.some(
864:              (route) =>
865:                route.routeNo?.trim().toLowerCase() ===
866:                String(routeNo).trim().toLowerCase(),
867:            )),
868:      )
869:    ) {
```

```
870:       return res.status(409).json({
871:         error: `Document Route No. ${routeNo} is already used by another
document.`,
872:       });
873:     }
874:
875:     const recipient = getUsersState().find(
876:       (user) =>
877:         user.id === toUserId &&
878:         user.active &&
879:         user.role !== 'SYSTEM_ADMIN' &&
880:         user.divisionCode !== 'ITMS',
881:     );
882:     const isCompletionWithoutRecipient =
883:       newStatus === 'COMPLETED' && !toUserId && !toUser;
884:     if (!recipient && !isCompletionWithoutRecipient) {
885:       return res
886:         .status(400)
887:         .json({ error: 'Select an active user account as the recipient.' });
888:     }
889:     if (recipient && findPreviousDelivery(doc, recipient)) {
890:       return res.status(409).json({ error: `Already routed to
${recipient.fullName}. Choose another recipient.` });
891:     }
892:
893:     const now = new Date().toISOString();
894:     const oldStatus = doc.currentStatus;
895:     if (newStatus === 'COMPLETED' && (toUserId || toUser)) {
896:       return res.status(400).json({ error: 'Complete your part without sending to
new recipients.' });
897:     }
898:     const nextStatus: DocumentStatus =
899:       newStatus || (wasCompleted
900:         ? 'IN_PROGRESS'
901:         : doc.currentStatus === 'NOT_YET_ROUTED'
902:           ? 'PENDING'
903:           : doc.currentStatus);
904:
905:     const stepNumber = (doc.routes || []).length + 1;
906:     markLatestRouteAsProcessed(
907:       doc,
908:       actingUser.id,
```

```
    actingUser.fullName,
    actionRequested || 'Document forwarded',
    now,
    Array.isArray(replyAttachments) ? replyAttachments : undefined,
  );
  const newRoute: DocumentRouteStep = {
    id: nextRouteId(),
    documentId: doc.id,
    stepNumber,
    routeNo: routeNo || doc.routeNo || '',
    fromDivision: fromDivision || doc.currentDivision,
    fromUserId: actingUser.id,
    fromUser: actingUser.fullName,
    toDivision: toDivision || doc.currentDivision,
    toUser: recipient?.fullName,
    toUserId: recipient?.id,
    actionRequested: actionRequested || 'For Appropriate Action',
    remarks: remarks || '',
    statusBefore: oldStatus,
    statusAfter: nextStatus,
    receivedAt: now,
    attachments: Array.isArray(replyAttachments)
      ? replyAttachments.map((file: any) => ({
          ...file,
          id: nextAttachmentId(),
          attachmentScope: 'RECIPIENT',
          uploadedByUserId: actingUser.id,
        }))
      : undefined,
    createdAt: now,
  };

  doc.routes.push(newRoute);
  if (routeNo) doc.routeNo = routeNo;
  doc.currentDivision = toDivision || doc.currentDivision;
  const pendingRecipients = nextStatus === 'COMPLETED' ?
getPendingRecipients(doc) : [];
  doc.currentStatus = nextStatus === 'COMPLETED' &&
pendingRecipients.length ? 'IN_PROGRESS' : nextStatus;
  doc.actionRequested = actionRequested || doc.actionRequested;
  if (recipient) {
    doc.assignedUser = recipient.fullName;
```

```
949:       doc.assignedUserId = recipient.id;
950:     } else {
951:       doc.assignedUser = pendingRecipients[0]?.toUser;
952:       doc.assignedUserId = pendingRecipients[0]?.toUserId;
953:     }
954:     doc.updatedAt = now;
955:     if (wasCompleted) {
956:       doc.completedDate = undefined;
957:       doc.destinationOffice = undefined;
958:     }
959:     setNotificationsState(
960:       getNotificationsState().filter(
961:         (notification) =>
962:           !(
963:             notification.documentId === doc.id &&
964:             notification.userId === actingUser.id
965:           ),
966:       ),
967:     );
968:
969:     if (doc.currentStatus === 'COMPLETED' && !doc.completedDate) {
970:       doc.completedDate = now;
```

E.12 Authentication, Notifications, Statistics, Logs, and Application Bootstrap

Source file: backend/server.ts

```
// ----- lines 1033-1323 -----
1033:   app.get('/api/health', (_req, res) => {
1034:     res.json({
1035:       status: 'ok',
1036:       system: 'BLGF Region II Document Tracking System API',
1037:       databaseLoaded: true,
1038:       activeDatabase: mysqlConnected ? 'mysql' : 'local-json',
1039:       mysql: getMySQLReplicaStatus(),
1040:       recordsCount: {
1041:         documents: documentsState.length,
1042:         users: usersState.length,
1043:         auditLogs: auditLogsState.length,
1044:       },
1045:     });
1046:   });
1047:
1048:   // Explicit full MySQL replication for system administrators. Ordinary saves
```

```
1049:  // persist to the local JSON store only.
1050:  app.post('/api/admin/sync', (req, res) => {
1051:    const actingUser = usersState.find(
1052:      (user) => user.id === req.header('X-User-Id'),
1053:    );
1054:    if (!actingUser || actingUser.role !== 'SYSTEM_ADMIN') {
1055:      return res.status(403).json({ error: 'System administrator access required.'
});
1056:    }
1057:    queueDatabaseSync(true);
1058:    void flushDatabaseSync().then(() =>
1059:      res.json({ synced: true, recordsCount: getDatabaseStateEntries().length
}),
1060:    );
1061:  });
1062:
1063:  // Auth / Current Session Mock Login
1064:  app.post('/api/auth/login', (req, res) => {
1065:    const username = String(req.body?.username ?? '')
1066:      .trim()
1067:      .toLowerCase();
1068:    const password = String(req.body?.password ?? '');
1069:    if (!username || !password) {
1070:      return res
1071:        .status(400)
1072:        .json({ error: 'Username and password are required.' });
1073:    }
1074:    const matchingUsers = usersState.filter(
1075:      (u) =>
1076:        String(u.username ?? '')
1077:          .trim()
1078:          .toLowerCase() === username ||
1079:        String(u.email ?? '')
1080:          .trim()
1081:          .toLowerCase() === username,
1082:    );
1083:
1084:    if (!matchingUsers.length) {
1085:      return res.status(401).json({ error: 'User account not found' });
1086:    }
1087:
1088:    if (!matchingUsers.some((user) => user.active)) {
```

```
1089:       return res
1090:         .status(403)
1091:         .json({ error: 'Account is deactivated. Contact Admin.' });
1092:     }
1093:
1094:     const user = matchingUsers.find(
1095:       (candidate) =>
1096:         candidate.active &&
1097:         typeof candidate.password === 'string' &&
1098:         candidate.password === password,
1099:     );
1100:     if (!user) {
1101:       return res.status(401).json({ error: 'Incorrect password.' });
1102:     }
1103:     if (
1104:       user.temporaryPasswordExpiresAt &&
1105:       new Date(user.temporaryPasswordExpiresAt).getTime() <= Date.now()
1106:     ) {
1107:       return res.status(401).json({
1108:         error: 'Temporary password expired. Request a new temporary
password.',
1109:       });
1110:     }
1111:
1112:     addAuditLog(auditLogsState, {
1113:       userId: user.id,
1114:       userName: user.fullName,
1115:       userRole: user.role,
1116:       action: 'LOGIN',
1117:       details: `User ${user.fullName} (${user.role}) logged in successfully.`,
1118:       ipAddress: req.ip || '127.0.0.1',
1119:     });
1120:
1121:     const {
1122:       password: _password,
1123:       temporaryPasswordExpiresAt: _temporaryPasswordExpiresAt,
1124:       ...safeUser
1125:     } = user;
1126:     // Signing in with a temporary password does not finish the login: the
1127:     // client has to collect a new password first. The expiry check above
1128:     // already rejected stale temporary passwords, so any remaining value
means
```

```
1129:     // this credential is a live temporary one.
1130:     const mustChangePassword =
Boolean(user.temporaryPasswordExpiresAt);
1131:     return res.json({ user: safeUser, mustChangePassword });
1132:   });
1133:
1134:   // Finishes a temporary-password login. The temporary credential is
verified
1135:   // again here because no session exists yet, so the new password can
only be
1136:   // stored after proving ownership of the account.
1137:   app.post('/api/auth/complete-password-setup', async (req, res) => {
1138:     const username = String(req.body?.username || '').trim().toLowerCase();
1139:     const temporaryPassword = String(req.body?.temporaryPassword || '');
1140:     const newPassword = String(req.body?.newPassword || '');
1141:
1142:     if (!username || !temporaryPassword || !newPassword) {
1143:       return res
1144:         .status(400)
1145:         .json({ error: 'Username, temporary password, and new password are
required.' });
1146:     }
1147:     if (newPassword.length < 6) {
1148:       return res
1149:         .status(400)
1150:         .json({ error: 'Password must be at least 6 characters long.' });
1151:     }
1152:     if (newPassword === temporaryPassword) {
1153:       return res
1154:         .status(400)
1155:         .json({ error: 'Your new password must be different from the temporary
password.' });
1156:     }
1157:
1158:     const user = usersState.find(
1159:       (candidate) =>
1160:         candidate.active &&
1161:         candidate.username.trim().toLowerCase() === username,
1162:     );
1163:     if (
1164:       !user ||
1165:       typeof user.password !== 'string' ||
```

```
1166:       user.password !== temporaryPassword ||
1167:       !user.temporaryPasswordExpiresAt
1168:     ) {
1169:       return res
1170:         .status(401)
1171:         .json({ error: 'Temporary password is no longer valid. Sign in again.' });
1172:     }
1173:     if (new Date(user.temporaryPasswordExpiresAt).getTime() <=
Date.now()) {
1174:       return res
1175:         .status(401)
1176:         .json({ error: 'Temporary password expired. Request a new temporary
password.' });
1177:     }
1178:
1179:     user.password = newPassword;
1180:     user.temporaryPasswordExpiresAt = null;
1181:     try {
1182:       await saveUserDirect(user);
1183:     } catch (databaseError) {
1184:       console.warn('[AUTH] New password persistence warning:',
databaseError);
1185:     }
1186:
1187:     addAuditLog(auditLogsState, {
1188:       userId: user.id,
1189:       userName: user.fullName,
1190:       userRole: user.role,
1191:       action: 'UPDATE_USER',
1192:       details: `${user.fullName} replaced a temporary password with a new
account password.`,
1193:       ipAddress: req.ip || '127.0.0.1',
1194:     });
1195:
1196:     const {
1197:       password: _newPassword,
1198:       temporaryPasswordExpiresAt: _clearedExpiry,
1199:       ...safeUser
1200:     } = user;
1201:     return res.json({ user: safeUser });
1202:   });
1203:
```

```
1204:   const handleForgotPassword = async (req: express.Request, res:
express.Response) => {
1205:     const identifier = String(
1206:       req.body?.identifier ||
1207:         req.body?.usernameOrEmail ||
1208:         req.body?.email ||
1209:         req.body?.username ||
1210:         '',
1211:     )
1212:       .trim()
1213:       .toLowerCase();
1214:     if (!identifier) {
1215:       return res.status(400).json({
1216:         error: 'Username or email is required.',
1217:       });
1218:     }
1219:
1220:     const matchingUsers = usersState.filter(
1221:       (user) =>
1222:         user.active &&
1223:         (user.username.trim().toLowerCase() === identifier ||
1224:           (user.email && user.email.trim().toLowerCase() === identifier)),
1225:     );
1226:     const targetUser =
1227:       matchingUsers.find(
1228:         (user) => user.username.trim().toLowerCase() === identifier,
1229:       ) ||
1230:       matchingUsers.find(
1231:         (user) => user.email && user.email.trim().toLowerCase() === identifier,
1232:       ) ||
1233:       matchingUsers[0];
1234:
1235:     if (!targetUser) {
1236:       return res.status(404).json({
1237:         error:
1238:           'No active user account matches that username or email.',
1239:       });
1240:     }
1241:
1242:     const lastRequest = adminPasswordResetRequests.get(targetUser.id) ||
0;
1243:     if (Date.now() - lastRequest < 15_000) {
```

```
1244:       return res.status(429).json({
1245:         error:
1246:           'A temporary recovery password was recently generated. Please
wait 15 seconds before trying again.',
1247:       });
1248:     }
1249:
1250:     const recipientEmail = targetUser.email?.trim();
1251:     if (!recipientEmail || recipientEmail === 'N/A' ||
!recipientEmail.includes('@')) {
1252:       return res.status(400).json({
1253:         error: `User account "${targetUser.fullName}" (${targetUser.username})
does not have an official email address on file. Please contact your system
administrator to configure your email address.`,
1254:       });
1255:     }
1256:
1257:     const smtpConfig = getSmtpConfig();
1258:     if (!smtpConfig.isConfigured) {
1259:       return res.status(503).json({
1260:         error:
1261:           'Email delivery is currently unconfigured. Please configure SMTP_USER
and SMTP_PASSWORD in .env so the temporary password can be delivered
directly to your email address.',
1262:       });
1263:     }
1264:
1265:     // Generate secure, readable recovery password
1266:     const temporaryPassword = `BLGF-
${crypto.randomBytes(4).toString('hex').toUpperCase()}`;
1267:     const temporaryPasswordExpiresAt = new Date(Date.now() + 15 * 60 *
1000); // 15 minutes validity
1268:
1269:     try {
1270:       const emailResult = await sendTemporaryPasswordEmail(
1271:         targetUser,
1272:         temporaryPassword,
1273:         15,
1274:       );
1275:
1276:       if (!emailResult.success) {
1277:         return res.status(500).json({
```

```
1278:           error: `Failed to deliver recovery email to ${recipientEmail}:
${emailResult.error || 'SMTP delivery failed'}. Please verify SMTP settings in .env.`,
1279:         });
1280:       }
1281:
1282:       // Update password only after email is successfully dispatched
1283:       targetUser.password = temporaryPassword;
1284:       targetUser.temporaryPasswordExpiresAt =
temporaryPasswordExpiresAt.toISOString();
1285:
1286:       try {
1287:         await updateUserPassword(
1288:           targetUser.id,
1289:           temporaryPassword,
1290:           temporaryPasswordExpiresAt,
1291:         );
1292:       } catch (databaseError) {
1293:         console.warn('[AUTH] Direct MySQL password update warning:',
databaseError);
1294:       }
1295:       adminPasswordResetRequests.set(targetUser.id, Date.now());
1296:
1297:       addAuditLog(auditLogsState, {
1298:         userId: targetUser.id,
1299:         userName: targetUser.fullName,
1300:         userRole: targetUser.role,
1301:         action: 'UPDATE_USER',
1302:         details: `A temporary password was successfully emailed to
${targetUser.fullName} (${targetUser.username}) at ${recipientEmail}.`,
1303:         ipAddress: req.ip || '127.0.0.1',
1304:       });
1305:
1306:       return res.json({
1307:         success: true,
1308:         emailSent: true,
1309:         message: `A temporary password has been dispatched to your email
address: ${recipientEmail}.`,
1310:         targetEmail: recipientEmail,
1311:         username: targetUser.username,
1312:         expiresInMinutes: 15,
1313:       });
1314:     } catch (emailError: any) {
```

```
1315:       console.error('[AUTH] SMTP dispatch failed:', emailError);
1316:       return res.status(500).json({
1317:         error: `Failed to deliver recovery email to ${recipientEmail}:
${emailError?.message || 'SMTP connection failed'}. Please verify SMTP settings in
.env.`,
1318:       });
1319:     }
1320:   };
1321:
1322:   app.post('/api/auth/forgot-password', handleForgotPassword);
1323:   app.post('/api/auth/forgot-admin-password', handleForgotPassword);
// ----- lines 1642-1765 -----
1642:   app.get('/api/stats', async (_req, res) => {
1643:     try {
1644:       const liveDocuments = await loadLiveDocuments();
1645:       if (liveDocuments) documentsState = liveDocuments as
DocumentRecord[];
1646:     } catch {
1647:       // ignore
1648:     }
1649:     const totalIncoming = documentsState.filter(
1650:       (d) => d.direction === 'INCOMING',
1651:     ).length;
1652:     const totalOutgoing = documentsState.filter(
1653:       (d) => d.direction === 'OUTGOING',
1654:     ).length;
1655:     const pendingCount = documentsState.filter(
1656:       (d) => d.currentStatus === 'PENDING' || d.currentStatus ===
'NOT_YET_ROUTED',
1657:     ).length;
1658:     const inProgressCount = documentsState.filter(
1659:       (d) => d.currentStatus === 'IN_PROGRESS',
1660:     ).length;
1661:     const completedCount = documentsState.filter(
1662:       (d) => d.currentStatus === 'COMPLETED',
1663:     ).length;
1664:     const urgentCount = documentsState.filter(
1665:       (d) => d.priority === 'URGENT' || d.priority === 'VERY_URGENT',
1666:     ).length;
1667:     const returnedCount = documentsState.filter(
1668:       (d) => d.currentStatus === 'RETURNED',
1669:     ).length;
```

```
1670:
1671:     // Division breakdown: active workload per division. Each active
document
1672:     // counts once for every division still holding it (pending recipient), so a
1673:     // document awaiting action in LU and AD counts for both. Completed
documents
1674:     // are not workload.
1675:     const divMap: Record<string, number> = {};
1676:     divisionsState.forEach((d) => {
1677:       divMap[d.code] = 0;
1678:     });
1679:     documentsState.forEach((doc) => {
1680:       if (doc.currentStatus === 'COMPLETED') return;
1681:       const heldBy = new Set<string>();
1682:       getPendingRecipients(doc).forEach((route) => {
1683:         if (route.toDivision) heldBy.add(route.toDivision);
1684:       });
1685:       if (!heldBy.size && doc.currentDivision) heldBy.add(doc.currentDivision);
1686:       heldBy.forEach((code) => {
1687:         divMap[code] = (divMap[code] | | 0) + 1;
1688:       });
1689:     });
1690:
1691:     const divisionBreakdown = Object.entries(divMap).map(([code, count])
=> ({
1692:       division: code,
1693:       count,
1694:     }));
1695:
1696:     // Status breakdown
1697:     const statusMap: Record<DocumentStatus, number> = {
1698:       NOT_YET_ROUTED: 0,
1699:       PENDING: 0,
1700:       IN_PROGRESS: 0,
1701:       FOR_SIGNATURE: 0,
1702:       COMPLETED: 0,
1703:       RETURNED: 0,
1704:       ON_HOLD: 0,
1705:     };
1706:     documentsState.forEach((d) => {
1707:       statusMap[d.currentStatus] = (statusMap[d.currentStatus] | | 0) + 1;
1708:     });
```

```
1709:
1710:    const statusBreakdown = [
1711:      {
1712:        status: 'NOT_YET_ROUTED' as DocumentStatus,
1713:        label: 'Not Yet Routed',
1714:        count: statusMap.NOT_YET_ROUTED,
1715:        color: '#8b5cf6',
1716:      },
1717:      {
1718:        status: 'COMPLETED' as DocumentStatus,
1719:        label: 'Completed',
1720:        count: statusMap.COMPLETED,
1721:        color: '#10b981',
1722:      },
1723:      {
1724:        status: 'IN_PROGRESS' as DocumentStatus,
1725:        label: 'In Progress',
1726:        count: statusMap.IN_PROGRESS,
1727:        color: '#3b82f6',
1728:      },
1729:      {
1730:        status: 'FOR_SIGNATURE' as DocumentStatus,
1731:        label: 'For Signature',
1732:        count: statusMap.FOR_SIGNATURE,
1733:        color: '#6366f1',
1734:      },
1735:      {
1736:        status: 'PENDING' as DocumentStatus,
1737:        label: 'Pending',
1738:        count: statusMap.PENDING,
1739:        color: '#f59e0b',
1740:      },
1741:      {
1742:        status: 'RETURNED' as DocumentStatus,
1743:        label: 'Returned',
1744:        count: statusMap.RETURNED,
1745:        color: '#f43f5e',
1746:      },
1747:      {
1748:        status: 'ON_HOLD' as DocumentStatus,
1749:        label: 'On Hold',
1750:        count: statusMap.ON_HOLD,
```

```
1751:         color: '#ea580c',
1752:       },
1753:     ];
1754:
1755:     res.json({
1756:       totalIncoming,
1757:       totalOutgoing,
1758:       pendingCount,
1759:       inProgressCount,
1760:       completedCount,
1761:       urgentCount,
1762:       returnedCount,
1763:       avgTurnaroundHours: 18.5,
1764:       divisionBreakdown,
1765:       statusBreakdown,
// ----- lines 1851-2070 -----
1851:   app.get('/api/audit-logs', (_req, res) => {
1852:     res.json(auditLogsState);
1853:   });
1854:
1855:   app.delete('/api/audit-logs', async (_req, res) => {
1856:     const deletedCount = auditLogsState.length;
1857:     auditLogsState = [];
1858:     await deleteAuditLogsDirect();
1859:     res.json({
1860:       success: true,
1861:       deletedCount,
1862:       message: 'System audit records were deleted. Envelope logs were
preserved.',
1863:     });
1864:   });
1865:
1866:   app.post('/api/audit-logs', (req, res) => {
1867:     const body = req.body || {};
1868:     const actingUser = getRequestUser(req);
1869:     if (!actingUser) {
1870:       return res.status(401).json({ error: 'Active database user required.' });
1871:     }
1872:     const log = addAuditLog(auditLogsState, {
1873:       userId: actingUser.id,
1874:       userName: actingUser.fullName,
1875:       userRole: actingUser.role,
```

```
1876:       action: body.action || 'ENVELOPE_LOG',
1877:       documentTrackingNumber: body.documentTrackingNumber,
1878:       details: body.details || 'System activity recorded.',
1879:       ipAddress: req.ip || '127.0.0.1',
1880:     } as Omit<AuditLog, 'id' | 'timestamp'>);
1881:     res.status(201).json(log);
1882:   });
1883:
1884:   app.get('/api/envelope-logs', (_req, res) => {
1885:     res.json(envelopeLogsState);
1886:   });
1887:
1888:   app.post('/api/envelope-logs', async (req, res) => {
1889:     const body = req.body || {};
1890:     const actingUser = getRequestUser(req);
1891:     if (!actingUser) {
1892:       return res.status(401).json({ error: 'Active database user required.' });
1893:     }
1894:     const log: AuditLog = {
1895:       id: nextReadableId('ENV', envelopeLogsState),
1896:       timestamp: new Date().toISOString(),
1897:       userId: actingUser.id,
1898:       userName: actingUser.fullName,
1899:       userRole: actingUser.role,
1900:       action: 'ENVELOPE_LOG',
1901:       documentTrackingNumber: body.documentTrackingNumber,
1902:       details: body.details || 'Outgoing envelope dispatched.',
1903:       ipAddress: req.ip || '127.0.0.1',
1904:     };
1905:     envelopeLogsState.unshift(log);
1906:     await saveEnvelopeLogDirect(log);
1907:     res.status(201).json(log);
1908:   });
1909:
1910:   app.delete('/api/envelope-logs', async (_req, res) => {
1911:     const deletedCount = envelopeLogsState.length;
1912:     envelopeLogsState = [];
1913:     await deleteEnvelopeLogsDirect();
1914:     res.json({ success: true, deletedCount });
1915:   });
1916:
1917:   // GET Notifications
```

```
1918:  app.get('/api/notifications', async (req, res) => {
1919:    const userId = typeof req.query.userId === 'string' ? req.query.userId : '';
1920:    if (!userId) return res.json([]);
1921:    const requestingUser = getRequestUser(req);
1922:    if (!requestingUser || requestingUser.id !== userId) {
1923:      return res.status(403).json({ error: 'Notification access denied.' });
1924:    }
1925:    const notificationPermissions =
1926:      requestingUser.permissions?.allowedActions ||
1927:      DEFAULT_ROLE_PERMISSIONS[requestingUser.role]?.allowedActions ||
1928:      [];
1929:    const canViewAllTransactionNotifications =
1930:      requestingUser.permissions?.notificationViewAllConfigured === true
1931:        ? notificationPermissions.includes('NOTIFICATION_VIEW_ALL')
1932:        : (
1933:            DEFAULT_ROLE_PERMISSIONS[requestingUser.role]?.allowedActions
|| []
1934:          ).includes('NOTIFICATION_VIEW_ALL');
1935:    const activeDocTrackings = new Map<string, DocumentRecord>();
1936:    for (const d of documentsState) {
1937:      if (d.id) activeDocTrackings.set(d.id.trim().toUpperCase(), d);
1938:      if (d.trackingNumber)
activeDocTrackings.set(d.trackingNumber.trim().toUpperCase(), d);
1939:      if (d.routeNo) activeDocTrackings.set(d.routeNo.trim().toUpperCase(),
d);
1940:    }
1941:    // The transaction-activity feed may only ever describe this account's
own
1942:    // documents. Without this an activity log on somebody else's routing
would
1943:    // surface in the requester's notification list.
1944:    const isOwnDocument = (document: DocumentRecord) =>
1945:      isDocumentParticipant(document, requestingUser, auditLogsState);
1946:
1947:    // Clean up any stale/orphaned notifications in memory
1948:    notificationsState = notificationsState.filter((n) => {
1949:      if (n.documentId &&
!activeDocTrackings.has(n.documentId.trim().toUpperCase())) {
1950:        return false;
1951:      }
1952:      if (n.trackingNumber &&
!activeDocTrackings.has(n.trackingNumber.trim().toUpperCase())) {
```

```
1953:         return false;
1954:       }
1955:       return true;
1956:     });
1957:
1958:     const storedNotifications = notificationsState
1959:       .filter((notification) => notification.userId === userId)
1960:       .map((notification) =>
1961:         notification.title === 'Routing Action Reminder'
1962:           ? { ...notification, type: 'URGENT' as const, requiresDecision: false,
decisionStatus: undefined }
1963:           : notification,
1964:       );
1965:     const globalTransactionNotifications: NotificationItem[] =
1966:       canViewAllTransactionNotifications
1967:         ? auditLogsState
1968:             .filter(
1969:               (log) =>
1970:                 Boolean(log.documentTrackingNumber) &&
1971:
activeDocTrackings.has(log.documentTrackingNumber.trim().toUpperCase()) &&
1972:                 ['CREATE_DOC', 'ROUTE_DOC', 'TRANSFER_DOC',
'UPDATE_STATUS'].includes(
1973:                   log.action,
1974:                 ) &&
1975:                 !log.details?.startsWith('Routing reminder sent to'),
1976:             )
1977:             .map<NotificationItem | null>((log) => {
1978:               const matchedDoc = log.documentTrackingNumber
1979:                 ?
activeDocTrackings.get(log.documentTrackingNumber.trim().toUpperCase())
1980:                 : undefined;
1981:               if (!matchedDoc || !isOwnDocument(matchedDoc)) return null;
1982:               const decision = log.details.match(
1983:                 /(?:Action:\s*|^)(APPROVED|DISAPPROVED)\b/i,
1984:               )?.[1]?.toUpperCase();
1985:               const status = log.details.match(/Status:\s*([^|]+)/i)?.[1]?.trim();
1986:               const recipient = log.details.match(/To:\s*([^|]+)/i)?.[1]?.trim();
1987:               const title =
1988:                 decision === 'APPROVED'
1989:                   ? `Approved by ${log.userName}`
1990:                   : decision === 'DISAPPROVED'
```

```
1991:                 ? `Disapproved by ${log.userName}`
1992:                 : status === 'COMPLETED'
1993:                   ? `Document Completed by ${log.userName}`
1994:                   : log.action === 'CREATE_DOC'
1995:                     ? `Document Logged by ${log.userName}`
1996:                     : `Document Routed by ${log.userName}`;
1997:             const message = decision
1998:               ? `${log.documentTrackingNumber} was
${decision.toLowerCase()} by ${log.userName}.${
1999:                   decision === 'DISAPPROVED'
2000:                     ? ` Reason:
${log.details.match(/Remarks:\s*([^|]+)/i)?.[1]?.trim() || 'No reason provided.'}`
2001:                     : ''
2002:                 }`
2003:               : `${log.documentTrackingNumber} - ${title}${recipient ? ` to
${recipient}` : ''}${status ? ` (${status.replaceAll('_', ' ')})` : ''}.`;
2004:             return {
2005:               id: `transaction-activity-${log.id}`,
2006:               userId,
2007:               title,
2008:               message,
2009:               trackingNumber: log.documentTrackingNumber,
2010:               documentId: matchedDoc.id,
2011:               type: decision === 'DISAPPROVED' ? 'URGENT' : 'INFO',
2012:               requiresDecision: false,
2013:               decisionStatus:
2014:                 decision === 'APPROVED' || decision === 'DISAPPROVED'
2015:                   ? decision
2016:                   : undefined,
2017:               createdAt: log.timestamp,
2018:             } satisfies NotificationItem;
2019:           })
2020:           .filter((n): n is NotificationItem => n !== null)
2021:       : [];
2022:   const dedupedGlobalNotifications = [
2023:     ...globalTransactionNotifications
2024:       .reduce((map, item) => {
2025:         const key = item.trackingNumber || item.documentId || item.id;
2026:         const existing = map.get(key);
2027:         if (
2028:           !existing ||
2029:           new Date(item.createdAt).getTime() >
```

```
2030:             new Date(existing.createdAt).getTime()
2031:           ) {
2032:             map.set(key, item);
2033:           }
2034:           return map;
2035:         }, new Map<string, NotificationItem>())
2036:         .values(),
2037:     ];
2038:     try {
2039:       const rawLiveNotifications = await getLiveDisplayNotifications(userId);
2040:       const liveNotifications = rawLiveNotifications
2041:         ? rawLiveNotifications.filter((n) => {
2042:             if (n.documentId &&
!activeDocTrackings.has(n.documentId.trim().toUpperCase())) return false;
2043:             if (n.trackingNumber &&
!activeDocTrackings.has(n.trackingNumber.trim().toUpperCase())) return false;
2044:             return true;
2045:           })
2046:         : null;
2047:       if (liveNotifications) {
2048:         const liveDocKeys = new Set(
2049:           [...liveNotifications, ...storedNotifications].flatMap((n) =>
2050:             [n.documentId, n.trackingNumber].filter(Boolean),
2051:           ),
2052:         );
2053:         const filteredGlobal = dedupedGlobalNotifications.filter(
2054:           (notification) =>
2055:             (!notification.documentId ||
2056:               !liveDocKeys.has(notification.documentId)) &&
2057:             (!notification.trackingNumber ||
2058:               !liveDocKeys.has(notification.trackingNumber)),
2059:         );
2060:         return res.json([
2061:           ...filteredGlobal,
2062:           ...storedNotifications,
2063:           ...liveNotifications.filter(
2064:             (notification) =>
2065:               !storedNotifications.some(
2066:                 (stored) => stored.id === notification.id,
2067:               ),
2068:           ),
2069:         ]);
```

2070: }

Appendix F. Curriculum Vitae



Address: Purok 1, Mungo, Tuao, Cagayan
Email Address: catraltom@gmail.com

TOM TOM G. CATRAL

EDUCATION BACKGROUND

Elementary:

SACPIL ELEMENTARY SCHOOL
CONNER, APAYAO
2022 - 2008

Secondary:

SAINT TERESITA'S HIGH SCHOOL
PINUKPUK, KALINGA
2008 - 2012

Tertiary:

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY (BSIT)
CAGAYAN STATE UNIVERSITY
CARIG CAMPUS, TUGUEGARAO CITY, CAGAYAN
2012 – 2016

Graduate School:

MASTER IN PUBLIC ADMINISTRATION
APAYAO STATE COLLEGE
MALAMA, CONNER, APAYAO
2021 - 2024

ELIGIBILITY

LICENSURE EXAMINATION FOR TEACHER
Date of examination: January 29, 2022

EXPERIENCES

COMPUTER MAINTENANCE TECHNOLOGIST I
Provincial Government of Cagayan
Tuguegarao City, Cagayan
2023 to present

HUMAN RESOURCES MANAGEMENT AIDE
Bureau of Local Government Finance (BLGF)
Tuguegarao City, Cagayan
2020 - 2023

INFORMATION SYSTEM ANALYST I
Bureau of Local Government Finance (BLGF)
Tuguegarao City, Cagayan
2018 – 2019

GEOGRAPHIC INFORMATION SYSTEM/ MANAGEMENT INFORMATION SYSTEM ANALYST
Department of Environment and Natural Resources (DENR)
San Isidro, Luna, Apayao
2017

SURVEY AIDE
Department of Environment and Natural Resources (DENR)
Buluan, Conner, Apayao
2016